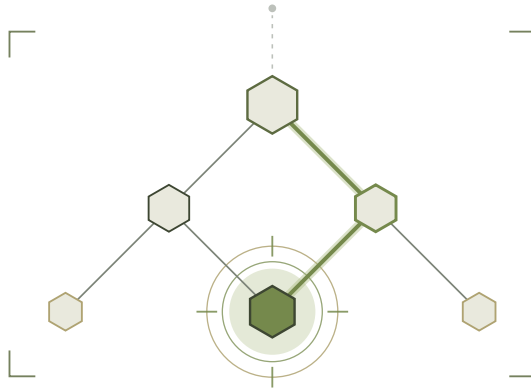




THE DURABLE7 FIELD GUIDE

Persistent Data Structures

Nine languages, one contract,
forty-odd immutable collections

A complete tour of the collections in the
Durable7 repository — what each one is,
why it exists, and when to reach for it.

Durable7: The Persistent Data Structure Field Guide

This document describes the collections implemented in the Durable7 repository. It is a companion to — not a replacement for — the workspace API specifications and public headers, which remain the normative source for contracts, complexity, allocation behavior, and validation evidence. Where this guide and a workspace specification disagree, the specification wins and this guide has a bug.

The repository ports every family to **C**, **C++**, **C#**, **Haskell**, **Kotlin**, **OCaml**, **Python**, **Rust**, and **TypeScript**. Code samples are drawn from whichever port makes the point most clearly; every behavior described in the shared-contract sections holds in all nine unless the text says otherwise.

Set in Libertinus with Inconsolata for code; diagrams drawn in TIKZ; typeset with L^AT_EX.

Licensed **MIT No Attribution (MIT-0)**. Use it, fork it, ship it.

Contents

How to read this guide	xi
I Foundations	1
1 What Persistence Buys	3
1.1 Path copying, and the shape of an update	3
1.2 The version DAG	4
1.3 What it costs	5
1.4 Where mutation is allowed	6
1.5 Why nine implementations	6
2 Laziness, and the Price of Persistence	9
2.1 The credit that gets spent twice	10
2.2 Suspend, then memoize	10
2.3 Nine languages, nine publication primitives	11
2.4 The chain that eats the stack	13
2.5 The parity campaign	14
2.6 Pinning a claim that tests cannot see	16
3 The Shared Contract	19
3.1 Vocabulary	19
3.2 Policies are values, and they are retained	20
3.3 Representatives	21
3.4 No-op identity	21
3.5 Failure atomicity	21
3.6 Ordering, stated honestly	22
3.7 Ownership, nine ways	22

II	The Unordered World	25
4	CHAMP: The Hash Trie at the Center	27
4.1	From HAMT to CHAMP	28
4.2	Equality and diff without a lookup pass	29
4.3	Collisions	30
4.4	Single-descent factories	30
4.5	Sets, and what changes	31
4.6	Nine ports, one representation	32
5	Companions of the Trie	35
5.1	Persistent hash bag	35
5.2	Persistent bidirectional map	36
5.3	Set-valued multimap and relation	37
5.4	Map patch	38
5.5	Directed graph	39
5.6	Indexed map	39
5.7	Persistent chunked bit set	40
6	Patricia: When the Key Is Its Own Hash	41
6.1	Path compression and the branching bit	42
6.2	The reason it exists: structural merge	43
7	Ctrie: Snapshots Without Stopping the World	45
7.1	How a constant-time snapshot is possible	46
7.2	Crossing back into the persistent world	46
7.3	Why only two ports have the real thing	47
8	Builders, Sessions, and the Frozen Tier	49
8.1	Construction-only bulk builders	50
8.2	The one-way editing session	50
8.3	The frozen tier, and why it has not shipped	52
III	Sequences	55
9	Finger Trees	57
9.1	The deque	58

9.2	The measured tree	59
10	Measures: Monoids as Connective Tissue	61
10.1	The idea	61
10.2	The measures that ship	62
10.3	How the nine ports spell it	63
11	RRB Vectors	65
11.1	Regular and relaxed branches	66
11.2	The representation boundary, stated up front	66
11.3	Choosing between three sequences	67
12	Ropes and Text	69
12.1	Three layers, one machine	70
12.2	Builders	70
13	The Range-Update Sequence	71
13.1	The invariant that makes it work	72
13.2	The algebra, and its laws	73
13.3	In use	73
14	The Reversible Deque	75
15	DABA Lite: The Deliberate Outlier	77
15.1	Six cursors and a bounded fixup	78
15.2	The parts that differ by runtime	78
15.3	What it will not do	79
15.4	Space, and a validator that knows nothing	79
IV	Order and Priority	81
16	Sorted Collections	83
17	Canonical Sorted Sets: Zip and Zip-Zip Trees	85
17.1	Shape from contents	86
17.2	Getting ranks right	86
18	Insertion Order as a First-Class Property	89

18.1	Two indexes, one meaning	90
19	Priority: Three Structures, Three Reasons	93
19.1	The measured priority queue	93
19.2	The Brodal–Okasaki heap	94
19.3	The priority search queue	95
20	Intervals	97
V	Narrow Questions	99
21	Ancestry as a Sequence	101
21.1	A forest that only grows leaves	102
21.2	The ancestral slice queue	106
21.3	The bilateral ancestral deque	108
21.4	What the family costs	112
22	Lifting a Machine into a Measure	115
22.1	Why the obvious repairs are linear	116
22.2	The lift	116
22.3	Rank and select in one descent	118
22.4	What it costs	121
22.5	Why this chapter is the parity campaign’s best exhibit	122
22.6	Where the guarantees stop	123
22.7	On novelty, stated narrowly	124
23	Checkpoints and Deltas	125
23.1	The checkpoint-differential map	126
23.2	The run-delta vector	131
23.3	The reference-identity invariant	134
23.4	The relation that decides everything	135
23.5	Where the ports differ	137
24	The Monotone-Action Heap	139
24.1	The algebra, and the one law that carries the structure	140
24.2	Composition has a direction	141
24.3	Where the tags live	142

24.4	The clamp family, and why it is the interesting case	143
24.5	Temporal semantics: expose before you attach	144
24.6	Where the guarantees stop	145
25	The Ancestral Connection Forest	147
25.1	The mechanism: an edge remembers its own birthday	148
25.2	Union by size, and the ceiling it alone guarantees	150
25.3	What “branching” means here	150
25.4	The path factor, and a guarantee that was strengthened	151
25.5	Where the guarantees stop	152
VI	Authenticated Collections	153
26	The Merkle Search Tree	155
26.1	Shape from hashes, not from history	156
26.2	Two prerequisites, taken seriously	157
27	Blocks, Proofs, Synchronization, and Merge	159
27.1	Blocks and stores	159
27.2	Packs, and the transport boundary	160
27.3	Seven budgets	160
27.4	Proofs	161
27.5	Synchronization	161
27.6	Three-way merge	162
VII	Navigation	163
28	Cursors	165
28.1	Gap semantics	165
28.2	Which families have one	166
28.3	One focused representation, many checkpoints	167
VIII	Reference	171
A	Complexity at a Glance	173

B	The Same Structure in Nine Languages	179
C	Roads Not Taken	185
C.1	Rejected outright	185
C.2	Postponed, with a named gate	186
C.3	Shipped as documentation rather than as a type	187
C.4	Analyzed, found implementable, and declined	187
C.5	Regressions taken in the open	188
D	Further Reading	191
	Index	193

How to read this guide

There are two honest ways to use a book about data structures. You can read it straight through, which is what you want when you are trying to understand a design space. Or you can open it at the one page that answers today’s question, which is what you want at 4pm on a Thursday. This guide is arranged so that both work.

If you are reading straight through. Part I explains what persistence buys and what it costs, and sets out the vocabulary that the rest of the book leans on — policies, representatives, no-op identity, failure atomicity. Nothing after it will quite land without it. Parts II through VI then walk the families in the order a curious reader would want them: the hash trie first (it is the substrate for a third of everything else), then sequences, then ordered and priority collections, then the authenticated tier. Part VII covers cursors, which cut across every family. The appendices are reference material.

If you are looking something up. Every structure gets a *specimen card* — a compact panel with its purpose, its complexity, its policy requirements, and its nine-language spellings — followed by prose that explains the representation and the decisions inside it. Appendix A is a single sorted complexity table across the whole library. Appendix B is a name-to-name index across the nine ports. The subject index at the back covers both concepts and identifiers.

Three kinds of panel appear throughout.

The contract

Normative behavior that every port upholds. If a port cannot honor one of these, it says so explicitly in its own API notes rather than quietly differing.

Caveat

The place where a claim stops being true. These are the paragraphs worth reading twice — most production surprises with persistent structures live exactly at the

edge of an amortization argument or a policy assumption.

Aside

History, provenance, and the occasional road not taken. Skippable, but they are where the design reasoning usually hides.

A note on honesty. This library is unusually careful about the difference between a bound it can prove and a bound it would like to have. Several structures ship as *checkpoints*: the observable semantics match the reference port exactly, but the representation is simpler and the asymptotic claim is weaker. Wherever that is the case, this guide says so in the same breath as the feature. A guide that only reported the good news would be a worse guide.

PART I

Foundations

What Persistence Buys

The old version is still there. That one sentence is the whole library, and everything else is an argument about how to afford it.

Here is a small program, and a large idea hiding inside it.

```
var v1 = PersistentHashMap<int, string>.Empty.SetItem(1, "one");  
var v2 = v1.SetItem(2, "two");  
var v3 = v2.SetItem(1, "uno");
```

After three lines you are holding three maps. Not three references to one map that has been mutated three times — three genuinely independent collections, each of which will answer questions about its own contents forever. `v1` still has exactly one entry. `v2` still maps 1 to "one". You can hand `v1` to another thread while a third thread builds `v4` from `v3`, and no lock is involved anywhere, because nothing that either thread can see is ever written to again.

The obvious way to get that behavior is to copy the map on every update, and the obvious problem with the obvious way is that it costs $O(n)$ time and $O(n)$ space per edit. A library of persistent collections is, in essence, a long and inventive answer to the question: *how little can we copy and still tell the truth?*

1.1 Path copying, and the shape of an update

The answer, in one picture, is Figure 1.1. Update-shaped operations copy the nodes along the route from the root to the point of change and reuse every other node by reference. For a tree of branching factor b and n elements, that is $\log_b n$ new nodes rather than n — for a 32-way trie holding a million entries, four small nodes instead of a million entries.

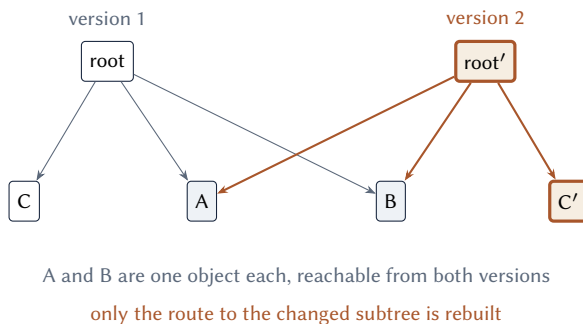


Figure 1.1: Path copying. Version 2 differs from version 1 in a single subtree. The two nodes on the route to that subtree are freshly allocated (copper); every other node is shared verbatim. Neither version can observe the other’s existence.

Two consequences follow immediately, and both matter more than they look.

Old versions cost almost nothing to keep. Retaining v1 does not pin a second copy of the data; it pins the handful of nodes that later versions replaced. A hundred snapshots of a slowly changing map are cheaper than one would guess and vastly cheaper than a hundred deep copies. This is what makes undo stacks, audit trails, and time-travel debugging practical rather than aspirational.

Readers need no synchronization. Every node reachable from a published root is immutable, so a reader that has captured a root is reading a frozen world. There is no torn read, no lock, no volatile fence in the read path, and no window during which a concurrent writer can make a traversal observe half of two different states. The synchronization problem does not get solved; it gets deleted.

1.2 The version DAG

Because an update produces a new value rather than modifying an old one, nothing forces versions into a line. Branch from v2 twice and you get two descendants that share v2’s structure and know nothing about each other (Figure 1.2). This is why persistent collections are the natural substrate for editors, transactional stores, speculative execution, and any system where “what if” has to be cheap.

That freedom has a price, and it is the single most important caveat in this book.

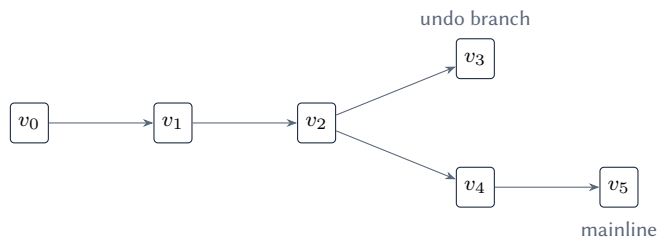


Figure 1.2: Versions form a directed acyclic graph, not a sequence. Every node in the picture is a live, complete collection.

Amortization does not survive branching

Amortized bounds are proved by a potential argument over *one linear history*: an expensive operation is paid for by cheap operations that preceded it. Branching breaks the argument. If a version sits just before an expensive rebalance, and you branch from it b times, each branch can trigger that same rebalance, and the accumulated credit pays for exactly one of them. Potential consumed by one child cannot pay for a sibling.

Durable7 therefore classifies every adaptive scheme before believing its amortized claim. Constant-size promotion thresholds are safe — re-branching re-pays only $O(1)$. Schemes that do $O(n)$ work at a threshold — relabeling, compaction, wholesale redistribution — are not, and the library either avoids them or states the worst case per produced version. Where a bound holds only along a linear lineage, this guide says “linear lineage” explicitly.

1.3 What it costs

Persistent collections are not free, and pretending otherwise makes for a bad guide. The honest ledger looks like this.

- **Allocation per update.** Every edit allocates its copied path. A tight loop of point updates on a persistent map will allocate far more than the same loop on a hash table. This is the motivation for the entire builder and editing-session tier (Chapter 8).
- **Indirection on reads.** A 32-way trie lookup is a handful of pointer hops with good branch prediction, but it is still pointer hops. A flat open-addressed table

wins on a hot small map, every time.

- ▶ **Cache behavior.** Node-based structures scatter. The library’s answer is chunking — ropes hold runs of elements in contiguous leaves, RRB vectors hold 32-element arrays, chunked bit sets hold 64-bit words — so that scans get array speed even though edits get tree behavior.
- ▶ **Small collections lose.** The regime in which a persistent map looks worst is the one most programs spend most of their time in: maps with three entries. The library records a size-tiered representation as a strong idea that is deliberately *not* shipped, because it has not yet been justified with benchmarks (Appendix C).

1.4 Where mutation is allowed

A library that is honest about persistence has to be equally honest about its exceptions. Durable7 confines mutation to four clearly labeled places, and every one of them is documented as mutable wherever it appears:

Construction-only bulk builders mutate unpublished trie nodes while building a collection from a sequence, then freeze. Nothing observable is ever mutated, because the nodes are not reachable until the freeze.

One-way editing sessions adopt a persistent root, accept a run of edits under single ownership, and publish once. Publication consumes the session.

The concurrent hash trie is a genuinely mutable lock-free map whose distinguishing feature is that `Snapshot()` is $O(1)$.

DabaLite is a mutable sliding-window aggregator. It is in the library because it reuses the monoid vocabulary, not because it is persistent; it explicitly is not.

1.5 Why nine implementations

Most collection libraries exist once, and a “port” is a family resemblance. Here, semantic parity is the product. The same ordering rules, the same first-representative retention, the same no-op identity, the same failure atomicity, and — for the authenticated tier — the same bytes on the wire, in C, C++, C#, Haskell, Kotlin, OCaml, Python, Rust, and TypeScript.

The interesting part is what *does not* have to match. Rust uses `Arc` and forbids `unsafe`; C# exposes `ReadOnlyList<T>`; OCaml returns `option` and `result`; C is type-erased with explicit ownership and fallible allocators; Haskell keeps its sessions in `IO`. Idioms

are local. Semantics are not.

The checkpoint idea

Nine simultaneous implementations of a subtle algorithm would normally mean nine opportunities to quietly claim a bound nobody proved. Durable7's device for avoiding that is the *checkpoint port*: a port that preserves the observable API semantics and passes the same behavioral tests, while documenting exactly which representation or asymptotic property it does *not* yet have. Most of the newer cursors are checkpoints: the multimap's cursor resolves a rank by walking the grouped flattening, so it is linear where the collection it navigates is logarithmic, and its specification says so in the same table that gives every other row. Saying so in the API notes is treated as part of shipping, not as an admission of failure — and a checkpoint is a debt with a name on it rather than a permanent concession. Chapter 2 follows one such debt, carried by five ports at once, to the point where it was paid.

Laziness, and the Price of Persistence

Amortization is credit spent by the operations that come after. Persistence lets a caller spend the same credit again, and again, out of the same version. Memoization is what closes the account.

Specimen — the memoized suspension

Purpose	Make an amortized bound survive persistent branching, by deferring cascading work and charging every version that shares it at most once.
Core	A one-slot cell holding either a pending operation or its result. Forcing runs the operation and publishes the result back into the cell.
Operations	A closed set of deferred spine steps: push an overflow node into the middle from the front, push one from the back, concatenate two middles around regrouped nodes, and — where reversal is a view — mirror a middle.
Complexity	What it buys the finger tree: $O(1)$ amortized endpoints and $O(\log \min(n, m))$ concatenation that stay true <i>under branching</i> . Without it both collapse to $O(\log n)$.
Mutability	One write per cell, ever, and only after the deferred computation has succeeded.
Ports	All nine, each with its own publication primitive (Section 2.3). Haskell gets it from the language; the other eight build it by hand.
Limits	Pending chains reach $\Theta(n)$ links. Forcing them — and releasing them — must be iterative, never recursive.

Chapter 1 closed its tour of the version DAG with a warning: amortized bounds are

proved over one linear history, and branching breaks the proof. That warning was abstract, a rule for classifying adaptive schemes. This chapter is what it looks like made concrete, because the structure it threatens is the one a third of this library is built on.

2.1 The credit that gets spent twice

A finger tree earns its cheap ends from its digits (Chapter 9). Pushing onto a digit that has room is $O(1)$: the digit widens and nothing else moves. Pushing onto a *full* digit is not. Four items are regrouped into a 2-3 node and pushed down into the middle, where they may meet another full digit and cascade again, as far as the spine goes. Classic amortized analysis absorbs this the way it absorbs a doubling array: the cheap pushes between cascades deposit credit, the rare cascade spends it, the average is constant. The argument is correct — for a structure with one past.

Now keep an old version. Let v sit exactly one push away from a cascade k levels deep, and let a caller retain v and push from it b times. Each push produces its own successor and each successor pays the whole cascade, because the results are not shared — only the input is. The aggregate is $\Theta(bk)$ where the bound promised $\Theta(b)$: credit deposited once, before v , and spent b times. Nothing about that program is adversarial. It is an undo stack retrying from a checkpoint, a speculative evaluator forking a candidate state a few dozen ways, an editor keeping every keystroke — the workloads persistent collections exist to serve. A naive persistent finger tree does not deliver the $O(1)$ ends it is famous for. It delivers them to the last version anyone touched.

2.2 Suspend, then memoize

Hinze and Paterson’s answer (*JFP*, 2006) is not to make the cascade cheaper. It is to not do it yet, and then to arrange that it is done at most once. When a digit overflows, the overflowing middle is not rebuilt; it is stored as an unevaluated computation — a *suspension*. When some later operation actually needs that middle, the computation runs and the result is written back into the very cell that held it. Every version whose spine passes through that node holds a reference to that one cell, so the first branch to force it pays and every other branch reads the published result.

That is worth stating as a thesis rather than as a technique: **laziness here is not an optimization; it is the correctness condition of the amortization argument.** A strict finger tree is not a slower finger tree; it is a different data structure with different bounds. Okasaki makes the general form of the argument in *Purely Functional*

Data Structures: amortization and persistence are reconciled by lazy evaluation with memoization and by essentially nothing else, and the worst-case variants of the same structures are then obtained by *scheduling* the forced work, not by removing the laziness.

Two restrictions keep this from turning the tree into a cloud of thunks. Only the middle subtree of each deep node is suspended, so a tree of n elements carries $\Theta(\log n)$ suspensions rather than one per element; and within them only the monoid’s `combine` is ever deferred, because element measures are computed eagerly at insertion and every node’s size is strict at every construction site. That second restriction is what keeps the promises of Chapter 10 honest: reading the first element, the last element, or the length invokes no policy callback at all, on any port, at any size.

What a suspension cell owes

A cell is written at most once, and only *after* the deferred computation has succeeded. A measure policy that throws or raises mid-force therefore publishes nothing: the cell stays pending, the force may be retried, and every retained version remains valid — the failure atomicity of Chapter 3, extended into the laziness machinery.

Pending operations are pure, so running one twice yields structurally identical results. That is what makes a concurrent force benign: two threads may duplicate the bounded computation, but they converge on one published value, and no reader observes a partially initialized node or two different element sequences.

2.3 Nine languages, nine publication primitives

In Haskell the compiler supplies all of this: a thunk is a memoized suspension, and the tree is written the way the paper writes it. In the other eight workspaces the cell has to be built by hand, and the interesting question is not whether a strict language can do it — it is what each language’s atomic publication primitive turns out to be.

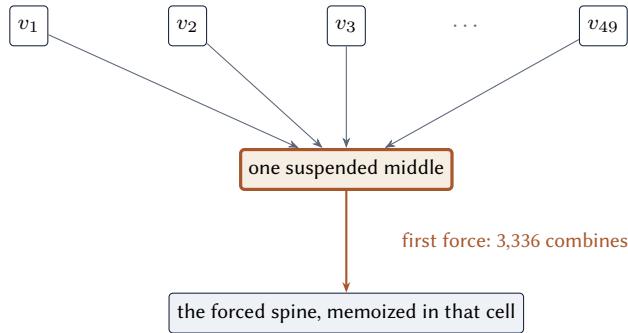
Language	Suspension cell	Note
C#	One field holding either the pending operation or the tree, published by an <code>Interlocked</code> compare-exchange	The reference. A pending <i>push</i> reports its measure without forcing; a pending <i>pop</i> cannot, so the node’s measure is memoized separately.

C++	An atomic box over <code>shared_ptr</code> , published by compare-exchange	Cells force themselves without external locking.
C	Hand-rolled memoized middle cells over reference counts	The existence proof that this needs no runtime support at all.
Haskell	The language	Native laziness; nothing to build.
Rust	Arc plus <code>OnceLock</code>	A lost race discards the duplicate result. Also needs an iterative <code>Drop</code> .
Kotlin	An <code>AtomicReference</code> over sealed operation classes, published by <code>compareAndSet</code>	The JVM collector handles teardown.
OCaml	One mutable field, written under the runtime lock	Native <code>Lazy.t</code> deliberately not used — see below.
TypeScript	Plain field assignment	The one single-threaded workspace, where assignment simply <i>is</i> the atomic publication — documented as such rather than left looking like an oversight.
Python	A thunk slot rewritten to its value under the GIL	

The four above the rule always had this; the five below received it in the campaign of Section 2.5. All nine publish identically: the result enters the cell only once the deferred computation has returned.

The one mutable cell in an immutable structure

Chapter 1 said a reader holding a root is reading a frozen world, with no write in the read path. In the finger-tree spine that is very nearly, but not exactly, true: the first read that needs a suspended middle both computes and *stores*. What survives is the part consumers depend on — no torn read, no lock, no observable state change, no traversal that sees half of two versions. What does not survive is the literal claim of a write-free read path, which is why eight of the nine ports reach for an atomic publication primitive rather than a plain store. The user-visible consequence is stated wherever it applies: reading a whole tree's measure is $O(1)$ *amortized*, not worst case, because the first such read on a freshly edited spine may force a chain of suspensions. Endpoint reads and length never force.



Each of the other 48 versions then pays 5 — only for the digits it does not share. Without memoization every one of them replays the spine.

Figure 2.1: Memoization under branching. Forty-nine versions retain the same unforced spine. The suspension is a cell they share, not a computation each of them owns, so the deferred work happens once and is read forty-eight times. These are the numbers the library’s probes assert.

2.4 The chain that eats the stack

Now run an ordinary loop: append n elements and read nothing. Each overflow suspends a push into a middle that is itself an unforced suspension, holding a pending push into another. After the loop the spine carries a chain of pending operations $\Theta(n)$ links long, none of it forced.

The natural implementation of a suspension is a closure capturing the next cell, and forcing it is a call — one stack frame per link. This is where a technique that is correct on paper meets a machine, and the library learned it three times in three different ways.

Python made it immediate. A default limit of a thousand frames turns a few thousand appends into a stack overflow, which at least has the virtue of failing in the first test run rather than in production.

OCaml made it instructive. “The language already has laziness” looks like an exemption from the whole problem, and it is not: a direct experiment, run before the OCaml core was designed, established that native `Lazy.force` itself overflows on a chain 200,000 deep. `Lazy.t` was then banned from that workspace’s spine — a port declining a language feature precisely because it needs the property the feature is named after.

Rust contributed a variant nobody planned for. A chain that is never forced still has to be *released*. Under reference counting, dropping the last handle to the head cell runs the destructor of its pending payload, which drops the next cell, which drops the

next: one frame per link again, on the teardown path this time. A tracing collector hides this completely; reference counting does not. Rust’s suspension therefore implements an iterative Drop that unlinks the chain with `Arc::into_inner`.

The remedy is the same in every case, and older than any of them: *defunctionalize* the suspension. Instead of a closure, store the deferred operation as *data* — a sealed class, an enum, a variant type — drawn from a small closed set: push an overflow node into the middle from the front, push one from the back, concatenate two middles around already-regrouped nodes, and, in the three workspaces where whole-value reversal is a lazy structural mirror (Chapter 14), mirror a middle. Forcing is then an explicit-stack interpreter over that data, and depth costs heap rather than frames. Every rebuilt core forces a 200,000-element organically built chain in its probe suite on every build; Rust drops one, unforced, as well.

How deep a suspension may nest is itself a design choice, and the reference answers it differently. C# forces the captured source *when the pending operation is created* — the paper’s own remark about forcing the middle subtree in the recursive case of cons to avoid building a chain of suspensions — so a C# cell is exactly one deferred step deep and the deepest chain a force can meet is the height of the tree. Both realizations deliver the same asymptotic bounds; they trade a bounded eager force at creation against an unbounded lazy one later. One gap was recorded rather than papered over: the three strict donor cores were never stress-tested against the same 200,000-deep force-and-release the five rebuilt cores now run, and that probe is filed as open.

2.5 The parity campaign

For most of this library’s life, everything above described four workspaces.

The discovery began with a small question during a port review: why did one Python collection document a $\Theta(s \log n)$ endpoint update where the C# reference said $O(s)$ amortized? The honest answer — because Python’s substrate was not a finger tree — prompted a survey of what each workspace’s substrate really was, and the survey immediately found worse. Rust’s core module described itself as a 2-3 finger tree, documented $O(1)$ front and back and $O(\log \min)$ concatenation, and stored a node of `{ left, right, len, height, measure }` — a plain height-balanced binary tree — whose front recursed the entire left spine. The documentation was not conservative. It was false.

Nine censuses then ran in parallel, one per workspace, reading node representations and delivered bounds *from the code* and treating every documented bound as a claim to

check. Four workspaces — C#, C, C++, Haskell — carried the real lazy Hinze–Paterson structure. Five carried height-balanced binary *join trees* instead: AVL or weight-balanced, one element per node, a measure cached at each node. A join tree delivers the same $O(\log n)$ interior operations and fundamentally cannot deliver $O(1)$ ends or $O(\log \min)$ concatenation, having neither digits nor laziness. Three of the five nevertheless *called* themselves finger trees, and two documented bounds their code did not deliver.

The parity principle

Every language uses the same algorithms and the same data structures and offers the same complexity guarantees, wherever that is possible at all. No fix may make any delivered bound asymptotically worse. When one workspace delivers a *stronger* guarantee than the reference, that guarantee becomes the target for every workspace where it is achievable. And where a bound depends on laziness, strict languages implement memoized suspensions by hand — which C#, C++, and C had already proven feasible here, including in plain C.

All five cores were rebuilt in place: same module, same public API, checked mechanically against the previous export lists, so every facade above them — dequeues, sorted collections, priority queues, interval trees, ropes, chunked bit sets — inherited the new bounds without a source change, and their existing behavioral tests served as free regression harnesses.

Operation	Join-tree cores, before	All nine cores, now
first / last read	$\Theta(\log n)$ spine descent (Rust’s docs claimed $O(1)$)	$O(1)$ worst case — a digit field read
endpoint push / pop	$\Theta(\log n)$, worst <i>and</i> amortized: joining against a singleton rebuilds the spine	$O(1)$ amortized, valid under persistent branching; $O(\log n)$ worst case per call
concatenation	$O(\text{height difference})$, so appending a short run to a long sequence paid the long sequence’s height	$O(\log \min(n, m))$ amortized — the suspended middle recursion stops at the shallower operand
split / index / locate	$O(\log n)$	$O(\log n)$, unchanged — the no-regression rule

whole-value Reverse	$\Theta(n)$ rebuild through an array (Python, TypeScript, OCaml)	$O(1)$ — a lazy structural mirror, the fourth deferred operation
mixed-orientation concat	$\Theta(n + m)$ re-materialization (same three)	$O(\log \min)$, structural: mirror the smaller operand lazily

The “wherever possible at all” clause was exercised twice, and both rulings were written down so a later maintainer can revisit them with the original reasoning in hand. One workspace’s placeholder collections stored each version as a flat sorted array — $O(1)$ rank-select, $\Theta(n)$ writes; since no persistent structure is known that offers $O(\log n)$ writes *and* $O(1)$ select, rank-select regressed to the reference’s $O(\log n)$, and every affected interface says so. Separately, an append-only node arena shared by two of the newer collections has a $\Theta(\sqrt{M})$ allocation spike at block boundaries; removing it would have cost permanent complexity in eight hot paths, so the spike was kept and all seven arena ports made to document it identically rather than let one quietly differ. A no-regression rule needs an escape valve with a paper trail. These are it.

Equalizing *upward* fired in a direction nobody planned. Python’s first lazy Reverse reused each mirrored node’s cached measure, which is correct only when combine commutes, and documented that as a caller obligation; the TypeScript port instead recombined each mirrored summary in mirrored order — constant extra work per node, paid only when the reversal forces, correct under *every* monoid. That is strictly stronger, so Python was rewritten to match the same day and OCaml was required to have the stronger shape from birth.

2.6 Pinning a claim that tests cannot see

The most transferable result of the whole campaign is one sentence: **asymptotic claims are invisible to correctness tests**. A join tree passes every behavioral test a finger tree passes. It returns the same elements in the same order, preserves the same representatives, honors the same no-op identities, fails atomically in the same places. Every suite in all five workspaces was green on the wrong data structure.

So the library pins bounds the only way they can be pinned: with *counting probes*. Every policy object a structure takes — comparator, measure, tag algebra — has a counting variant, and the suites assert against the counters rather than against a stopwatch: an exact call count where the number is deterministic, and a size-independence equality plus a tight budget where it is not. What fails is then arithmetic, not intuition. The

signature result of the campaign is that the five rebuilt cores produce *byte-identical* numbers.

Probe	Pinned result
512 endpoint pushes (256 at each end) onto a tree of 1,024, and onto one of 32,768	exactly 340 combine calls — <i>the same number at both sizes</i> , which is the amortized- $O(1)$ claim written as an equality
Concatenating a 4,096-element tree with a singleton, and with a 1,024-element operand	0 immediate combines, and 4 — cost tracking the smaller operand, not the height difference
A 10,000-append spine branched into 49 retained versions	3,336 combines when the first branch forces it; exactly 5 per branch for the other 48
front, back, length	zero policy calls, at any size

The third row is Figure 2.1: the Hinze–Paterson argument turned into an observable quantity. Five languages, five different memoization primitives — OnceLock, compare-and-set, plain assignment, a mutable field, a GIL-protected slot — one algorithm, one fingerprint.

A probe that has never been seen to fail proves nothing, so every load-bearing mechanism was deliberately broken, shown failing with a characteristic signature, and restored. The one that matters most here is the reversed view: its correctness under a non-commutative monoid is checked with a *spelling* measure — each element measures to its own string, combine is concatenation — against the mirrored root summary, both halves of an interior split of the mirror, and a double reversal. The cached-measure implementation fails that probe; only reversed-order recombination passes it. And where the obvious counter cannot see a fix, the discipline is to measure something that can: replacing a flat-array sorted family is invisible to a comparator counter, because the array’s binary search was already logarithmic and the $\Theta(n)$ write copies perform no comparisons at all, so that probe pins bytes allocated per write instead.

What the fingerprint does not prove

Identical counts prove that the same algorithm is running in nine places. They do not prove it is the right algorithm, and they say nothing about constant factors. Representation differences that are not asymptotic are deliberately out of scope: ropes hold chunked leaves in some ports and one element per leaf in others,

which is a real performance difference and not a parity violation. The rule the library enforces is about the exponents.

The durable artifact of all this is not any individual fix. It is the pinned number. A future substrate change that quietly breaks parity — a well-meaning strictness annotation, a suspension replaced by an eager call, a memoization cell that stops being shared — no longer produces a subtly slower library that passes its tests. It produces a failing equality, in nine workspaces, with the expected value printed next to the observed one.

The Shared Contract

A contract that only one implementation upholds is a comment.

Before the structures, the rules they all obey. This chapter is short, dense, and load-bearing: nearly every specimen card later in the book is an application of something defined here.

3.1 Vocabulary

Term	Meaning in this library
Persistent value	An operation that looks like an update returns a new value or handle and preserves every retained old version.
One-way editing session	A single-owner edit-then-publish lifecycle. It is unsynchronized, consumed by publication, and is <i>not</i> a reusable staging builder.
Structural sharing	Versions reuse immutable substructure; no-op updates often preserve the same root or instance where the API exposes that diagnostic.
Reference-first equality	Two provably identical references may be accepted as equal without invoking user equality. A <i>non</i> -identical reference never proves inequality.
Policy preservation	Hash, equality, comparison, measure, allocator, ownership, and callback policies flow into derived versions unless an API explicitly creates a new policy.
Stable but unspecified order	Enumeration is deterministic for an unchanged version, but is not sorted, not insertion order, and not a serialization contract.

Insertion / explicit-position order	Enumeration follows first insertion plus unmistakable positional operations. Equality decides membership; it does not decide order.
Measure	A monoidal summary cached on a node, chunk, or facade element, used for split, locate, rank, priority, interval, and text navigation.
Tag action	A tag monoid acting consistently on elements and on cached ordered measures; composition order and distribution laws are part of the public policy contract.
Version-bound cursor	An immutable value owning one collection version and a position within it. Navigation and editing never silently redirect it to a different version.
Checkpoint port	A port that preserves observable semantics and tests while documenting a remaining representation or asymptotic parity boundary.
Facade	A public collection built on a shared core engine — sorted sets on measured trees, text ropes on measured ropes.

3.2 Policies are values, and they are retained

Almost every collection in this library carries a *policy*: an equality and hash pair, a comparison, a measure monoid, a tag algebra, a set of ownership callbacks, a codec, an allocator. The policy is not a construction-time convenience that gets forgotten — it is part of the collection’s identity and flows into every successor, *including empty ones*. Clear a map and the result is an empty map with the same comparer, not a default one.

Two rules follow that catch people out.

The receiver’s policy wins. When you union two sets, the result uses the receiver’s equality policy. The argument is normalized under that policy first — eagerly and completely, before any shortcut is applied — so that a policy or enumeration failure in the argument is visible rather than hidden behind an optimization that happened to skip it.

Policy identity gates structural shortcuts. Several structures can compare or merge in lockstep, interpreting stored hashes structurally instead of performing lookups, but only when the implementation has a positive witness that both operands retain *one* hash-policy identity. Two independently created but semantically compatible policies are not enough: their coherent hash states can differ. In that case the implementation falls back to lookup-based comparison, or rejects the operation explicitly, and says which it does.

3.3 Representatives

When two distinct objects are equal under the policy, exactly one of them ends up stored. Which one is not left to chance.

First representative, last payload

Construction and duplicate insertion retain the *first* stored key representative. Replacing the value for an equivalent key preserves the originally stored key object and installs the new value — so a map is “first key wins, last value wins”. Set algebra preserves receiver representatives for surviving receiver classes, and the first normalized argument representative for argument-only classes.

This sounds pedantic until the stored objects carry information that equality ignores — a cached parse, a source location, a case-preserved spelling under a case-insensitive comparer. Then it is the difference between a correct program and a mystery.

3.4 No-op identity

Operations that logically change nothing return the receiver. Removing an absent key, clearing an empty collection, adding an item a set already has, replacing a value with one that compares equal, applying an identity tag, seeking to the position you are already at: all of these hand back the same instance, and where a language has reference identity, the same reference.

This is more than an optimization. It is an observable, tested property that downstream code uses to short-circuit change propagation — a UI that re-renders when the model reference changes, an incremental compiler that skips a subtree whose table did not move. Because it is observable, the library treats it as contract and tests it, rather than as a happy accident of the implementation.

3.5 Failure atomicity

User code runs inside these structures: comparers, hash functions, measure policies, tag algebras, codecs, factory delegates, allocators. Any of it can fail.

Nothing partial is ever published

If a comparison throws, an allocation fails, a codec rejects input, or a factory raises, the operation publishes no successor. The receiver remains valid, every retained version remains valid, and no composite is left with one index updated and another not. Composites — bimap, relations, indexed maps, interval maps, graphs — publish *all* of their constituent indexes together or publish nothing at all.

Validation order is part of the contract too, and it runs in a deliberate sequence: argument validation precedes policy callbacks, range checks precede tag application, proof-shape limits precede allocation and hashing. The reason is that callbacks are the expensive and dangerous part; a malformed request should be rejected before any of it runs.

3.6 Ordering, stated honestly

Three different order contracts appear in this library, and conflating them is the most common way to write code that breaks on an unrelated change.

Contract	Who has it, and what you may rely on
Stable but unspecified	CHAMP maps, sets, bags, bimap, multimaps, relations. Deterministic for an unchanged version; <i>not</i> sorted, <i>not</i> insertion order, and never a serialization format. Tests exist specifically to stop consumers depending on the current trie order.
Insertion / explicit position	The neutral Ordered family. First insertion plus explicit moves, ranges, reversal, and one-shot stable sort. Equality decides membership, never position.
Comparer order	Sorted collections, canonical sets, priority search queues, interval structures, Patricia tries (ascending signed key order), Merkle trees (policy comparer order). Fully specified and safe to depend on.

3.7 Ownership, nine ways

The same semantic contract lands in nine memory models. This table is the translation key for the rest of the book.

Language	Ownership model, and what its docs must pin down
C#	GC references, immutable collection values, single-owner transient sessions. Exceptions, nullable/miss behavior, structural sharing, builder publication versus one-way transient consumption.
C	Opaque handles with explicit copy/destroy, callback policy tables, injected allocators. Status codes, allocation failure, retained versus borrowed values, alias-wide consumption, callback lifetime.
C++	RAII values over shared immutable nodes; move-only sessions. Move/copy cost, exception behavior including throwing policy moves, policy object lifetime.
Haskell	Pure immutable values plus explicitly scoped IO sessions. Total versus Maybe operations, session consumption, package-local classes, strictness.
Kotlin	Immutable JVM values, runtime policies, dynamically consumed sessions. Null/result/exception shape, version-bound views, comparator behavior.
OCaml	Immutable algebraic values, runtime policy records, dynamically consumed sessions. <code>option/result</code> boundaries, policy identity, Unicode-scalar text positions, mutex-coordinated facades.
Rust	Owned values, borrows, Arc sharing, consuming sessions. Clone requirements, borrowed lookup results, panic boundaries, <code>Send/Sync</code> claims only when proven.
TypeScript	GC objects, immutable versions, runtime policies, dynamically consumed sessions. ESM exports, undefined/miss shapes, stable hashing versus JavaScript identity, isolate-local concurrency, UTF-16 indexing.
Python	GC objects, immutable versions, retained policies, dynamically consumed sessions. Typing and runtime validation, <code>None</code> versus absence, equality/hash coherence, iterator invalidation, code-point indexing.

Presence versus null

Every port has to distinguish “the key is absent” from “the key is present and its value is null-like”. The library never collapses the two. C# uses `TryGetValue` out-parameters; Rust and OCaml use `Option`; Haskell uses an outer `Maybe` (so a stored `Nothing` at element type `Maybe a` comes back as `Just Nothing`); Kotlin, TypeScript, and Python use explicit presence-bearing result wrappers rather than a bare nullable return. The authenticated three-way merge takes this furthest: it distinguishes a deleted key from a key whose merged value is present-and-null, because on the wire those are different facts.

PART II

The Unordered World

CHAMP: The Hash Trie at the Center

Every third structure in this library is a hash trie wearing a different hat.

Specimen — persistent hash map and set

Purpose	Unordered key/value and membership collections with structural sharing, policy-defined equality, and structural equality/diff.
Core	CHAMP: a 32-way bitmap-indexed prefix tree with split data/node bitmaps and canonical deletion topology.
Lookup	$O(\log_{32} n)$ — at most seven levels for 32-bit hashes, plus bucket work on a full-hash collision.
Update	$O(\log_{32} n)$ path copy.
Equality/diff	Lockstep slot alignment with reference pruning where policy identity is witnessed; $O(n + m)$ for independently built operands.
Order	Stable but unspecified. Never sorted, never insertion order.
Ports	PersistentHashMap/PersistentHashSet (C#, Kotlin, Rust, TypeScript, Python), persistent_hash_map/_set (C++), HashMap/HashSet (Haskell), d7_hamt_map/d7_hamt_set (C), Persistent_hamt / Persistent_hash_set (OCaml).

Start here, because a surprising amount of the library is downstream of this one structure. The hash bag, the bidirectional map, the set-valued multimap, the relation, the map patch, the directed graph, the indexed map, and the insertion-ordered set are all — in whole or in part — compositions over the persistent hash map. Understand its representation and you have understood the substrate for a third of the collections in

this book.

4.1 From HAMT to CHAMP

Bagwell’s hash array mapped trie is the classic design: consume five bits of the hash per level, use a 32-bit bitmap to record which of the 32 slots are occupied, and store only the occupied slots in a compact array. Population count on the bitmap below the slot’s bit gives the array index. Lookup is a handful of hops; the tree is shallow because $32^7 > 2^{32}$.

CHAMP — the Compressed Hash-Array Mapped Prefix-tree, from Steindorfer and Vinju’s OOPSLA 2015 paper — keeps that skeleton and changes two things.

4.1.1 The bitmap split

A CHAMP node keeps *two* bitmaps: a datamap recording which slots hold an inline key/value payload, and a nodemap recording which slots hold a pointer to a sub-node. The single backing array is then divided into two regions: payloads packed at the front, sub-node references packed at the back (Figure 4.1).

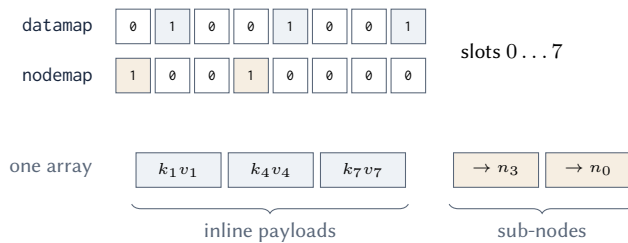


Figure 4.1: A CHAMP node. Two bitmaps classify each of the 32 logical slots as empty, inline payload, or sub-node. The payloads live contiguously at the front of one array and the sub-node references at the back, so iteration over entries never has to step past pointers.

The immediate win is layout. Iteration touches payload runs contiguously instead of interleaving pointers and payloads, and a lookup that lands on a data slot reads the entry directly rather than dereferencing a leaf object. The deeper win is the second change.

4.1.2 Canonical deletion topology

In a plain HAMT, deletion leaves scar tissue. A sub-node that shrinks to a single entry stays a sub-node, so the tree’s shape encodes its update history: two maps with identical

contents can have different skeletons depending on what was inserted and removed along the way.

CHAMP normalizes on the way out. When a sub-node shrinks to a single payload, it is collapsed into its parent’s data region; when a wrapper node ends up holding nothing but a single child, the wrapper is removed (Figure 4.2). Bitmap topology is therefore determined by the hashes that remain, not by the order in which they arrived.



Figure 4.2: Canonical deletion. A sub-node that would be left holding one payload is promoted into its parent, so shape depends on contents rather than on history.

That canonicalization is what makes the interesting operations possible.

4.2 Equality and diff without a lookup pass

If two maps with equal contents have the same skeleton, then comparing them does not require looking every key of one up in the other. You can walk both trees in lockstep, aligning logical bitmap slots, and compare structure directly. Better: when two versions share ancestry — which they will, constantly, in a persistent library — entire subtrees are *reference-identical*, and a reference comparison prunes them without touching a single key.

Structural equality and diff

Where the implementation has a positive witness that both operands retain one hash-policy identity, `MapEquals` and `Diff` traverse aligned logical slots using stored hashes and prune every reference-identical descendant. Work then tracks the visited non-shared regions plus the reported differences. Independently created but compatible policies fall back to lookup-based comparison, because their coherent hash states may differ.

Identical topology is not reference identity

Two maps built independently — same entries, same policy, different allocations — share nothing. Comparing them is an honest $O(n + m)$ traversal, and CHAMP by itself does *not* guarantee work proportional to logical divergence. The benchmark suite deliberately separates “shared ancestry, one change” from “independent histories, equal contents”, because those two cost profiles are different and a single number would hide it.

Collision buckets add their own asymmetry. Their internal order is insertion order, so equality has to match keys as an unordered set, costing up to $O(c^2)$ comparisons for a bucket of size c . Equal maps may also retain different concrete key objects, because “first representative wins” is per-history.

Structural algebra — union, intersection, difference over two same-type maps or sets — uses the same machinery: align slots, prune shared descendants, and build the result bottom-up.

4.3 Collisions

Two keys whose *entire* 32-bit hash is equal cannot be separated by any number of levels. CHAMP handles this with an immutable collision bucket: a small array of entries that share a hash, searched linearly under the equality policy. This is strictly better than the alternative of extending the trie past the hash — there is nothing left to extend it with — and it degrades gracefully, because a pathological hash function turns the map into an association list rather than into an infinite loop.

Unary bitmap chains are a related and often-misunderstood shape. Even under canonical deletion, a node can legitimately have exactly one child: two keys whose hashes agree on the first several five-bit groups must be separated deeper down, and the intervening nodes have nothing else to hold. Canonical here means *canonical topology given the hashes present*, not “no node ever has one child”.

4.4 Single-descent factories

Two operations recur so often in real code that all nine ports expose them directly rather than making callers compose lookup with update:

```
| var withThree = map.GetOrAdd(
```

```

3,
key => $"value-{key}",
out string threeValue);

var counted = PersistentHashMap<string, int>.Empty.AddOrUpdate(
    "requests",
    _ => 1,                                // absent: seed
    (_, current) => checked(current + 1),    // present: update
    out int requestCount);

```

Both hash once and descend once. That is the point: the naive composition (TryGetValue then SetItem) hashes twice and walks the trie twice. The contract around them is tight, and worth stating in full because it is easy to get subtly wrong.

One descent, one factory, no retry

Delegates are validated *before* hashing. Exactly one factory is selected and invoked exactly once; GetOrAdd invokes none on a hit. There is no retry loop — unlike a mutable concurrent map, a persistent update has no competing writer to lose a race to. An update factory receives the caller’s lookup key and the stored value, while the map retains the originally stored key object. If the produced value compares equal to the stored one, the source map and the stored value object are both retained. Any factory or comparer exception leaves the source unchanged and publishes nothing.

4.5 Sets, and what changes

PersistentHashSet is the same trie with payloads that carry no value. Its distinctive surface is set algebra and the six set relations — subset, proper subset, superset, proper superset, overlap, and set equality — all of which preserve the receiver’s policy and normalize the argument under it first.

Set algebra benefits from the same reference pruning as map equality: unioning a set with a descendant of itself skips every shared subtree. Unioning two unrelated sets does not, and the API notes say so.

4.6 Nine ports, one representation

Every port ships canonical split-bitmap CHAMP nodes, collision buckets, persistent path copying, retained hash policies, and stored representatives. Where they differ is in the optimization tier layered on top:

Port	Notes beyond the shared representation
C#	The reference implementation, and the only one with the owner-token transient kernel (Chapter 8). Lockstep reference-pruned MapEquals, bitmap-aligned structural Diff.
Kotlin	Same split bitmaps and canonical promotion; exact-policy mapEquals and typed diff traverse logical slots in lockstep using stored hashes.
Rust	Safe Arc nodes, <code>#![forbid(unsafe_code)]</code> . Both path copying and BulkBuilder freezing produce canonical nodes. Independently created hasher policies keep semantic lookup fallback.
Haskell	Strict split data/node maps, inline <code>(hash, key, value)</code> runs, typed MapDifference values, and indirection-aware pointer pruning. validStructure checks the CHAMP invariants.
C++	Header-first C++20, move-only bulk builder, compact inline payload vectors. Independently created stateful hash policies retain semantic equality/diff.
C	C17, split maps, inline type-erased payloads, flexible-array child runs, retain/release policy balance, allocation-failure rollback. Diff is visitor-based so it does not impose an allocator on callers.
TypeScript	Canonical CHAMP with reusable public bulk builders; retains the structural reference-pruning checkpoint.
Python	Canonical CHAMP with the same public semantics, but deliberately performs semantic lookup-based equality and diff (with exact-policy fast paths) rather than claiming the lockstep descendant-pruning optimization.
OCaml	Immutable 32-way CHAMP with the same public semantics; descendant-pruning is implementation-specific and not claimed.

Why enumeration order is a tested non-promise

It would be easy, and wrong, to let consumers depend on the current trie order. Durable7 has tests whose entire purpose is to stop that: they assert that independently built equal maps may enumerate differently, and they exist so that a

future representation change — a size-tiered small layout, say — cannot break downstream code that quietly assumed otherwise. The one place a hash trie *does* promise an order is Ctrie snapshot conversion (Chapter 7), which fixes canonical data-run-before-node-run order precisely so that snapshot-to-persistent conversion preserves the captured sequence.

Companions of the Trie

The cheapest new data structure is two old ones that agree with each other.

Nine families in this library are built by composing existing substrates rather than by introducing a new core tree. That is a deliberate rule, not an accident of laziness: a new core is a new set of invariants, a new validator, a new adversarial test suite, and nine new implementations. A composition inherits all of that from its parts, and the only genuinely new obligation is keeping the parts consistent with each other.

The composite rule

A composite publishes *all* of its constituent indexes together or publishes nothing. Counts agree across indexes; no entry exists in only one of them. Policy-bearing composites retain independent key/value, left/right, or equality/ordering policies as appropriate, and every mutation preserves retained versions and returns the receiver for documented logical no-ops.

5.1 Persistent hash bag

Specimen — PersistentHashBag

Purpose	Unordered multiset: one representative and one positive multiplicity per equality class.
Built from	One CHAMP map from representative to count.

Counts	Distinct-class count and expanded total count are separate values. No ambiguous single “count” is exposed.
Algebra	Maximum union, minimum intersection, saturated difference, checked additive sum.
Ports	All nine.

The bag stores multiplicities in $1 \dots 2^{31} - 1$ per class — never zero, because a zero-multiplicity class is an absent class. The expanded total is carried in a wider checked integer, and the choice of type is a small tour of the nine runtimes: C# uses a checked long, TypeScript a `bigint`, Python an unbounded `int`, OCaml a checked native `int`, and the native ports their corresponding bounded wide integer.

Bag algebra is where the receiver-policy rule earns its keep. A reference-different argument comparer is normalized eagerly under the receiver’s policy *before* any operation-specific shortcut, and collapsed argument multiplicities are checked for overflow and use the first representative observed in that argument version’s trie order. Expanded enumeration repeats each representative contiguously; distinct-item and entry views enumerate one class each, in the usual stable-but-unspecified order.

5.2 Persistent bidirectional map

Specimen — PersistentBiMap

Purpose	A strict bijection: each key class and each value class appears at most once.
Built from	A forward CHAMP $K \rightarrow V$ and an inverse CHAMP $V \rightarrow K$, with independently retained policies.
Inverse	$O(1)$ in pair count; enumerates nothing.
Storage	Approximately two map entries per logical pair — stated plainly rather than hidden.
Ports	All nine.

The bimap is the clearest illustration of the composite rule, because its consistency obligation is symmetric and total: both maps have the same count, every forward

entry has exactly one equivalent inverse entry, and every inverse entry has exactly one forward entry. There is no operation that can be halfway done.

The interesting decisions are in the update semantics.

- ▶ **Strict add rejects both ways.** Adding a pair whose key class is occupied fails; so does adding one whose value class is occupied — even when the complete pair is equivalent to one already present. The key domain is checked first, so a result-bearing API reports key conflict before value conflict. (C# deliberately keeps its established boolean `TryAdd` shape rather than growing a richer result.)
- ▶ **Set is not force-put.** Setting adds a missing free pair; treats a policy-equivalent value update as a no-op that retains both representatives; replaces a present key's value *only* when the new value class is free; and never displaces another key. There is no displacing mode, deliberately.
- ▶ **Replacement goes the long way round.** It removes and reinserts both directions rather than using the ordinary map's value-equality shortcut, because that shortcut consults a different policy than the one that governs the inverse domain.
- ▶ **Double inversion is free and exact.** C#, Kotlin, TypeScript, and Python cache reciprocal facade objects, so `map.Inverse.Inverse` is the original object. The value-semantic ports (C, C++, Haskell, Rust) guarantee the analogue: double inversion shares the original two immutable roots.

The family intentionally exposes no algebra, no builder, no transient, and no force-put. Each of those would need a bijection-preserving story, and none has a consumer asking for it.

5.3 Set-valued multimap and relation

Specimen — `PersistentHashMapMultimap` and `PersistentRelation`

Purpose	One key to many values (multimap); many-to-many with cheap inverse traversal (relation).
Built from	Multimap: CHAMP map from key to non-empty CHAMP set. Relation: two multimaps, forward and reverse.
Counts	Distinct-key count and checked pair count are separate.

Inverse

Inverse swaps the relation’s two persistent indexes without scanning the pair set.

Ports

All nine.

A multimap stores *only* non-empty value sets: removing the last pair for a key removes the key. That single invariant is what makes “does this key have any values” a membership test rather than a set-emptiness test, and it is why duplicate pair insertion and removal of an absent pair are identity no-ops.

The relation adds the reverse index, so asking “which lefts point at this right” is a lookup rather than a scan. Adding or removing a pair updates both directions atomically; on failure, both indexes and every retained version remain mutually consistent.

5.4 Map patch

Specimen — PersistentMapPatch

Purpose

A first-class, invertible, composable description of a change to a map.

Semantics

Distinguishes absence from every present value, including null-like ones.

Apply

Validates *all* before-states before applying *any* edit; preserves the source map on conflict.

Invert

Swaps before- and after-states.

Compose

Only through equal intermediate states.

Ports

All nine.

The patch is the library’s answer to “I want to describe a change, inspect it, send it somewhere, and maybe undo it”. Its preflight discipline is the whole design: a patch either applies completely or does not apply at all, and it will refuse rather than apply half of itself to a map whose before-state has drifted. The absence-versus-null distinction matters here more than almost anywhere else, because “set this key to null” and “delete this key” are different patches that a sloppier encoding would conflate.

5.5 Directed graph

Specimen — PersistentDirectedGraph

Purpose	Explicit vertices and unique directed edges with forward and reverse adjacency.
Built from	A vertex set plus two adjacency multimaps.
Vertices	Stored explicitly, so an isolated vertex is a real vertex. Adding an edge adds any missing endpoints.
Removal	Removing a vertex removes every incident edge.
Ports	All nine.

Storing vertices separately from edges is the design decision worth noticing. An edge-list-only graph cannot represent an isolated vertex, which means it cannot represent “I have declared this node but not yet connected it” — a state that occurs constantly in dependency graphs and build systems.

5.6 Indexed map

Specimen — PersistentIndexedMap

Purpose	A primary map with one automatically maintained non-unique secondary index.
Built from	The primary CHAMP map plus a secondary multimap keyed by a selector result.
Invariant	Exactly one secondary pair per primary key; changing a row’s selector class moves its membership atomically.
Ports	All nine.

This is the “find all users in this city” structure. The selector is applied on insertion, and the composite’s job is to keep the derived index honest across every edit — including the awkward one, where updating a row changes which secondary bucket it belongs to.

5.7 Persistent chunked bit set

Specimen — PersistentChunkedBitSet

Purpose	A sparse set of non-negative 32-bit integers with population rank and select.
Core	64-bit words at the leaves of a measured tree, with a popcount sum measure.
Membership	$O(\log c)$ for c represented chunks.
Rank/select	Inclusive rank and zero-based select by measured descent.
Algebra	Word-wise merge over sparse streams — never a scan to the largest index.
Ports	All nine.

The chunked bit set is filed with the trie companions because it plays the same role — a set — but its representation belongs to the sequence world: 64-bit words at the leaves of a measured tree whose measure is the population count of its subtree. That measure is what turns Rank and Select into descents rather than scans, and it is a small, satisfying demonstration of how much the measure framework (Chapter 10) buys.

Zero words are never stored, so storage is proportional to represented chunks rather than to the largest bit index, and set algebra walks two sparse word streams rather than iterating a dense range.

Which set do you actually want?

Three families in this library can represent “a set of integers”, and they are good at different things. `PersistentHashSet` is the general answer with no ordering. `PersistentIntSet` (Chapter 6) gives ascending signed order and *structural* merge. `PersistentChunkedBitSet` gives rank and select, and wins hardest when the integers are dense within clusters and the workload is set algebra. Reach for the third only when rank/select or word-wise algebra is the reason you are here.

Patricia: When the Key Is Its Own Hash

A hash trie hashes the key. If the key is already a number, that step is pure ceremony.

Specimen — Patricia integer maps and sets

Purpose	Persistent maps and sets over signed 32- or 64-bit integer keys, with ascending key order and structural merge.
Core	Big-endian path-compressed binary trie: each internal node stores a common prefix and the single bit on which its two subtrees differ.
Point ops	$O(\min(n, W))$ for key width W — in practice a few node hops.
Algebra	Prefix-aligned union, intersection, difference, with combining-function overloads. Work follows visited non-shared prefix regions; $O(n + m)$ worst case.
Order	Ascending <i>signed</i> order, via a sign-bit transform.
Ports	All nine, at both widths: PersistentIntMap/Set and PersistentLongMap/Set and their local spellings.

The hash trie is a fine general answer, and integer keys are the case where it is least differentiated: hashing an `int` to route it through a trie is a detour, because the key already *is* a well-distributed bit string. Okasaki and Gill’s 1998 paper on mergeable integer maps supplies the structure that takes the shortcut, and Haskell’s `Data.IntMap` has been the proof of its practicality for two decades.

6.1 Path compression and the branching bit

A binary trie over the bits of an integer key would be W levels deep and mostly empty. Patricia collapses the empty stretches: each internal node stores the prefix its subtree shares, together with the single bit position at which its two children diverge. Descent is then a prefix test and a bit test — no comparisons, no rebalancing, and no rotations, ever.

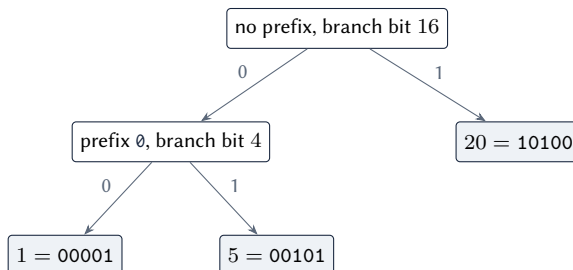


Figure 6.1: A Patricia trie holding $\{1, 5, 20\}$. Keys 1 and 5 agree on every bit above position 4, so the intervening levels are compressed away and the node records only the prefix and the branching bit.

Two pleasant properties fall out immediately. There are no collision buckets, because distinct keys are distinct bit strings — the structure is strictly simpler than the hash trie. And enumeration is in unsigned key order for free, since the trie is ordered by construction.

Signed order, by transform

Raw trie order is unsigned, which puts -1 after $2^{31} - 1$. Every port applies a sign-bit transform so that public enumeration is in ascending *signed* order, and every port ships both an explicit 32-bit and an explicit 64-bit surface rather than a platform-sized one. Python holds arbitrary-precision integers at runtime but validates and normalizes keys to the named fixed-width domain at the public boundary; Haskell exposes `Int32/Int64` aliases so the width contract does not depend on a platform `Int`.

6.2 The reason it exists: structural merge

Point operations are not the interesting part. The differentiator — the thing the hash trie genuinely cannot match — is *structural merge*.

Union, intersection, and difference align compressed prefixes and descend both tries together. Where the two prefixes are disjoint, the answer is known without looking inside either subtree. Where the two subtrees are reference-equal, the answer is one of them, and the recursion short-circuits. Work is proportional to the visited non-shared prefix regions plus the materialized result, and a no-op merge preserves reference identity outright.

That is a categorically different cost profile from merging two hash maps, which must enumerate one and look up each key in the other. Merge-heavy workloads — version-set algebra, sparse indices, identifier-set reconciliation, permission masks — stop paying a per-key hash-and-descend and start paying only for where the two operands actually differ.

```
var baseline = PersistentIntMap<string>.CreateRange(
    new[] { KeyValuePair.Create(-1, "old"), KeyValuePair.Create(10, "ten") });
var delta = PersistentIntMap<string>.CreateRange(
    new[] { KeyValuePair.Create(-1, "new"), KeyValuePair.Create(20, "twenty") });

var merged    = baseline.Union(delta);                // right-biased
var combined = baseline.Union(delta,
    (key, left, right) => $"{key}:{left}+{right}");

foreach (var key in merged.Keys)
    Console.WriteLine(key);    // -1, 10, 20 -- ascending signed order
```

The combining-function overloads are what make structural merge *useful* rather than merely fast. Without them, a union has to pick a side and discard information; with them, a merge can reconcile. The vocabulary is deliberately modeled on `Data.IntMap`'s `unionWith/intersectionWith`, because that is the shape that has proven itself in practice.

Sharing lowers the constant, not the bound

Retained sharing reduces the number of visited regions; it does not define a separate asymptotic class. The honest worst case remains $O(n + m)$, and two independently constructed tries with identical contents will be walked in full.

As always in this library, the good case is documented as a good case rather than promoted to a promise.

Ctrie: Snapshots Without Stopping the World

A mutable map you can photograph in constant time.

Specimen — `ConcurrentHashMap`

Purpose	A lock-free <i>mutable</i> concurrent hash map whose <code>Snapshot()</code> is $O(1)$ and yields an immutable view.
Core	Bitmap C-nodes, singleton leaves, equal-hash collision nodes, and tomb nodes, with generation-stamped indirection nodes.
Mechanism	Node-local GCAS for safe indirection replacement; root/main RDCSS for the snapshot transition; lazy generation renewal on later writes; tomb promotion on deletion.
Snapshot order	Canonical CHAMP order — data run before node run at every level — so conversion preserves the captured sequence exactly.
Ports	Full lock-free protocol: C# and Kotlin/JVM. Synchronous or lock-coordinated snapshot facades: OCaml, TypeScript, Python. Deliberately absent elsewhere.

Every other structure in this book is immutable. This one is not, and it earns its place by solving a problem the immutable structures create.

If your workload is a hot shared cache — hundreds of thousands of writes a second from many threads — a persistent map is the wrong tool, because every write allocates a path and every writer contends on publishing a new root. But if that same cache periodically has to hand a *consistent view* to a reporting thread, a checkpointer, or a

serializer, then a plain concurrent hash map is also the wrong tool, because taking a consistent view means locking everybody out or copying everything.

Prokopec, Bronson, Bagwell, and Odersky’s concurrent trie (PPoPP 2012) sits exactly in that gap: mutable-map throughput with $O(1)$ snapshots.

7.1 How a constant-time snapshot is possible

The trick is generation stamping. Every indirection node carries the generation in which it was created. `Snapshot()` does not copy anything; it atomically installs a new root with a bumped generation. From that moment, any writer that wants to modify a node belonging to an older generation must first *renew* it — copy it into the current generation — before writing. Readers holding the old root keep seeing old-generation nodes, which nobody will ever mutate again.

The subtlety, and the reason this algorithm is famous, is that “atomically install a new root” has to compose with “atomically replace a node” without losing a writer in between. Two mechanisms cooperate:

GCAS — generation-compare-and-set — makes an indirection node’s replacement safe by publishing a descriptor that readers help complete. A reader that finds a node mid-transition finishes the transition rather than waiting for it.

RDCSS — restricted double-compare single-swap — makes the root/main transition atomic with respect to the main-node observation, so a writer cannot slip between “I read the main node” and “the root was replaced”. The C# implementation gives GCAS priority in that transition specifically to close that window.

Deletion adds a third concern. Removing entries from a deep trie would otherwise leave a skeleton of empty interior nodes that future traversals still have to walk. The Ctrie contracts these paths by promoting *tomb* nodes — empty or singleton markers — up through their parents, so the structure does not accumulate scar tissue.

7.2 Crossing back into the persistent world

A snapshot is only useful if you can hand it to code that expects an immutable collection. Conversion is explicit and $O(n)$, and its contract is unusually specific:

Snapshot conversion preserves the captured sequence

Snapshot enumeration follows canonical CHAMP ordering: at each bitmap level, singleton and data entries precede multi-entry child runs; frozen empty tombs are skipped; frozen singleton tombs are promoted logically into the data run; and equal-hash collision buckets retain their local order. Explicit snapshot-to-`PersistentHashMap` conversion then preserves that exact sequence, the exact policy object, stored key and value representatives, present-null semantics, and isolation from every later live write.

This is the one place in the library where a hash structure commits to an enumeration order, and the reason is precisely that conversion must be *exact* rather than merely correct.

7.3 Why only two ports have the real thing

The reclamation problem

Lock-free node replacement is easy to write and hard to free. A thread may be holding a pointer into a node you have just unlinked, and nothing in the algorithm tells you when the last such thread is done.

On the CLR and the JVM the garbage collector solves this for free, which is why C# and Kotlin ship the complete protocol. In C and C++ it would need epoch-based reclamation or hazard pointers — a separate project with its own correctness burden. Rust's `Arc` does not help either: sharing an immutable graph is not the same problem as atomically swapping child pointers, and a safe design would mean an `ArcSwap` or epoch dependency and a new core. Haskell has a collector but no use for an imperative GCAS algorithm behind a pure API; STM or an atomic root wrapper would be a different structure wearing the same name. The parity economics therefore cap the lock-free tier at two managed ports, and the porting guide says so out loud rather than leaving a hole in a matrix.

OCaml, TypeScript, and Python ship something honest but different: a facade with the same consumer-level vocabulary — a mutable map, $O(1)$ immutable snapshots — implemented by serializing operations (an `RLock` in Python, a mutex in OCaml, isolate-locality in TypeScript) and swapping immutable CHAMP roots. The names preserve

the *use case*. They do not claim GCAS, RDCSS, generation renewal, tomb cleanup, or a lock-free progress guarantee, and their API notes say so in the first paragraph.

Builders, Sessions, and the Frozen Tier

Immutability is a promise about what others can observe, not a vow of celibacy toward your own unpublished nodes.

Building a million-entry persistent map one `SetItem` at a time allocates a million paths, and only the last one survives. Everyone who has built a persistent collection library has arrived at the same conclusion: there must be a way to mutate structure that nobody can see yet.

Durable7 keeps *five* distinct concepts here, and refuses to blur them, because each has a different lifecycle and a different set of promises (Figure 8.1).

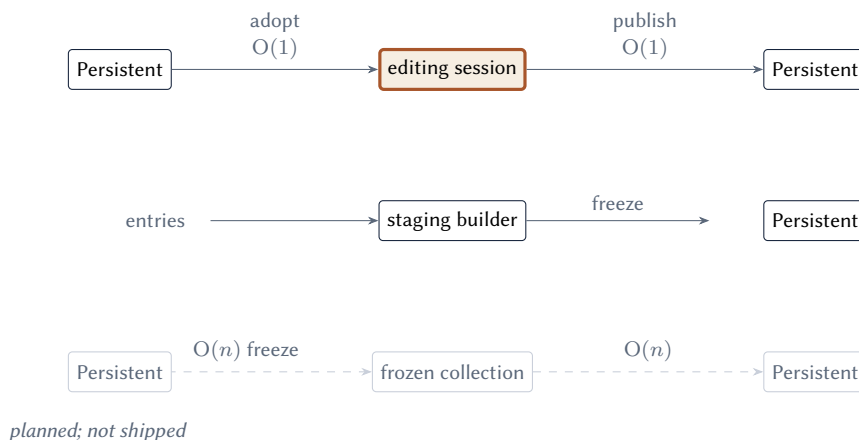


Figure 8.1: Five distinct lifecycles, kept deliberately separate. The session is one-way and consumed by publication; the staging builder is reusable and produces detached snapshots; the frozen tier remains an evidence-gated proposal.

8.1 Construction-only bulk builders

A bulk builder mutates unpublished leaf, collision, and bitmap nodes in place during a single construction pass, avoiding a path copy between successive input entries. It is narrower than an editing session and its contract says so.

Bulk builder

Nodes under construction are unpublished and mutated in place. Duplicate assignment retains the *first* stored key representative, keeps an earlier equal value representative, and otherwise installs the last distinct value. Freezing produces a *detached* immutable trie — later builder mutation cannot affect an earlier snapshot — and a reusable freeze leaves the builder active. Map and set bulk factories use the builder internally; ordinary updates on an existing persistent collection continue to use structural-sharing path copies.

C# uses one internally; C++, Rust, TypeScript, and Python expose the facility publicly. Rust additionally offers a consuming freeze that moves owned nodes rather than sharing them. OCaml’s reusable staging builder preserves the detached-freeze semantics and the representative rules but performs persistent path copying internally, and explicitly does not claim the unpublished mutable-node construction bound.

8.2 The one-way editing session

A session is the answer to “I have an existing map and I want to make eighty changes to it”. It adopts a persistent root in $O(1)$, accepts edits under single ownership, and publishes once.

One-way lifecycle

Adoption and successful publication perform $O(1)$ trie work and do not traverse the graph. Sessions preserve the exact equality/hash policy, stored representatives, unchanged-version trie order, retained source isolation, and receiver-policy set-relation semantics.

Publication is *one-way*: it consumes the logical session, and the language either rejects later access dynamically or makes it unavailable through ownership. A session is unsynchronized and has one logical owner, even where the language

permits explicit handle aliases or moves.

Logical no-ops preserve the current root and do not invalidate a version-bound view. A clean adopted session therefore republishes the exact source instance. A failed edit never installs a partial successor.

8.2.1 One optimized kernel, eight semantic siblings

This is the sharpest example in the library of the reference-versus-checkpoint distinction, and it is worth being precise about.

C# implements the *owner-token transient*: edits mutate only nodes and arrays proven to be owned by the active edit token, and path-copy anything shared or sealed. `Persist()` seals the token, publishes without traversing the graph, and invalidates the mutable object along with every view and enumerator obtained from it. A clean `source.ToTransient().Persist()` returns the exact source object, including after logical no-op edits.

The other eight ports implement the same *observable lifecycle* over ordinary persistent path copying. Each changed point edit computes a normal path-copied successor and replaces the session's current value. They preserve policy identity, stored representatives, exact clean/no-op root identity, retained sources, and failure atomicity — and they make *no* transient-throughput or allocation-win claim. That is stated in their API notes, not discovered in a profiler.

Port	Lifecycle and failure shape
C	Explicit clone/destroy handles over one reference-counted state. Clones are lifecycle <i>aliases</i> : one successful publication consumes them all. Operations report a consumed or modified status; failed edits and relation queries preserve session and output atomicity.
C++	Sessions are movable but not copyable and publish only through <code>std::move(session).persist()</code> . Generation-bound iterators reject changed or consumed sessions. A throwing custom policy move terminally invalidates the source (and, for assignment, the destination), and publication after such a move carries no retry guarantee — nothrow-movable policies avoid both boundaries.
Haskell	Sessions live in IO and are consumed by <code>persistMap/persistSet</code> . Candidate construction precedes a masked commit, so neither synchronous nor asynchronous failure can partially install an edit.

Kotlin	Consumption is enforced at runtime. Acquired views capture the current snapshot and session version, survive logical no-ops, and fail after content changes. Callback failure leaves the session unchanged and retryable.
OCaml	The session retains one persistent root, replaces it with path-copy successors, and dynamically rejects access after publication. The reusable bulk builder is a separate facility.
Rust	Publication is <code>into_persistent(self)</code> , so the type system prevents use-after-publication. <code>into_transient</code> moves a source; <code>to_transient</code> shares its root.
TypeScript	One-way publication enforced at runtime inside one JavaScript isolate; no cross-worker progress claim.
Python	Consumes on publication and invalidates iterators after content changes or publication.

8.3 The frozen tier, and why it has not shipped

The third lifecycle — $O(n)$ conversion to a read-optimized layout with no update-shaped API — is designed, prototyped, and deliberately unshipped.

Its intended contract is already written down: separate `FrozenHashMap` and `FrozenHashSet` types, one offline-built general hash layout for every count and key type, preserved comparer identity and stored representatives, preserved source traversal sequence, no library-controlled allocation on point lookup, and enumeration that is stable for an unchanged instance but otherwise unspecified — never insertion, sorted, canonical, or serialization order. Freezing an already-frozen value is identity; converting back is visibly $O(n)$.

What is missing is evidence. The track is staged: a faithful packed-index signal first, fixed layouts compared only once a credible read-heavy regime appears, and a public type authorized only behind lookup, enumeration, memory, and realistic construction break-even numbers. The benchmark-local candidates already cover the low-risk half — rebuilding canonical CHAMP from a packed source-order entry sequence through the existing bulk builder, with an untimed verifier locking comparer identity, traversal sequence, representative rules, null and collision behavior, and round-trip reconstruction.

Deliberate non-adaptivity

Tiny flat layouts, string prefix and length dispatch, integer-key routing, binary-fuse and ribbon filters, Eytzinger and PGM sorted layouts, and count-derived representation selectors are all postponed — not rejected, postponed. The stated reason is methodological: the value of the *temporal* lifecycle (transient, persistent, frozen) is being isolated before it is combined with size or key specialization, so that a later benchmark can tell which of the two produced a gain. Benchmarks may contain tiny and string datasets; that is a workload, not permission to select a representation.

PART III

Sequences

Finger Trees

A tree that keeps its hands in its pockets: whatever else is deep, both ends are always within reach.

Specimen — `FingerTreeDeque` and `FingerTree`

Purpose	A persistent indexed sequence with cheap endpoints and cheap concatenation (<code>FingerTreeDeque</code>), and a general monoid-measured tree used directly and as the engine beneath a dozen facades (<code>FingerTree</code>).
Core	Hinze and Paterson’s 2-3 finger tree: digits at each end, a recursively nested spine of nodes in the middle.
Endpoints	$O(1)$ amortized push and pop at either end.
Concat	$O(\log \min(n, m))$ — proportional to the <i>smaller</i> operand.
Index / split	$O(\log d)$, where d is the distance from the nearer end. The measured U-shape.
Ports	All nine: <code>FingerTreeDeque</code> / <code>PersistentDeque</code> / <code>persistent_deque</code> / <code>Deque</code> , plus the measured tree under each language’s local name.

Most balanced trees treat all positions alike. A finger tree does not: it keeps a small unbalanced “digit” of one to four elements at each end and pushes the balanced structure into the middle, where a recursively nested spine holds *nodes* of two or three elements, which themselves hold nodes, and so on (Figure 9.1).

That asymmetry is the whole design. Pushing onto an end usually just grows a

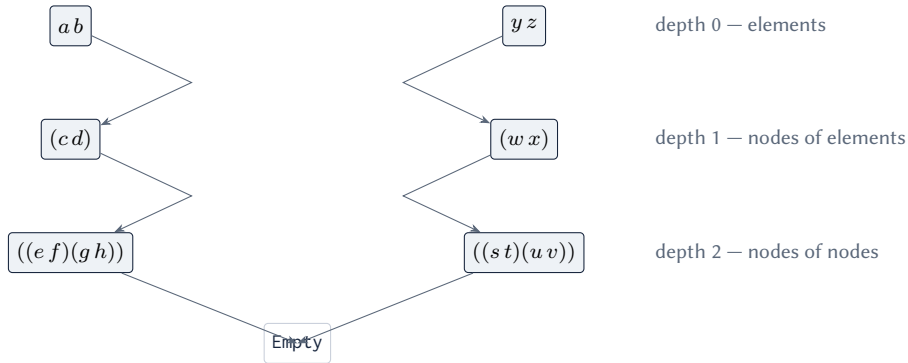


Figure 9.1: The finger-tree shape. Both ends sit at depth zero and stay there; only the middle gets deeper. This is why endpoint operations are constant-amortized while indexing costs a logarithm of the distance to the nearer end.

digit, which is $O(1)$; occasionally a digit overflows and a node is pushed one level down, and the amortized analysis — with memoized lazy suspensions spreading the debt across later forces — keeps the average constant. Concatenation is proportional to the logarithm of the *smaller* operand, because the two spines only have to be zipped together as deep as the shorter one goes. Indexing costs a logarithm of the distance to the nearer end, so both `seq[0]` and `seq[n-1]` are nearly free while the middle is logarithmic — the “measured U-shape” the API notes refer to.

9.1 The deque

`FingerTreeDeque<T>` is the tuned sequence facade. It exposes endpoint operations, concatenation, splitting, positional edits, and range operations.

```
var deque    = FingerTreeDeque<int>.Create(1, 2, 3);
var extended = deque.AddFirst(0).AddLast(4);

var first    = extended.PopFirst();
var rest     = first.Rest;           // [1, 2, 3, 4]

var split    = rest.SplitAt(2);
var joined   = split.Left.Concat(split.Right);
var replaced = joined.SetItem(1, 20); // [1, 20, 3, 4]
```

It also carries a small set of helpers for sequences the caller *knows* to be sorted

— `SortedLowerBound`, `SortedUpperBound`, `InsertSorted`, `SplitAtSortedEqualRange`. These assume sortedness rather than maintaining it, which is exactly the right split of responsibilities: if sortedness is a long-lived invariant, the sorted collections of Chapter 16 maintain it for you; if it is a passing property of a sequence you happen to have, these helpers exploit it without committing you to a different type.

9.2 The measured tree

The general `FingerTree<TElement, TMeasure, TMeasureOps>` is the deque’s more powerful sibling and the engine underneath sorted bags, sets and maps, priority queues, interval trees, ropes, and the chunked bit set. Every node caches the monoidal summary of its subtree, and the tree can be split at the first position where a predicate over the running prefix measure flips from false to true.

That single operation — *split by measure* — is startlingly general. Measure by element count and you get positional indexing. Measure by maximum key and you get sorted lookup. Measure by minimum priority and you get a priority queue. Measure by maximum interval endpoint and you get interval search. Measure by newline count and you get a text editor’s line index. Chapter 10 is entirely about this idea.

Finger-tree core obligations

Empty, singleton, endpoint, concatenation, split, index, insert, remove, and range operations preserve immutable snapshot semantics. Measure policies supply an identity, an element measure, and an associative combine. Split and locate depend on a documented predicate boundary over prefix measures, and any new measure or predicate helper must document the ordering assumptions it needs. Product and sum measures preserve component order and component-specific predicate mapping. Lazy middle publication and cached measures must never expose partially initialized data to a reader.

This was once a checkpoint boundary

Until August 2026 this box named a real gap: five ports built their sorted collections, priority queues, and interval trees over a measured AVL or another balanced tree rather than a lazy-spine finger tree, and their API notes recorded the weaker asymptotic claim rather than inferring the stronger one. All five have

since been rebuilt, so every port now carries the lazy Hinze–Paterson core and the bounds above hold in all nine. Chapter 2 is the account of why the difference mattered, how it was found, and what a strict language must build by hand to close it.

Measures: Monoids as Connective Tissue

Give a tree an associative summary and it will answer questions you never built it to answer.

This chapter has no data structure of its own. It describes the abstraction that makes a third of the library possible, and it is the single best return on conceptual investment in this book.

10.1 The idea

A *measure* is a monoid — a type with an identity element and an associative binary combine — plus a function from an element to a value of that type. Cache the measure of every subtree on its root node, and two things become cheap:

1. **Aggregate queries.** The measure of the whole sequence is the measure at the root, read in $O(1)$. The measure of a range is a logarithmic boundary computation.
2. **Measure-guided descent.** Walk from the root, accumulating the running prefix measure. At each node, ask whether the prefix *including* the left subtree satisfies a predicate. If yes, descend left; if no, add the left measure to the accumulator and descend right. You have located the first position where the predicate flips, in $O(\log n)$, without visiting the elements you skipped.

Associativity is what makes step 2 sound — it lets the accumulated prefix be computed in any grouping. Commutativity is *not* required, and the library is scrupulous about this: ordered measures are combined strictly left to right, and several tests exist specifically to prove that a non-commutative measure behaves.

What a measure policy must supply

An identity value; an element measure; an associative combine. Nothing more. Predicates used for split and locate must be monotone over the running prefix measure — false, then true, with no oscillation — because descent is a binary search and an oscillating predicate makes the answer meaningless. Collections cannot repair an unlawful policy, and such policies are outside the logical contract.

10.2 The measures that ship

Measure	What it turns the tree into
Size / count	A positionally indexed sequence: split at “prefix count $> i$ ” gives element i .
Sum	A prefix-sum sequence; range sums by boundary subtraction where the element type allows it, or by ordered range measure where it does not.
Maximum	An ordered structure: split at “prefix max $\geq k$ ” is a sorted lower bound. Underpins sorted sets, bags, and maps.
Minimum	A priority queue: the root measure is the front priority, and descent locates it.
Order statistic (count + last key)	Sorted collections that also support rank and select.
Interval (count, last low, max high)	An interval tree: a subtree whose maximum high endpoint is below the probe cannot contain an overlap, so it is pruned.
Newline	A text rope: line count in the measure turns line/column navigation into a descent.
Popcount	The chunked bit set: inclusive rank and zero-based select by measured descent over 64-bit words.
Product / pair	Two measures carried together, preserving component order and mapping predicates to the right component. This is how “sorted <i>and</i> indexable” works.

10.3 How the nine ports spell it

The abstraction is the same everywhere; only its encoding is local. C#, C++, Kotlin, and Python expose typed measure abstractions (with comparison and predicate abstractions where their surfaces need them); C exposes equivalent policy callbacks with context pointers; Haskell uses an ordinary type class and plain functions $v \rightarrow \text{Bool}$ as predicates, because in Haskell a predicate abstraction would be ceremony. Rust exposes a `MeasurePolicy` trait plus a generous set of ready-made measures — `SizeMeasure`, `SumMeasure`, `MaxMeasure`, `MinMeasure`, `KeyMeasure`, `ProductMeasure`, `OrderStatisticMeasure`, and the combined `SizeAndSum/SizeAndMax/SizeAndMin` forms.

Where measures paid off unexpectedly

The chunked bit set was originally imagined as a HAMT-backed structure, and it half-failed the library's own test for whether a composition is load-bearing. The recommended prototype instead put `ulong` chunks at the leaves of a measured tree with a popcount sum measure — and got membership, inclusive rank, select, and reference-short-circuiting set algebra out of machinery that already existed. That is the measure framework doing what it is for: a new public capability with no new core.

RRB Vectors

The persistent vector everybody means when they say “persistent vector”.

Specimen — RrbVector

Purpose	A persistent sequence with <i>uniform</i> random access — no U-shape — plus logarithmic concatenation and splitting.
Core	Relaxed radix-balanced tree: 32-way branches, 32-element leaves. Regular branches are radix-indexed with pure bit arithmetic and carry no size table; only branches touched by concatenation carry cumulative sizes.
Index	$O(\log_{32} n)$ with a small constant, uniform across positions.
Endpoints	$O(\log_{32} n)$ — there is deliberately no persistent tail buffer.
Concat	$O(\log_{32}(n + m))$ boundary-spine merge.
Builder	Append-only, freezes full leaves safely and caches immutable snapshots.
Ports	All nine.

Bagwell and Rompf’s relaxed radix-balanced tree (2011, with the practical transient treatment in Stucki et al., ICFP 2015) is the structure behind Scala’s persistent vector, Clojure’s `core.rrb-vector`, C++ `immer`, and Rust `im`. Its central trick is to be a radix trie *most* of the time.

11.1 Regular and relaxed branches

In a plain radix trie of branching factor 32, finding index i is pure bit arithmetic: take five bits of i per level and you are there. No size tables, no comparisons. That works only if every node is full, which stops being true the moment you concatenate two vectors.

RRB's answer is to allow both kinds of node (Figure 11.1). A *regular* branch is full and radix-indexable; it stores no size table at all. A *relaxed* branch carries a small array of cumulative sizes and is searched rather than computed. Concatenation introduces relaxed branches only along the seam; everything away from the seam stays regular and stays fast.

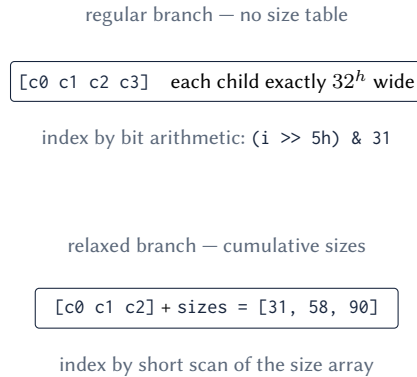


Figure 11.1: RRB nodes come in two flavors. Only nodes disturbed by concatenation pay for a size table; the rest keep the radix trie's pure bit-arithmetic indexing.

11.2 The representation boundary, stated up front

Boundary-spine concatenation, not global redistribution

All ports deliberately implement *boundary-spine* concatenation: the two child arrays at the seam are repartitioned, and nothing is promised about minimum fullness for leaves or branches away from that seam. The full Stucki-et-al. redistribution pass is not performed.

The adversarial density assertions in the test suites are regression gates for the histories they exercise — not a validator-certified representation

contract. What the validators *do* certify is a size-derived height cap: $\lfloor (\text{bit width of the count type} - 1)/5 \rfloor + 1$. The first term is the greatest minimum height required anywhere in the count domain; the extra level admits the boundary-only contract’s legal “minimum height + 1” slack even in the top count band. That works out to seven for the C#/Kotlin `Int` count domain and thirteen on 64-bit `size_t/Int/usize` targets. TypeScript and Python apply their own checked runtime count domains.

11.3 Choosing between three sequences

The library ships three persistent sequences with overlapping capabilities, and declines to pretend one of them wins.

Workload	Finger-tree deque	Rope	/	RRB vector
Endpoint push/pop	$O(1)$ amortized — best	$O(\log n)$, staged amortized	$O(1)$ / $O(\log_{32} n)$, no persistent tail	
Random index, middle	$O(\log n)$, U-shaped	$O(\log n)$ chunked	/ $O(\log_{32} n)$, small constant — best	
Concatenation	$O(\log \min)$ — best	$O(\log \min)$	/ $O(\log n)$ with relaxed nodes	
Cache density	pointer-heavy	chunked, best for scans	/ 32-way arrays, good	
Bulk build	element-wise	chunk-staged builder — best	/ builder tier	

The RRB vector’s differentiator is uniform indexing. The shipped persistent value deliberately has *no* dedicated tail buffer, so endpoint operations remain logarithmic and the separate append-only builder is the bulk-construction path. Concatenation is logarithmic but with worse constants than the deque’s.

The library’s stance is unusually restrained here: it ships the vector, ships benchmark comparisons against the rope and the platform’s own immutable list, and tells consumers to choose on their measured workload rather than on the asymptotics. Shipping an implementation does not, by itself, erase the overlap with two existing sequence families.

Ropes and Text

A string that does not mind being edited in the middle.

Specimen — Rope, MeasuredRope, TextRope

Purpose	Persistent chunked sequences (Rope), the same with a custom split/locate measure (MeasuredRope), and a newline-aware text specialization (TextRope).
Core	Runs of elements held contiguously in immutable leaf chunks at the leaves of a measured tree.
Edits	$O(\log n)$ plus bounded chunk work.
Concat / split	$O(\log \min) / O(\log n)$.
Scans	Array speed within a chunk — the reason to prefer a rope over a pointer-per-element sequence.
Ports	All nine, each with positional, measured, and text cursors.

The rope's insight is that the atoms of a sequence do not have to be its elements. Put a run of elements in one contiguous immutable leaf, and you get tree behavior for edits with array behavior for scans — which is exactly the trade a text editor wants, since editing is local and rendering is sequential.

12.1 Three layers, one machine

Rope<T> is positional. Its measure is element count; indexing, splitting, insertion, removal, concatenation, and slicing are all defined over logical element offsets.

MeasuredRope<T, M> replaces the count measure with a caller's monoid, so split and locate can be driven by anything associative. It inherits the measure-law requirements of Chapter 10 wholesale.

TextRope is the measured rope specialized to a newline measure, plus the helpers that make text navigation pleasant: line and column conversion, line extraction, newline-style handling.

Because **TextRope** is a specialization rather than a separate core, its line index is not a side table that can drift out of sync — it is the measure, maintained by the same code path that maintains everything else.

What a text position counts

This is the least portable thing in the library, and the docs are correspondingly blunt about it. Text positions count: UTF-16 code units in C#, Kotlin, and TypeScript; char-sized bytes in C and C++; Haskell `Char` elements; Unicode scalar values in OCaml and Rust; Unicode code points in Python. *None* of them denotes a grapheme cluster index. Line and column rules are zero-based, and the C port uses LF-only newline recognition.

If you are writing cross-language code that exchanges text offsets, this is the paragraph that will save you.

12.2 Builders

Ropes are commonly built from a stream rather than edited into existence, so every port has a builder that stages chunks and publishes an immutable rope. The contract that matters is publication isolation: later builder changes cannot affect a previously produced rope. As with the trie builders, mutation is confined to nodes nobody can see.

The Range-Update Sequence

Lazy propagation, but you are allowed to keep the old version.

Specimen — RangeUpdateSequence

Purpose	An immutable indexed sequence that can apply one algebraic transformation to a contiguous range, and query that range's aggregate, without visiting the covered elements.
Core	A separately owned path-copied implicit AVL — <i>not</i> a tagged modification of either finger-tree engine.
Whole-sequence update	$O(1)$: one new root.
Subrange update / query	$O(\log n)$ boundary work.
Point edit, split, concat	$O(\log n)$.
Policy	A tag monoid acting lawfully on elements and on cached ordered measures, with an explicit composition direction.
Ports	All nine. OCaml ships an immutable-array checkpoint with the same public algebra and persistence semantics and makes no implicit-AVL lazy-update claim.

Competitive programmers know the ephemeral version of this structure as a segment tree with lazy propagation, and it is one of the great practical algorithms: “add 7 to every element between positions 200 and 9,000” in logarithmic time, by writing a pending

instruction on a handful of internal nodes instead of touching nine thousand elements.

Making it persistent is harder than it sounds, because the classic implementation *pushes* pending tags down as it descends — and pushing is mutation. In a persistent structure a read must not mutate anything, because another version is looking at those same nodes.

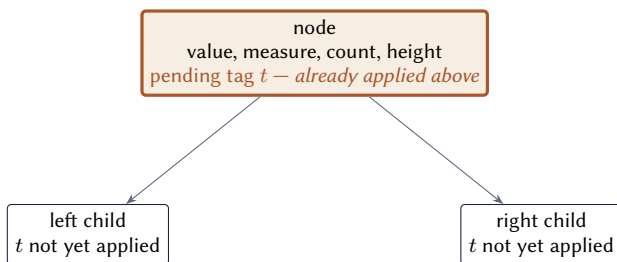
13.1 The invariant that makes it work

The library resolves this with a carefully chosen convention (Figure 13.1):

The node invariant

A node's stored value and cached measure *already include* that node's own pending tag. Its children do *not*.

Structural descent and rotations push a pending tag down before rearranging nodes. Read-only indexing, range measurement, and enumeration instead carry the inherited tag down the path and compose it after the node's older tag — never writing to shared storage.



Reads carry t downward in an accumulator and compose it with each child's own tag. Nothing is written.

Structural writes push t into both children first, then rotate or rebuild — on freshly copied nodes.

Figure 13.1: The pending-tag convention. Because a node's own summary already reflects its tag, a whole-subtree transformation is a single-node path copy, and cached measures stay queryable without ever visiting an element.

The payoff is direct. Applying a non-identity tag to the entire sequence allocates exactly one new root — $O(1)$, regardless of length. A proper subrange decomposes into

$O(\log n)$ maximal covered subtrees plus two boundary paths, so it is logarithmic. And because a node's cached measure is already tag-inclusive, range aggregate queries are logarithmic too, with no element visits at all.

13.2 The algebra, and its laws

The policy — `IRangeUpdateAlgebra<TElement, TMeasure, TTag>` in C#, with local spellings elsewhere — extends the ordinary measure monoid with a tag monoid and two actions. The single most important detail is the composition direction, and it is stated everywhere it appears:

`Compose(newer, older)` means: apply *older* first, *newer* second.

The executable law gate covers the following, and a policy that violates them is outside the collection's logical contract — the structure cannot repair an unlawful action and does not pretend to:

- ▶ the measure has an identity and an associative ordered combine;
- ▶ tags have an identity and an associative composition in that direction;
- ▶ the tag action on an element composes in the same order;
- ▶ the tag action on a cached measure composes in the same order *and receives the represented element count*;
- ▶ applying a tag to a singleton measure agrees with measuring the tagged element;
- ▶ the action distributes over ordered measure combination with the correct left and right counts — tested with a non-commutative measure;
- ▶ applying any tag to the empty measure at count zero yields the empty measure; and
- ▶ every tag that `IsIdentity` recognizes — including a value-distinct representation of identity — obeys the full identity equations.

13.3 In use

The canonical example is an affine algebra whose tags either assign a value or add to it, over a sum measure:

```
using Sequence = Durable7.FingerTree.RangeUpdateSequence<
```

```

long, long, AffineTag, AffineSumAlgebra>;

var source    = Sequence.Create([1, 2, 3, 4]);
var added     = source.ApplyRange(1, 2, AffineTag.Add(10));
// [1, 12, 13, 4], Measure == 30

var assigned = added.ApplyRange(2, 2, AffineTag.Assign(7));
// [1, 12, 7, 7], Measure == 27

long middleSum = assigned.MeasureRange(1, 2);    // 19
var (prefix, suffix) = assigned.SplitAt(2);
var reconstructed = prefix.Concat(suffix);

// source is still [1, 2, 3, 4].

```

Note what the API deliberately does *not* have: there is no `AddRange` or `AssignRange` method on the generic core, because those verbs belong to a particular tag algebra rather than to the sequence. The core knows about tags; it does not know what they mean.

Behavioral obligations

Retained versions stay immutable and share unaffected interiors. Range validation precedes tag callbacks; empty ranges preserve the receiver and the empty measure without invoking the policy at all; recognized identity updates preserve the receiver. Direct insert, replace, and remove push old tags before installing new content, so a prior range tag never leaks onto newly inserted elements. Any policy exception publishes no successor.

Enumerator copying

The concrete enumerator is a mutable struct that is deliberately *not* safe to copy mid-traversal: copies share traversal state and the stale copy fails fast once the other advances. Independently created enumerators are fully independent and safe for concurrent reads. This is the standard trade for a non-allocating enumerator, and the fail-fast behavior exists so that the mistake surfaces immediately rather than as silent misreads.

The Reversible Deque

Reversal as a fact about how you are looking, not about where things are.

Specimen — ReversibleDeque

Purpose	A persistent sequence for which logical reversal is a primary operation.
Core	Persistent deque storage plus an orientation bit or mirrored root.
Reverse	$O(1)$ — a new view over shared storage, not a materialized sequence.
Everything else	Endpoint operations, indexing, splitting, concatenation, traversal, and materialization all observe the current logical orientation.
Ports	All nine.

This is the smallest structure in the book and it makes a point worth more than its size: a great many “operations” are really changes of perspective, and a persistent library is in an unusually good position to represent them as such.

```
var deque    = ReversibleDeque<int>.Create(1, 2, 3);
var reversed = deque.Reverse();

var first    = reversed[0];           // 3
var restored = reversed.Reverse(); // shares storage with deque
```

Orientation obligations

Reversal returns a new value and preserves the original. Every operation — endpoints, indexing, splitting, concatenation, traversal, materialization — observes the logical orientation. Mixed-orientation concatenation must not silently reinterpret one operand in physical order. An $O(1)$ reversal claim belongs only where the implementation genuinely uses an orientation bit, mirrored root, or equivalent view rather than materializing a new sequence.

That last clause is the interesting one. It would be easy for a port to implement `Reverse` by building a reversed copy, keep every observable behavior identical, and quietly break the one promise anybody chose the type for. The repository maintains a dedicated complexity audit for exactly this family, and it is the required reading before touching any orientation-aware implementation.

DABA Lite: The Deliberate Outlier

Not persistent, not sorry.

Specimen — DabaLite

Purpose	Maintain the aggregate of a FIFO sliding window under any associative monoid — no commutativity, no inverse required.
Core	The De-Amortized Banker’s Aggregator (Lite), over a linked queue of 64-slot chunks with six ordered cursors.
Callback ceilings	Insert invokes Combine at most three times; evict at most twice; query at most once; an empty query, none. Unconditional.
Operations	Worst-case $O(1)$ structural work, given $O(1)$ monoid callbacks.
Mutability	Mutable and unsynchronized. This is the one collection in the book that is not persistent.
Ports	All nine.

Every other structure in this book preserves its old versions. This one overwrites them on purpose, and it belongs here anyway for two reasons: it is the measure framework’s first consumer outside the trees, and it answers a question the trees answer worse.

The question is: what is the sum — or max, or concatenation, or any associative combination — of the last k items in a stream, updated as items arrive and depart? A finger tree answers it in $O(\log n)$. Tangwongsan, Hirzel, and Schneider’s aggregator (DEBS 2017, with the Lite representation in the 2021 journal article) removes the logarithm entirely for this deliberately mutable special case.

15.1 Six cursors and a bounded fixup

The window is a queue of partial aggregates, overlaid with six cursors that always satisfy $F \leq L \leq R \leq A \leq B \leq E$ (Figure 15.1). Between them they delimit the front, left, right, accumulator, and back regions; two extra fields carry the products that do not live in queue slots.



Every insertion or eviction performs exactly *one* bounded fixup: front exhaustion collapses the cursor state, $L = B$ initializes the next flip, $L = R$ advances the three work cursors, and otherwise one left and one right partial aggregate are rewritten. There is no unbounded reversal loop — that is the “de-amortized” in the name.

Figure 15.1: The DABA Lite cursor state. The schedule is what makes the operation bounds worst-case rather than amortized.

The invocation ceilings — three, two, one — are unconditional and tested directly by counting callbacks. What is *not* unconditional is the wall clock: the complete operations are worst-case $O(1)$ only when the monoid’s `Combine` and `identity` are themselves $O(1)$. The library states the ceiling and the condition separately, which is the honest way to do it.

15.2 The parts that differ by runtime

Clear is not $O(1)$ everywhere

On the tracing runtimes — C#, Kotlin/JVM, TypeScript, Python — `Clear` swaps in one fresh empty chunk in $O(1)$ and invokes `Combine` zero times.

In C, C++, and Rust it is deliberately $O(n + c)$ for n values across c chunks, because deterministic destruction has to actually release n generic owned values. All three break the chain iteratively, neither leaking nor deferring an unbounded retired representation. Insert, eviction, and query keep their worst-case $O(1)$ structural and callback work in every port.

Python is a third case: it clears by replacing the active chain in $O(1)$, but unreachable cyclic backing may be reclaimed later by the collector, so it does not

claim the native ports' prompt deterministic destruction.

Failure behavior is similarly runtime-shaped but semantically uniform: a throwing monoid callback leaves the published window unchanged for every mutating operation. C++ achieves this by building a private fixup plan under a nothrow-move value constraint and committing without throwing; Rust plans candidate aggregates, slot rewrites, and a possible successor chunk before publication so a panic cannot expose a partial mutation; C stages callback-produced values in seven fixed aligned temporaries and transfers ownership by byte move.

15.3 What it will not do

No oldest-value access, no enumeration, no synchronization

The Lite schedule intentionally overwrites raw values with partial aggregates, so there is no API to read the oldest value or enumerate the window — the information is genuinely gone. There is no synchronization either; callers must not overlap access to one instance without external serialization. And the type is *ephemeral*: it is a fit for transient streaming state, not a persistent replacement for a deque.

15.4 Space, and a validator that knows nothing

The paper's logical accounting is $n + 2$ values: n queue partial aggregates plus the two aggregate fields. The chunked implementations instead allocate capacity for n live positions plus 1 to 127 slack slots, and an empty instance retains one 64-slot block.

`ValidateStructure` is deliberately callback-free and content-blind. In $O(c)$ time and space for c active chunks it checks links, cursor reachability and order, the count and distance equations, the DABA region-size equations, and the chunk/slack bound — and returns a statistics record with the count, five region lengths, active block count, allocated capacity, and slack. It cannot check that the aggregate is *correct*, because the original values may already have been overwritten; correctness is tested against an external FIFO model instead. That division of labor is a small model of the whole repository's approach to validation.

PART IV

Order and Priority

Sorted Collections

A measured tree with a maximum-key measure is a sorted set. Everything else is presentation.

Specimen — SortedBag, SortedSet, SortedDictionary

Purpose	Immutable comparer-ordered multisets, sets, and key/value maps, with rank and boundary queries.
Core	Order-statistic measures over the measured tree — typically cached count plus last key — in the mature ports; an idiomatic ordered storage checkpoint elsewhere.
Identity	<i>Comparison</i> defines equality here, not hashing. Two values that compare equal are the same element.
Enumeration	Bag: sorted, with duplicates. Set: sorted, one representative per equivalence class. Map: sorted by key.
Builders	Batched edits published as one immutable snapshot.
Ports	All nine.

These are the collections most people reach for first, and in this library they are facades: put a measure that summarizes “the largest key in this subtree” on a measured tree, and splitting at “prefix maximum $\geq k$ ” is a sorted lower bound. Add a count to the measure — a product measure — and the same tree also supports rank and select. The sorted collections are that observation, dressed for use.

Sorted-collection obligations

Comparison policy defines equality. If two values compare equal, the API must document whether duplicates are counted, rejected, replaced, or merged — and each of the three families answers differently, which is the point of having three. Sorted bag enumeration includes duplicates in sorted order. Sorted set enumeration contains one representative per comparison-equivalent value. Sorted map lookup, rank, and boundary queries use key comparison, never hash equality. Rank and index APIs must define zero- versus one-based indexing, miss behavior, and duplicate handling. Builders must document when they stop sharing with immutable snapshots and when snapshots are published.

Comparison equality is not hash equality

A value can be equal under a comparer and unequal under an equality comparer, or vice versa, and mixing the two families in one program is a reliable source of confusion. A case-insensitive comparer makes "Key" and "key" the same element in a `SortedSet` and different elements in a `PersistentHashSet` — unless the hash policy agrees, which it will only do if someone made it agree. The library keeps the two policy kinds firmly separate and never derives one from the other.

Canonical Sorted Sets: Zip and Zip-Zip Trees

For collections compared far more often than they are modified.

Specimen — CanonicalSortedSet and ZipTreeRankPolicy

Purpose	A sorted set whose <i>shape</i> is determined by its contents under a retained rank policy, so equal sets converge on equal topology and inequality is usually rejected in $O(1)$ by a memoized digest.
Core	Zip-zip tree: a randomized-rank binary search tree where insertion and deletion are single root-to-position <i>unzip</i> and <i>zip</i> passes — no rotations.
Ranks	A geometric primary coordinate and a fixed 64-bit secondary coordinate, both derived from HMAC-SHA-256 over the equivalence class.
Policy modes	An unexposed random key, a public reproducibility seed, or a caller-retained secret key.
Equality	Memoized root digest rejects most inequality in $O(1)$; shared nodes prune the rest; independently built equal sets still cost $O(n)$.
Ports	All nine.

Tarjan, Levy, and Timmel’s zip trees (2018/2019) reformulate the treap so that insertion and deletion are single passes: to insert, *unzip* the search path into two spines and hang them off the new node; to delete, *zip* the two child spines back together. Expected restructuring is $O(1)$ and there is no rotation code at all. Gila, Goodrich, and Tarjan’s

zip-zip trees (2023) refine the rank distribution so that expected depth approaches the information-theoretic optimum.

What Durable7 wants from them is not speed. It is *canonicity*.

17.1 Shape from contents

If a node's rank is derived from its key rather than drawn fresh at insertion time, then under one retained coherent rank policy, the tree's shape is a function of its contents (Figure 17.1). Insert the same elements in a different order and you converge on the same topology.

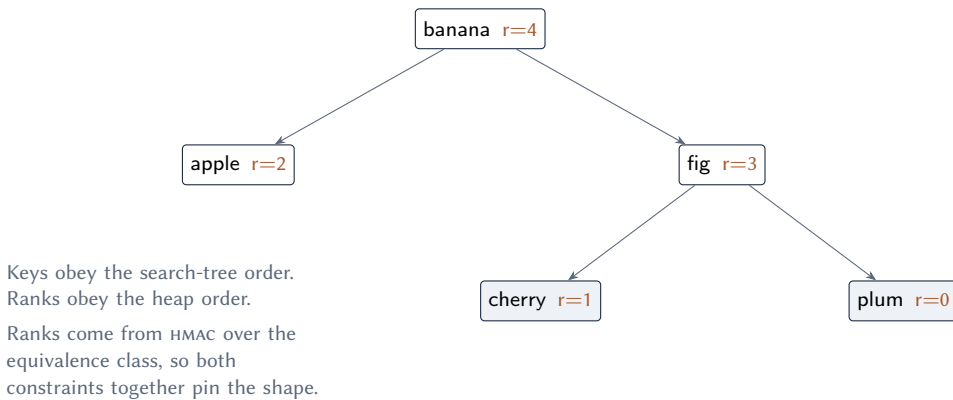


Figure 17.1: A zip-zip tree. Because ranks are derived rather than drawn, the search order and the heap order jointly determine the topology for a given content set under a given policy.

Canonical shape is worth having when a collection is compared far more often than it is modified: memoization keys, configuration fingerprints, cache validity stamps. A memoized root digest rejects most inequality in constant time; shared node identity prunes the equal case; and independently built equal sets fall back to an honest $O(n)$ comparison.

17.2 Getting ranks right

The rank source has to be stable, well distributed, and coherent with the comparer — and this turns out to be where the design gets interesting.

Rank policy obligations

A policy may use an unexposed random key, a public reproducibility seed, or a caller-retained key. An explicit comparer must *also* supply a rank hash that is constant on its equivalence classes; duplicate representatives dynamically check that contract.

Set equality is semantic across policy families, but canonical structural algebra is gated on policy identity. First-representative retention means equivalent values can still retain different concrete objects, so this is *policy-canonical* topology — not a claim of globally unique representation.

What the fixed-width variant does not inherit

This is a practical fixed-width implementation, not the paper's compact representation. Its 64-bit secondary coordinate does not inherit the paper's $O(\log \log n)$ -metadata theorem, and the 64-bit caller hash (32-bit for the fallback) can collide before HMAC ever runs. Expected-depth claims therefore require a well-distributed, stable, equivalence-class-coherent rank source.

A keyed policy resists adaptive rank prediction only while the caller-retained key stays secret; a public seed buys reproducibility, not adversarial security. And persistence changes the cost profile: an ephemeral zip tree changes $O(1)$ expected pointers, but a persistent update still copies its search/zip/unzip path — expected $O(\log n)$ nodes, $O(n)$ under adversarial ranks.

The implementations take the stack-safety problem seriously, because a fully colliding rank chain is a legitimate adversarial input: zip and unzip are explicit-stack rather than recursive, and destruction uses an intrusive non-allocating worklist so that reclaiming a 16,384-node chain neither recurses nor allocates.

Cryptographic sourcing is likewise concrete rather than hand-waved: CNG on Windows and OpenSSL elsewhere for C and C++, RustCrypto HMAC/SHA-256 with `getrandom` for Rust, `crypton` for Haskell, `hashlib` and `hmac` for Python. Rust and Python both pin stable rank hashes explicitly rather than trusting a default hasher, because `DefaultHasher` and `randomized str` hashing are exactly the failure this structure cannot tolerate.

Insertion Order as a First-Class Property

Two collections can hold the same elements and still not be the same list.

Specimen — the neutral Ordered family

Purpose	Sets, maps, and multimaps whose enumeration follows first insertion plus explicit positional operations — neither hash order nor comparison order.
Core	A persistent hash membership/stamp index composed with a persistent ordered sequence. Deliberately independent of any other family.
Identity	Equality policy decides membership and representatives. It never decides position.
Operations	Explicit movement, positional reads, ranges, reversal, stable one-shot sort, receiver-policy set algebra, all six set relations.
Ports	All nine: PersistentOrderedSet, PersistentOrderedMap, PersistentOrderedMultimap.

Insertion order is the order humans expect. It is what a configuration file means, what a UI list means, what a pipeline stage sequence means. Hash order is an implementation detail and comparison order is somebody else's opinion; if the program's meaning depends on the sequence in which things were added, that has to be represented deliberately.

18.1 Two indexes, one meaning

The family composes a hash index — which answers membership and holds representatives — with a persistent ordered sequence, which answers position. The composite obligation from Chapter 5 applies in full: both indexes publish together or neither does, counts agree, and no entry exists in only one.

Ordered-family obligations

Equality policy defines membership, duplicate collapse, stored representatives, algebra, and relations; every result retains the exact receiver policy, including empties.

Construction and duplicate additions retain the *first* representative *and its position* — a duplicate add never moves or replaces anything. Movement is explicit, retains the stored representative, and interprets a supplied destination as the final result index.

Union appends argument-only classes after receiver order; intersection and difference retain receiver order; symmetric difference emits receiver-only then argument-only classes. All algebra and all six relations eagerly enumerate and normalize the complete argument under the receiver policy *before* applying any shortcut, so a late enumeration or policy failure is visible rather than optimized away.

Documented logical no-ops return the receiver; failures publish nothing; private sparse position labels are not API, and a relabel rebuild leaves retained versions untouched.

The map adds one rule that is easy to get wrong and important to get right: setting an existing key changes *only* its payload. It does not replace the stored key representative and it does not move the entry. Range construction retains the first key representative and its first position while the last payload for that class wins. Equality policy controls key identity and never payload equality or sort order.

The multimap nests the same idea: key groups retain first-insertion order, and the distinct values within each group retain their own first-insertion order. Enumeration is grouped, and an empty group is never stored.

On sparse labels and branching

Insertion-ordered structures are usually implemented with sparse integer position labels and midpoint insertion, which occasionally require a wholesale relabel. That relabel is an $O(n)$ -at-threshold amortization — precisely the kind that Chapter 1 warns does not survive branching. The library’s response is to treat the labels as strictly private, to state the worst case per produced version rather than the amortized one, and to guarantee that a relabel rebuild leaves retained versions untouched. The related general-purpose `OrderMaintenanceList` — the Dietz–Sleator structure that answers “does A precede B ?” — remains unbuilt for want of a named consumer (Appendix C).

Priority: Three Structures, Three Reasons

“Give me the smallest one” has more variations than it looks.

Three families in this library answer priority questions, and they are not redundant — each exists because the other two are wrong for a specific need.

19.1 The measured priority queue

Specimen — PriorityQueue

Purpose	The general persistent minimum-priority queue.
Core	A minimum-priority measure over the measured tree.
Bounds	Amortized: $O(1)$ peek, $O(\log n)$ enqueue and dequeue, $O(\log \min)$ meld.
Ties	Equal priorities drain in insertion order.
Ports	All nine.

This is the default, and it is a measure facade: cache the minimum priority of each subtree, and locating the front entry is a descent. Melding two queues is concatenation. It costs nothing conceptually beyond the measure framework, which is exactly why it is the default.

```
var queue = PriorityQueue<string, int>.Empty
    .Enqueue("slow", 10).Enqueue("fast", 1).Enqueue("normal", 5);
```

```

if (queue.TryPeek(out var element, out var priority))
{
    // element == "fast"; priority == 1
}

```

19.2 The Brodal–Okasaki heap

Specimen — BrodalOkasakiHeap

Purpose	Worst-case rather than amortized latency for insert, meld, and minimum.
Core	Bootstrapped skew-binomial queues (Brodal and Okasaki, JFP 1996).
Bounds	$O(1)$ <i>worst-case</i> insert, meld, and find-minimum; $O(\log n)$ worst-case delete-minimum.
Meld	Requires comparer object identity — even when one operand is empty.
Enumeration	Unspecified structural order. Equal elements have no stable tie order. Delete repeatedly for sorted output.
Ports	All nine.

The measured queue has the same bounds *amortized*. This heap’s entire reason for existing is the word “worst-case”: if a latency budget is a hard requirement rather than an average, an amortization spike is a bug. The implementation is a direct port of the fused bootstrapped skew-binomial representation, with comparison-count tests that assert the operation bounds rather than merely believing them.

One algorithm, two spellings

The C and C++ `skew_meld` implementations consolidate the child forest by rank buckets with carry; C#, Haskell, Kotlin, and Rust spell the equivalent step as `uniquify` followed by `unionUnique`. This is an intentional native implementation-shape difference. Both produce the same valid ranked-tree multiset, the same minimum, and the same public bounds — and the repository records the di-

vergence explicitly so that a future reader does not mistake it for a porting error.

Why not strict Fibonacci or hollow heaps?

Strict Fibonacci heaps (Brodal, Lagogiannis, and Tarjan, STOC 2012) and hollow heaps (Hansen, Kaplan, Tarjan, and Zwick, ICALP 2015) achieve optimal decrease-key bounds through aggressive pointer mutation — and persistent path copying destroys exactly the surgery they depend on. They are recorded as explicit non-goals rather than as omissions.

19.3 The priority search queue

Specimen — PrioritySearchQueue

Purpose	Keyed lookup <i>and</i> minimum-priority removal <i>and</i> a combined key-range plus priority-threshold query, in one structure.
Core	A persistent AVL ordered by key, each node caching its subtree's minimum-priority winner.
Bounds	$O(1)$ minimum; $O(\log n)$ keyed update, removal, and delete-minimum; $O(\log n + v)$ range query where v counts nodes pruning cannot exclude.
Ties	Priority ties break by key order — deterministic, not merely stable.
Ports	All nine.

A heap cannot find a key. A map cannot find the minimum. The priority search queue does both, and adds the query that motivates it: *which entries in this key range have priority at most p ?* That is the scheduler question, the expiry question, and the task-queue question.

```
var jobs = PrioritySearchQueue<string, int, string>.Create(
    StringComparer.Ordinal, Comparer<int>.Default)
    .SetItem("compile", 3, "Build the solution")
    .SetItem("deploy", 8, "Publish artifacts")
    .SetItem("audit", 1, "Check invariants");
```

```

var next    = jobs.Minimum;                                     // "audit", 1
var urgent = jobs.EnumerateAtMost("a", "z", maximumPriority: 3);
jobs = jobs.DeleteMinimum(out var removed);

```

A deliberate departure from Hinze

Hinze's ICFP 2001 priority search queue uses *priority-search pennants*: a loser tree over a weight-balanced search tree, whose threshold-only *at-most* operation has the output-sensitive bound $\Theta(r(\log n - \log r + 1))$ for r results.

Durable7 deliberately uses a different representation — a persistent AVL node carrying its own entry plus a cached subtree winner — and claims the bounds that representation actually delivers: $O(\log n)$ keyed operations, $O(1)$ minimum, and $O(\log n + v)$ for the inclusive key-range and priority-threshold traversal, where $v \leq n$ is the number of visited nodes that pruning could not exclude. The worst case is therefore $O(n)$ for a dense result, and no pennant-specific output bound is claimed. Choosing a simpler representation and reporting its real bounds is treated as preferable to importing a bound the code does not earn.

Intervals

Everything that has a beginning and an end wants one of these.

Specimen — IntervalTree and PersistentIntervalMap

Purpose	Ordered collections of intervals with overlap and point-stabbing queries (tree), and the same with one payload attached to each complete interval key (map).
Core	Interval annotations in the measure: typically count, last low endpoint, and maximum high endpoint.
Pruning	A subtree whose maximum high endpoint lies below the probe cannot contain an overlap, so the descent skips it entirely.
Map key	The <i>complete</i> (low, high) pair. Intervals sharing only one endpoint stay distinct.
Ports	All nine.

The interval tree is the measure framework's most elegant payoff. Put the maximum high endpoint of a subtree in its measure, and a stabbing query becomes a descent that prunes any subtree which *provably* cannot contain a match — without looking at a single interval inside it.

Interval obligations

Interval construction must define whether empty or inverted intervals are rejected, normalized, or accepted. Endpoint comparison policy defines ordering,

equality, overlap, and containment. Query APIs must document endpoint inclusivity and behavior when several overlaps exist. Removal must state whether it removes one match, all matches, or a comparison-equivalent representative.

The interval *map* adds a payload and, with it, a second index: an exact lookup index keyed by the complete interval pair, alongside the augmented search index. The composite rule applies — both contain the same intervals, payloads, and count. Replacing a payload preserves the stored interval representative and neither duplicates nor moves the key; stabbing and overlap queries return entries in the local deterministic interval order.

PART V

Narrow Questions

Ancestry as a Sequence

The sequence is not stored anywhere in particular. It is two boundaries on a path that already exists.

Seven of this library’s collections did not arrive the way the others did. The finger tree, the hash trie, the RRB vector and their relatives are implementations of published structures: the paper came first, the library followed. These seven began instead as scoped design studies — each one a normative proposal written before any code, asking whether a narrow question that the general structures answer badly could be answered well by giving something up.

The proposals share a discipline that is worth naming before any of the structures are described, because it is the reason their claims can be trusted.

How a research-derived collection is argued

It answers a narrow question. Every proposal states the question, names the general structure that answers it worse, and says plainly when the general structure is the right choice anyway.

It declares its non-goals. Concatenation of unrelated histories, point replacement, middle editing, confluent merge — whatever the design cannot do is listed, and no comparison table is ever allowed to count an omitted operation as a fast one.

It separates two complexity statements and never conflates them. One is the bound the shipped code delivers. The other is the bound the same reduction would deliver over a stronger component that exists in the literature but is not implemented here. Both appear; they are never merged into a single optimistic number.

Its novelty claim is scoped. “Not found in a targeted literature search” is written

as exactly that, and never as priority.

The seven are: `AncestralSliceQueue` and `BilateralAncestralDeque`, the subjects of this chapter; `ContextualRankSequence` (Chapter 22), which ranks and selects events whose occurrence is not a property of an element but of the finite-state context to its left; `PersistentDeltaMap` (Chapter 23), an ordered map that maintains its own exact checkpoint-relative change set and emits it in time proportional to the number of changes rather than the size of the map; `PersistentRunDeltaVector`, in the same chapter, which does the same for a fixed-length vector and reports *maximal dirty runs* rather than dirty positions; `PersistentMonotoneActionHeap` (Chapter 24), a meldable min-heap that applies a monotone transformation to every entry in $O(1)$ and still melds in $O(1)$ afterwards, even when the transformation has no inverse; and `PersistentAncestralConnectionForest` (Chapter 25), an insertion-only connectivity structure that can name the earliest version on a lineage at which two vertices became connected, for the price of one ordinary connectivity query.

All seven were prototyped in C#, lived for a time in `*.Experimental` namespaces, and were then promoted and ported to all nine languages — the TypeScript port last, in August 2026. The proposals in `docs/proposals/` remain their normative documents; where this book states a bound, that bound comes from the shipped code and agrees with the proposal that predicted it.

This chapter takes the two that share a substrate.

21.1 A forest that only grows leaves

Both structures rest on the same idea, and it is a strange one: *a sequence value need not contain its elements*. It can be a pair of boundaries on a path through a structure that somebody else owns.

The structure that owns the path is an *incremental level-ancestor arena*. It is a forest that grows one leaf at a time and answers exactly one interesting question:

Which ancestor of node v sits at absolute depth d ?

Specimen — the incremental ancestor arena

Purpose	An append-only monotone forest of immutable nodes, addressed by stable handles, answering level-ancestor queries. The shared substrate of the two ancestry-interval sequences.
Core	MyersIncrementalAncestorArena: Myers' two-link applicative-stack scheme — one parent link plus one coalesced jump link per node — over a store of odd-sized blocks.
Leaf add	$O(1)$ amortized. A call that crosses a block boundary pays $\Theta(\sqrt{M})$ for the next odd block.
Ancestor query	$O(\log M)$ worst case after M published nodes.
Primitive reads	Depth, parent, and value are $O(1)$; the consuming collections' bounds assume it.
Space	$O(M)$ total; $M + 1 + O(\sqrt{M + 1})$ record slots, with an $O(\sqrt{M + 1})$ -entry directory.
Mutability	The arena is mutable. It accumulates nodes in place, under one private lock, while every collection value built on it stays immutable.
Ports	All nine. Haskell has no arena at all — see below.

Three properties of the arena do the real work. Nodes are *immutable* once published, so a handle retained forever keeps reading the same answer. Handles are *never recycled*. And a leaf may be added below *any* previously published node, not only below the most recent one — which is what makes the arena a persistence substrate rather than a log. Appending to an old version does not extend the newest branch; it starts a sibling branch, and every earlier version's root-to-node path is untouched. A distinguished *bottom* node sits at depth -1 , carries no value, and is the parent of every depth-zero node.

21.1.1 Two links per node

The shipped backend is Eugene Myers' applicative random-access stack from 1983, adapted from a single list to the root-to-node list of every branch in the forest. Each node stores its parent, its depth, its value, and one additional *jump* link with the distance that jump covers.

The jump is chosen when the node is created, in constant time, by a rule that

coalesces: if the parent's jump target itself has a jump no longer than the parent's, the new node inherits the far end of that pair and records the summed distance; otherwise the new node's jump is a plain unit hop to its parent. A query then climbs from the starting node toward the root, taking the jump whenever the jump does not overshoot the requested depth and the parent link otherwise. The coalescing rule keeps the reachable distances growing geometrically, so a query finishes in $O(\log M)$ hops.

That logarithm is easy to lose by accident and impossible to see in a correctness test — a broken jump rule returns exactly the same answers, only slower. The arena therefore exposes a statistics record with deterministic hop counters, and the test suites assert an envelope over them. The mutation check is worth recording: delete the coalescing case, and a query that took two hops takes 32,768, against an asserted bound of 64, while every answer stays correct.

21.1.2 Odd blocks, and the spike the library decided to keep

Node records live in blocks of sizes 1, 3, 5, 7, . . . The reason is arithmetic: the capacities before block r sum to $1 + 3 + \dots + (2r - 1) = r^2$, so a zero-based handle i lives in block $\lfloor \sqrt{i} \rfloor$ at offset $i - \lfloor \sqrt{i} \rfloor^2$. Addressing is $O(1)$ with no indirection beyond the directory, unused slots after M nodes are $O(\sqrt{M})$, and the directory itself has $O(\sqrt{M})$ entries.

The cost of that arithmetic is a spike. When a block fills, the next one — of size roughly $2\sqrt{M}$ — is allocated in a single call, and in a managed runtime allocating an array also zeroes it. So leaf addition is $O(1)$ *amortized*, and one unlucky call in every $\Theta(\sqrt{M})$ pays $\Theta(\sqrt{M})$.

A recorded ruling, not an oversight

The spike can be removed. The mechanism is real-time two-table migration: keep the old and new backing tables live at once and move a constant number of slots on every operation, so no single call performs a large copy. (The obvious cheaper fix — pre-initializing the next block incrementally — does not work in a garbage-collected language, precisely because allocation zeroes.)

Migration was analyzed, found implementable in all eight arena workspaces, costed — permanent extra complexity in the hottest path of eight implementations, to shave an allocation the memory allocator performs anyway — and **declined by the maintainer**. The $\Theta(\sqrt{M})$ boundary call is therefore the accepted shared contract, and every arena port carries the same two-part sentence: $O(1)$ amortized, with a block-boundary call paying $\Theta(\sqrt{M})$ for the next odd

block. The declined alternative is recorded so the decision can be revisited rather than rediscovered.

Haskell is the exception, and it is an interesting one. Its port has no arena object at all: a node is an ordinary immutable heap value that *is* its own handle, with the same two links and the same coalescing rule. There is no block store, so there is no boundary call — leaf addition is $O(1)$ *worst case*. There is also no lock, no handle-recycling question, and no way to present a node to the wrong forest, because there is no forest object to present it to. Under the library’s parity rule a stronger guarantee anywhere becomes the target everywhere, so this one is documented explicitly as *exceeding* the shared profile rather than being allowed to diverge from it silently.

21.1.3 Why the seam is parameterized

The arena is a public interface, not a private detail, and the collections above it state their bounds in terms of two parameters rather than in absolute terms: U , the time for one leaf addition, and Q , the time for one level-ancestor query, both after M published nodes. The interface additionally requires that depth, parent, and value reads be $O(1)$, because every parameterized bound assumes it.

This is not decoration. Alstrup and Holm proved in 2000 that a rooted tree updated *only by adding leaves* — exactly this arena’s update — admits level-ancestor queries and leaf additions in $O(1)$ *worst case* with linear space. Instantiate the seam with that structure and $U = Q = O(1)$: every scalar operation of both collections in this chapter, including indexed access and slice construction, becomes constant worst-case time.

The reduction is stated, not implemented

No Alstrup–Holm backend ships here. The shipped Myers arena gives $U = O(1)$ amortized and $Q = O(\log M)$, and nothing in the library upgrades those to worst-case constants. The stronger statement is a reduction to a published result; it is not evidence about this code.

Two tempting substitutions are also invalid. Static level-ancestor preprocessing cannot be used, because retained handles are extended online after preprocessing would have finished. Binary lifting gives logarithmic update time and space, so it does not establish the constant bound either. And Alstrup and Holm’s more general dynamic-forest result, which permits edge additions, carries an

inverse-Ackermann aggregate bound rather than a constant one — it is a different theorem and must not be substituted into the table.

21.2 AncestralSliceQueue

Specimen — AncestralSliceQueue

Purpose	A fully persistent FIFO queue whose every value — including every prefix, suffix, slice and split result — is one contiguous interval of one root-to-node path, and remains appendable.
Core	The pair (tail node, low depth), plus a redundant count cache, over a shared incremental ancestor arena.
Algebra	Append at the right; read or remove either visible end; index; Take, Drop, Slice, SplitAt; enumerate.
Complexity	AddLast costs U . First, the indexer, Take, Slice and a nontrivial SplitAt cost Q . Last, RemoveFirst, RemoveLast and Drop are $O(1)$.
Limits	No prepend, no point replacement, no middle edit, no concatenation of unrelated histories. Arena space is charged to every historical append, not to the visible length.
Ports	All nine.

A handle is two numbers and a node. Let t be the *tail* — the node holding the last visible value — and let ℓ be the *low depth*, the absolute depth of the first visible value. Then the queue denotes the labels of the ancestors of t at depths $\ell, \ell + 1, \dots, \text{depth}(t)$, in that order, and its length is $\text{depth}(t) - \ell + 1$. The count is cached, so Count and IsEmpty never touch the arena at all; the cache's agreement with the formula is part of the representation invariant.

Every operation is then arithmetic on those two boundaries. AddLast publishes a leaf below t and keeps ℓ . RemoveFirst adds one to ℓ . RemoveLast replaces t by its parent. Drop(k) adds k to ℓ — in constant time, regardless of k . Take, Slice and the prefix half of SplitAt need a new tail, which is one ancestor query. Nothing is copied, nothing is rebalanced, and every result is itself a queue that can be appended to, which is the point: a slice of a history is not a read-only view of that history but a new branch point in it.

21.2.1 The anchored empty

The subtle part is what happens when the window closes.

The invariant permits $\ell = \text{depth}(t) + 1$, which denotes the empty sequence. But t is still there, and it is still meaningful: it is the node *immediately before* the window. The proposals call it the *anchor*.

The rule this buys is exact and worth stating precisely: appending to an empty slice yields a queue containing exactly the new value, and never resurrects a discarded prefix. Appending publishes a leaf below the anchor, at depth ℓ , and the window — which starts at ℓ — admits that one node and nothing above it. Without the anchor there would be no node to append below and no way to keep zero-length results branchable; with a shared process-global empty value there would be one immortal arena coupling every unrelated workload in the process, which is why the factories always create or accept an arena and no such global exists.

What is *not* guaranteed is which node the anchor is — and it genuinely differs (Figure 21.1). Drain a queue from the front, and ℓ climbs past the tail: the retained anchor is the queue’s own last value. Drain the same queue from the back, and the tail walks up the path: the retained anchor is the node before the queue’s original first value. Both denote the empty sequence. Both obey the rule. But an append to the first lands as a sibling of whatever already grew below the old tail, and an append to the second lands high on the path as a sibling of the queue’s old front — two different branches of the arena, from two values that compare as equally empty. Empty values, in other words, have provenance; the library documents the rule as semantics and the anchor identity as provenance, and does not promise the latter.

21.2.2 Which operations ask the arena

The query discipline is stated exactly, because the difference between a constant-time and a logarithmic operation here is precisely whether it needs an ancestor.

Operation	Cost	Shipped, Myers-backed
AddLast	U	$O(1)$ amortized
Count, IsEmpty, Last	$O(1)$	$O(1)$ worst case
RemoveFirst, RemoveLast, Drop, TryRemoveLast	$O(1)$	$O(1)$ worst case
First, indexer, TryRemoveFirst	Q	$O(\log M)$ worst case
Take, Slice, nontrivial SplitAt	Q	$O(\log M)$ worst case

enumerate n visible values	$\Theta(n)$	$\Theta(n)$ time, $\Theta(n)$ transient slots
------------------------------	-------------	---

Two rows deserve comment. Last is free and First is not, which looks like an asymmetry until one remembers what the handle stores: the tail is a node, so its value is a direct read, while the front is an ancestor selected jointly by the tail and the low depth. And RemoveFirst is free while TryRemoveFirst is not, for the same reason — the removal moves an integer, but returning the removed value requires the query. Caching the front would not fix this; it would only move the query into the next RemoveFirst. Under a constant-query backend both become constant, which is the whole argument for parameterizing the seam.

Boundary cases are specialized: Take(Count), Drop(0), a slice whose result keeps the current tail, and a split at Count all return without querying. The table gives the general case.

Enumeration walks parent links from the tail into a temporary array and yields that array forwards, so it performs *zero* ancestor queries at $\Theta(n)$ time and $\Theta(n)$ transient slots. With a constant-query backend one could instead run one query per output and keep $O(1)$ iterator state. Both are output-optimal in time; the transient-space difference is part of the backend contract, and the shipped enumerator always buffers.

21.3 BilateralAncestralDeque

Specimen — BilateralAncestralDeque	
Purpose	A fully persistent deque built from two oppositely oriented ancestry intervals, closed under reverse and arbitrary slicing.
Core	Two <i>segments</i> over one shared arena. A segment is (anchor, base, tail, count); the handle’s value is the reversed left segment followed by the right one.
Reverse	$O(1)$ — the two segments change places. No flag, no rebuild.
Ends	Insertion at either end costs U . Both endpoint <i>reads</i> are query-free.
Removal	$O(1)$ on its own nonempty side; at most one query when it crosses the center.
Index / slice	At most one query to index; at most two to slice or split — including when the range crosses the center.

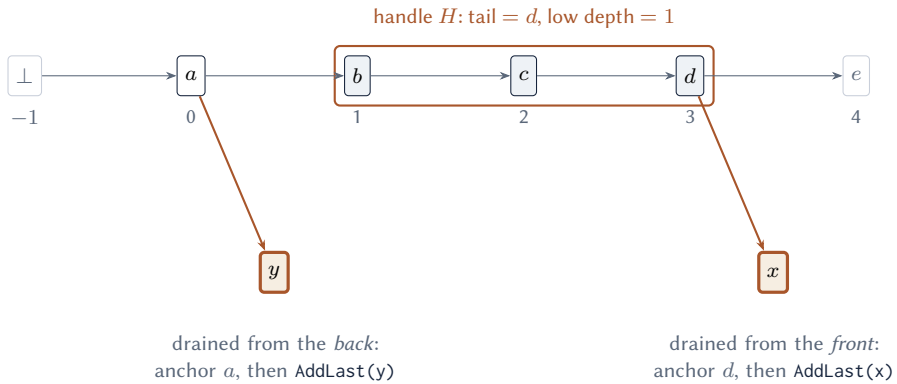


Figure 21.1: The anchored-empty rule. The handle H denotes $[b, c, d]$; node e is history below the window that H cannot see. Emptying H from either end leaves the node immediately before the window as the anchor — but that is a in one case and d in the other, so the two empty values append into different branches. Both are empty, and neither can resurrect a discarded value.

Limits

No unrelated concatenation, no point replacement, no middle edit.
Same retained-history model as the queue.

Ports

All nine.

The classical two-stack deque keeps a reversed front stack and a forward back stack, and pays for the arrangement by occasionally transferring one stack into the other. This structure keeps the arrangement and abolishes the transfer.

Each arm is an interval on some root-to-node path, exactly as in the queue, but with two extra cached fields. Written (anchor, base, tail, count), the invariant is that the anchor is an ancestor of the tail, that the count is the difference of their depths, and — when the arm is nonempty — that the base is the anchor's child on that path, that is, the first node strictly inside the interval. The handle's logical value is the left arm read *in reverse*, followed by the right arm read forwards.

Everything follows from that orientation. Adding at the front publishes a leaf below the left tail; because the left arm is read in reverse, the newest node reads first, which is precisely a prepend. Adding at the back publishes a leaf below the right tail, which is an ordinary append. Removing from an arm's own end moves its tail to its parent.

Removing from the opposite side of an arm advances its anchor to its cached base.

21.3.1 Reverse is an exchange

$\text{reverse}(\text{reverse}(L) ++ R) = \text{reverse}(R) ++ L$. The right-hand side is the representation of the handle whose left arm is R and whose right arm is L . So `Reverse` constructs a new handle with the two segment records exchanged and the count unchanged: no orientation flag to interpret at every subsequent call, no materialization, and no lazy mirror to force. It is the cleanest reversal in the book, and it is cheap for a structural reason rather than a bookkeeping one.

21.3.2 Both ends, without asking

The base field is redundant — it is recoverable by a query — and it is the reason both endpoint reads are free. For the left arm the logical first element is at the tail and the logical last is at the base; for the right arm those roles are swapped. Whichever arm is empty, one of the four cached nodes holds the answer, so `First` and `Last` are value reads.

That extends to indexing. Four positions are answered from cached endpoints with no query at all: index 0, the last index of the left arm, the first index of the right arm, and the last index of the deque. Any other position costs exactly one query — into the left arm counting down from its tail, or into the right arm counting up from its anchor.

It is worth putting this next to the queue, because the two structures made opposite trades with the same budget. The queue does not cache its front, so reading the front costs a query while advancing it is free. The deque caches its bases, so reading either end is free while a removal that crosses the center — one that must retire an arm's base and cache the next one — costs a query. Neither is a mistake, and under an Alstrup–Holm backend the distinction evaporates entirely; with the shipped arena it determines which of your operations carry the logarithm.

21.3.3 Slice closure, and the ceiling of two

The central property is that the representation is *closed* under arbitrary contiguous slicing: a slice of a bilateral handle is another bilateral handle, never a third fragment and never a rebuild. That is true because a contiguous range intersects each arm in at most one contiguous piece.

The at-most-two-query ceiling follows from a case analysis that repays reading, because the crossing case is the one that looks expensive and is not.

The requested range	What must move
Wholly inside one arm	Both endpoints of that arm's interval move; the other arm becomes empty. Two queries, fewer when a moved endpoint coincides with a cached one.
Crossing the center	The left intersection is a <i>suffix</i> of the reversed left arm, so it keeps that arm's anchor and base and moves only its tail. The right intersection is a <i>prefix</i> of the right arm, so it keeps that arm's anchor and base and moves only its tail. One query per arm.
Empty	No query.

So the expensive-looking case is the cheap one: crossing the center costs one query in each arm precisely because each intersection inherits one of its endpoints unchanged. Take and Drop are slices. SplitAt is expressed as a Take plus a Drop and still makes at most two queries in total, because the two results move complementary endpoints at the same boundary. The focused test suites assert the exact maximum query count per operation rather than a ceiling, which is how a claim like this stays true through nine ports.

Operation	Cost	Shipped, Myers-backed
AddFirst, AddLast	U	$O(1)$ amortized
Count, First, Last, Reverse, Clear	$O(1)$	$O(1)$ worst case
RemoveFirst, RemoveLast	$O(1)$; at most one Q across the center	$O(1)$; $O(\log M)$ across the center
indexer	at most one Q	$O(\log M)$ worst case
Take, Drop, Slice, SplitAt	at most two Q	$O(\log M)$ worst case
ToArray, enumerate n values	$\Theta(n)$, no queries	$\Theta(n)$ time, up to $\Theta(n)$ transient slots

The deque handles empties differently from the queue, and the difference is declared rather than incidental. An arm that empties while the other arm still holds values collapses its anchor, base and tail onto one node and keeps it. But a deque that empties completely — and the value returned by Clear — resets both arms to the arena's bottom node, so the next insertion starts a fresh branch at the root rather than continuing

under an old anchor. Exact anchor provenance is explicitly *not* part of the deque's public value semantics, where for the queue the anchoring rule is.

21.4 What the family costs

Both structures buy their bounds with the same three concessions, and the proposals insist on them being visible.

The logarithm is over the history, not the value

Every $O(\log M)$ in this chapter is logarithmic in M , the number of nodes the arena has *ever* published — including nodes on branches no application variable can still reach. It is not logarithmic in the length of the handle you are holding. A thousand-element slice of a ten-million-append history indexes at the cost of the history.

Nothing is ever reclaimed, and one handle retains everything

The arena keeps every node it has published, with its payload, until the arena itself becomes unreachable. Dropping a prefix does not release nodes; releasing a handle does not release nodes; garbage collection of a value must not be read as reclamation of its history. And because every handle retains its arena, one surviving handle retains the entire history — which is normal for an arena-backed persistent structure and unacceptable for an unbounded service. Reclamation would require tracing from live handles, explicit epochs, or an ancestor index that tolerates deletion. None of the three is here.

The third concession is the algebra. Neither structure concatenates unrelated histories, replaces a point, or edits the middle, and neither is confluent persistent: two versions that branched apart cannot be merged. Equality is a further casualty — two handles can enumerate identical values from different tails, different anchors, or different arenas, so reference identity is neither content equality nor a provenance test, and no constant-time sequence equality or canonical empty is promised.

The concessions are the mechanism, not an accident of implementation. It is exactly because there is no concatenation and no point update that a value can be two boundaries on somebody else's path, and exactly because a value is two boundaries that indexing and slicing reduce to one ancestor query apiece. When the missing op-

erations are needed, the library says so plainly and points at the general structures: `FingerTreeDeque` (Chapter 9) for a persistent sequence with concatenation and arbitrary edits, and `ReversibleDeque` (Chapter 14) when reversal is the primary operation over a full sequence algebra. These two are justified only when branching append histories with subrange and indexed access dominate, and when the operations they do not have are genuinely unnecessary.

Lifting a Machine into a Measure

When a summary cannot know what came before it, make it answer for every possible before.

Specimen — ContextualRankSequence

Purpose	A persistent sequence that ranks and selects <i>events</i> whose occurrence depends not on an element alone but on the finite context preceding it — the comma that separates fields here and is quoted data there.
Core	The measured finger tree under a lifted measure: for every subtree <i>and every one of the machine's states</i> , the outgoing state and the number of events emitted.
Algebra	A finite deterministic machine over s states emitting nonnegative event counts. The lifted summary is an ordered monoid — associative, and emphatically not commutative.
Operations	Evaluate, EvaluatePrefix, EventRank, TrySelectEvent, over the ordinary positional surface: index, insert, replace, remove, split, slice, concatenate.
Complexity	$O(1)$ whole-sequence evaluation; $O(s \log n)$ amortized for prefix evaluation, rank, select, and every edit; $O(s)$ amortized endpoint updates; $O(s \log \min(n, m))$ amortized concatenation; $O(sn)$ to build and to store.
Policies	The machine is compile-time in C#, C++, and Rust; a retained runtime object in Kotlin, TypeScript, and Python; a record of functions in Haskell; an identified callback struct in C and OCaml.
Ports	All nine.

Every measure in Chapter 10 shares an assumption so quiet that it is easy to miss: an element's contribution to the summary is a property of the element. A character contributes one to a length; a key contributes itself to a maximum; a newline contributes one to a line count. Nothing about the neighbors enters into it, which is exactly why the contributions can be combined in any grouping and cached on every subtree.

Now take a rule that breaks the assumption. In a CSV line, a comma is a field separator — unless it falls inside a quoted field, in which case it is data. The element is identical in both cases. What differs is everything that came before it. No fixed weight is correct for a comma, so no ordinary measure can rank commas, and the whole apparatus of measure-guided descent is unavailable for a question that is otherwise a textbook rank-and-select.

This chapter is about the way out, and it is the most elegant construction in the repository.

22.1 Why the obvious repairs are linear

Keep the characters in any persistent sequence and run a two-state scanner over them. That works, and it is what almost everyone does. Its cost is structural: to know whether the comma at position i is a separator you must know the scanner's state entering position i , and that state is a function of all i preceding characters. Asking for the event rank at a boundary p costs $\Theta(p)$; asking which element emitted the k -th event costs $\Theta(n)$ in the worst case; and a single insertion at the front invalidates the entering state of everything after it.

The tempting fix — cache the entering state on each node — fails for the same reason, and it fails completely rather than partially. A cached entering state is valid for exactly one prefix. Prepend one quotation mark and every cached entry in the tree is wrong. What is expensive here is not the scan; it is the *dependence of the cache on the prefix*. Any repair that leaves that dependence in place will be linear in disguise.

Binary search does not rescue it either. One could search for the boundary at which the running event count first exceeds k , but evaluating the predicate at a candidate boundary is itself a prefix replay, so the first probe alone costs $\Theta(n)$.

22.2 The lift

`ContextualRankSequence<TElement, TMachine>` removes the dependence instead of trying to maintain it. Its measure caches not the answer, but the answer *as a function*

of the entering state.

The policy supplies a finite deterministic machine: a state count s , and a transition that consumes one element from one state and reports the outgoing state together with a nonnegative count of emitted events. A subtree's summary — its *effect table* — is then two arrays of length s plus the ordinary element count:

$A.\text{next}(q)$ — the state after consuming A from q $A.\text{events}(q)$ — the events emitted while consuming A from q

tabulated for every q in $0 \dots s - 1$. Ordered composition feeds the left table's outgoing state into the right table:

$$\begin{aligned}(A \otimes B).\text{next}(q) &= B.\text{next}(A.\text{next}(q)) \\ (A \otimes B).\text{events}(q) &= A.\text{events}(q) + B.\text{events}(A.\text{next}(q)) \\ (A \otimes B).\text{count} &= A.\text{count} + B.\text{count}\end{aligned}$$

The identity maps every state to itself, emits nothing, and holds no elements. That is a monoid, and a monoid is precisely what the measured finger tree wants. Associativity is not a coincidence and needs no clever argument: the state component is function composition, which is associative, and the event component is a sum of terms each indexed by the state its subtree is entered in, so both parenthesizations end at $C.\text{next}(B.\text{next}(A.\text{next}(q)))$ with the identical three event terms. The algebraic shape is the machine's transition monoid decorated with additive output — the same object one composes when composing deterministic weighted transducers, in Mohri's sense.

The s columns are the price of not knowing the prefix, and it is worth naming the trade plainly: a factor of s in storage and in every composition buys the disappearance of the prefix dependence. Nothing else changes. The tree is the same measured finger tree that carries a size or a maximum; only the monoid is new.

22.2.1 Not commutative, and it matters

Effect-table composition is associative but decidedly not commutative. $A \otimes B$ and $B \otimes A$ generally disagree in *both* components — in Figure 22.1 they report event counts of five and two from the same starting state. The left operand chooses which row of the right operand's table is read, and swapping the operands changes the choice.

This is not a footnote. A measured tree that combines summaries in any order other than sequence order, or that reuses a cached summary for a subtree it has mirrored,

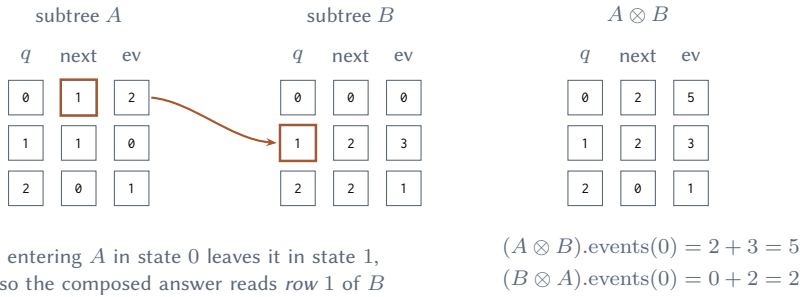


Figure 22.1: An effect table is a subtree’s entire behavior, tabulated for every state it might be entered in. Composition uses the left table’s outgoing state as a row index into the right table — which is why the order of the operands matters, and why no part of the construction ever needs to know the real entering state in advance.

silently produces a wrong answer here, and the answer is wrong in a way no type checker sees. The library is scrupulous about ordered combination for exactly this class of measure, and the discipline has been tested by events: when the complexity-parity campaign gave the dequeues an $O(1)$ lazy reversal, Python’s first implementation reused each mirrored node’s cached summary — correct only when `Combine` is commutative — and was rewritten to match TypeScript’s, which recombines the mirrored parts in mirrored order and is correct under every monoid, verified by a probe whose measure is a string and whose combine is concatenation. The contextual sequence is the kind of measure that makes that difference matter.

22.3 Rank and select in one descent

Because event counts are nonnegative, the running event total is monotone along the sequence, and so the predicate

$$P.events(q_0) > r$$

is false and then true as prefixes grow — a lawful split predicate in the sense of Chapter 10. Selecting the element that emitted event r is therefore one measure-guided descent, with no binary search over boundaries and no prefix replay anywhere.

What comes back is not merely an index. The summary immediately before the located element already contains everything the caller needs, so `TrySelectEvent` reports five things at once: the index of the emitting element, the event’s ordinal within that

element's transition (a single transition may emit several events), the machine state before the element, the state after it, and the transition's total event count. Rank is the same descent stopped at a boundary: EventRank is EvaluatePrefix's event component, and the two are exact inverses — adding the ordinal within the transition to the preceding prefix's count reconstructs *r*.

Notice which argument the initial state is. It is a parameter of the *query*, not of the structure. Because every subtree is tabulated for every state, the same stored sequence answers “what if the scanner arrives here already inside a quoted field?” at no extra cost and with no rebuild.

22.3.1 A scanner, and its answers

The machine that motivates the structure is two states wide.

```
readonly struct QuotedComma : IContextualEventMachine<char>
{
    public static int StateCount => 2;    // 0 = outside quotes, 1 = inside

    public static ContextualEventTransition Transition(int state, char value) =>
        value switch
        {
            '"' => new(1 - state, 0),      // toggle, emit nothing
            ',' when state == 0 => new(state, 1), // a real separator
            _ => new(state, 0)
        };
}

var text = ContextualRankSequence<char, QuotedComma>.Create("\"a,b\",c,d".AsSpan());
```

Index those nine characters from zero: the quotation marks are at positions 0 and 4, and the commas at 2, 5, and 7. One of the three commas is data and two are separators, and the structure knows which is which from either starting context:

```
text.Evaluate(0);    // (FinalState 0, EventCount 2)
text.Evaluate(1);    // (FinalState 1, EventCount 1)
text.EventRank(5, 0); // 0 -- no separator among the first five characters
text.EventRank(6, 0); // 1

text.TrySelectEvent(0, 0, out var first);
// (ElementIndex 5, EventIndexInElement 0, StateBefore 0, StateAfter 0,
//  ElementEventCount 1)
```

```
text.TrySelectEvent(0, 1, out var other);
// other.ElementIndex == 2 -- entering mid-quote, the inner comma is the separator

var edited = text.Insert(0, '');
// text is unchanged; edited ranks every later comma differently
```

Read the last two calls together. From the outside, the first separator is the comma at index 5 and the comma at index 2 is data. Entering already inside a quoted region, the first separator is the comma at index 2 — and the answer comes from the same cached tables that produced the first one, in one descent, with the machine consulted at most once. Prepending a quotation mark produces a new version with different ranks throughout while leaving the original version and its cached answers exactly as they were.

22.3.2 Writing your own machine

The recipe generalizes to anything expressible as a finite deterministic transducer with nonnegative output counts, and the proposal lists the family the design was drawn from: token or record boundaries in a finite-state lexer, frame boundaries in an escaped byte stream, delimiters outside quoted or commented regions, transitions into a selected protocol or workflow state, and Unicode text boundary policies once compiled to a finite-state machine — Annex #29 calls out the random-access difficulty explicitly, and this is one answer to it.

Four obligations make a machine lawful. Bound the context, so that a state count exists at all. Make the transition total and deterministic on every state, not merely on the states you expect to reach. Emit nonnegative counts, or select stops being monotone and the descent stops being meaningful. And if the rule needs *right* context, compile it into a streaming machine or pair it with a reverse machine yourself — the structure does not hide that transformation, and says so.

Every state, not just the reachable ones

Because a leaf's table is filled for all s states when the element is stored, the transition is validated for all s states too. A machine that returns an out-of-range state or a negative count only from state 3 is rejected the moment an element is inserted, even if no version of the sequence ever enters state 3. The failure surfaces where the bug is, not months later in a query.

22.4 What it costs

Let n be the element count and s the machine's state count.

Operation	Bound	Why
Evaluate	$O(1)$	one read of the cached root table
EvaluatePrefix, EventRank	$O(s \log n)$ amortized	$O(\log n)$ levels, one $O(s)$ composition each; $O(1)$ at either endpoint
TrySelectEvent	$O(s \log n)$ amortized	one monotone descent
Indexing, SetItem, Insert, RemoveAt, SplitAt, GetRange	$O(s \log n)$ amortized	measured locate or split, then a path copy
Prepend, Append	$O(s)$ amortized, $O(s \log n)$ worst case per call	digits absorb the work; see below
Concat	$O(s \log \min(n, m))$ amortized	the suspended middle recursion stops at the shallower operand
Create, CreateRange	$O(sn)$	one table per element
Enumeration	$O(n)$	a traversal stack, composing nothing
Storage	$O(sn)$	every node and every element caches an s -wide table; a changed spine adds $O(s \log n)$

Two of those rows deserve their fine print. The whole-sequence `Evaluate` is $O(1)$ in the reference because the wrapper forces the root summary before publishing a version — the same forcing that makes a faulty machine fail before any successor becomes visible. The TypeScript port states the same operation as $O(1)$ amortized on the ground that the first read of a freshly built spine may force its suspended middles once. The two ports are charging the same one-time work at different moments; the second wording is the more careful one.

And the $O(sn)$ storage should be read with its quantifier in place. For a fixed machine, s is a constant: the contextual queries are $O(\log n)$ and the storage is $O(n)$, which is why the structure is worth having at all. If instead s is treated as an input, the library is explicit that this structure does *not* dominate a plain persistent sequence — its edits and its storage carry the s factor openly, and a machine with a thousand states will feel like one.

22.5 Why this chapter is the parity campaign’s best exhibit

Look again at the two rows that make this structure attractive rather than merely correct: $O(s)$ amortized endpoint updates and $O(s \log \min(n, m))$ concatenation. Neither is a property of the effect-table monoid. Both are properties of the substrate underneath it, and they hold only on a genuine lazy Hinze–Paterson finger tree — digits at both ends to absorb an endpoint push in constant node work, and a memoized suspension in every deep node’s middle so that the amortization survives *persistent branching* rather than merely ephemeral linear use (Chapter 2).

For a period, four of the nine ports could not state them. Rust, Kotlin, OCaml, and Python shipped this collection over height-balanced join trees — one element per node, cached measures, no digits and no suspensions — and each documented $\Theta(s \log n)$ endpoints and a concatenation bound keyed to how the two operands’ shapes compared rather than to the smaller of them — a height difference in three of the ports, a weight ratio in weight-balanced OCaml. Those were honest notes, not bugs, and they were measured rather than assumed: instrumenting Python’s summary combiner showed a 16,384-element sequence concatenated with a small one costing 28 compositions whether the small operand held 1 element or 1,024, which is the signature of a height-difference bound rather than a log min one. All four carried substrate-divergence notes until their cores were rebuilt.

Wave 1 of the complexity-parity campaign replaced all of those cores in place, and every one of the four now states the reference’s bounds with a probe behind them; the campaign’s core measurements show concatenation of a 4,096-element tree costing zero immediate combines against a one-element operand and four against a 1,024-element one — cost tracking the smaller operand, not the height difference. TypeScript, ported last, is the only port that never had to weaken the claim in the first place.

The lesson generalizes past this one collection. A structure inherits its guarantees from whatever it is built on, and a collection can be entirely correct — passing every behavioral test, in every language — while quietly failing the bound that was the reason to choose it. That is the argument for the campaign in a single example.

Where the ports still differ

Three differences survive, all documented at their own function. Haskell’s enumeration is $\Theta(s \cdot n)$ rather than $O(n)$, because its measured core’s left-view fold rebuilds a spine node per step and each rebuild forces a fresh $O(s)$ compo-

sition, where the reference’s enumerator walks a traversal stack and combines nothing. Ports whose measured core carries an explicit element count alongside the measure — Rust, Kotlin, OCaml, Python, TypeScript — answer a positional index in $O(\log n)$ composing no table at all, which is better than the reference’s measure-directed $O(s \log n)$. And the ports that store each element’s effect row beside the element (Kotlin, Python, TypeScript) invoke the machine *zero* times on a successful select, where C# and Rust invoke it exactly once to report the located transition.

22.6 Where the guarantees stop

Deliberate limits

Context must be finite. Parenthesis depth, indentation stacks, and general parsers do not fit unless their state is explicitly bounded. The state count is validated once and then fixed for the lifetime of the sequence.

Context flows left to right. A right-context rule must be compiled into a streaming machine, delayed, or paired with a reverse machine by the caller.

Event weights must be nonnegative, or the select predicate stops being monotone and measure-guided descent stops being sound.

Totals are checked. Event counts accumulate in a checked 64-bit integer and element counts in a 32-bit one; a version whose composed total would overflow is rejected rather than wrapped. Haskell, whose counts are unbounded integers, keeps the negative-count rejection and drops the vacuous overflow contract.

One element per measured leaf. This is not a chunked text rope and makes no cache-density claim.

The machine cannot be changed. Concatenation requires identical machine semantics, enforced by the type system where a type system can carry the policy and by retained-policy or identifier equality where it cannot. Rebuilding an existing sequence under a different machine costs $O(sn)$.

Failure atomicity

Every operation builds its complete successor before returning it. A throwing transition, an out-of-range next state, a negative event count, or an overflowing total publishes no successor, and every retained version remains exactly as it was. In C# one wrinkle is worth knowing in advance: because the machine's state count is validated in the closed generic type's static initializer, a nonpositive state count reaches the caller as a `TypeInitializationException` whose inner exception carries the real diagnostic, and every later use of that same constructed type reports the same wrapped failure. That is how the runtime reports faulted static initialization; the type is documented to be read through its inner exception.

22.7 On novelty, stated narrowly

The repository is careful about what it claims here, and the care is worth reproducing. Every ingredient is established: Driscoll, Sarnak, Sleator, and Tarjan formalized persistence; Hinze and Paterson supplied the measured sequence and the monotone measure-guided split; Frandsen, Miltersen, and Skyum studied updates and prefix products in a fixed finite monoid under the name of dynamic word problems; Mohri's survey covers weighted transducer composition. The lift is mathematically natural the moment weighted deterministic transitions and measured trees are set side by side, and a tree that already stored exactly these arrays with exactly this predicate would be this construction under another name.

What the proposal records is therefore a deliberately small claim: a reproducible negative search result — that the searched primary sources did not expose this particular public synthesis of a fully persistent, insert-and-concatenate-capable sequence with all-start-state additive summaries and one-descent contextual rank and select that reports the emitting input position. Not a priority claim, not a patentability claim. An earlier or differently named instance may exist. It is a good habit, and this is the collection that most tempts one to break it.

Checkpoints and Deltas

Two roots prove that a version changed. Neither says where.

Persistence makes versions cheap. Keep the old root, edit the new one, and both remain readable forever at the cost of a path copy. What persistence does not do by itself is answer the question a caller asks immediately afterwards: *which* keys — or which positions — differ from the version I nominated as my baseline, and what did they hold before?

There are two obvious ways to answer it, and both are unsatisfying for the same reason. The first is to compare the two roots. That is correct, and a sharing-aware comparison can be very fast when the two versions share most of their structure, but its cost is set by the *representation* rather than by the size of the answer: a single edit path-copies its whole spine, so even one change leaves a logarithmic frontier of unshared nodes for a differ to walk. The second is to keep a write log. A log is cheap to append and names every operation, but it names operations rather than net effect — repeated writes to one key, a value that travels away and comes back, and the order a consumer should read the result in are all left to be resolved later, at a cost proportional to the log rather than to the difference.

The two structures in this chapter take a third route. They maintain the exact net answer *online*, while the edits are happening, so that asking the question later is a read rather than a computation. They differ in what they index: one records a change per changed key, the other records a change per contiguous *run* of changed positions, which is the right granularity when the state is a slab rather than a sparse keyspace.

Both began as research prototypes with normative design proposals of their own, were promoted out of the library's experimental namespaces once those proposals had been reviewed, and now ship in all nine languages.

23.1 The checkpoint-differential map

Specimen — PersistentDeltaMap

Purpose	A fully persistent ordered map that also knows, at every moment, exactly which key classes differ from its own checkpoint and what their checkpoint and current values are.
Core	Three immutable roots — checkpoint state B , current state S , and an ordered change index D carrying one presence-safe (before, after) record per key class on which B and S differ. All three are persistent sorted maps under the same key comparer.
Operations	$O(\log N)$ lookup, point update, removal, rank, and neighbor queries, where N counts the key classes in $\text{dom}(B) \cup \text{dom}(S)$ — the same bounds as the ordinary ordered map underneath.
Epoch control	Checkpoint and Rollback are $O(1)$. They exchange roots; they do not scan.
Enumeration	$\Theta(k + 1)$ to fully consume all k changes in ascending key order. The range-restricted overload seeks in $O(\log(k + 1))$ and then costs $\Theta(1)$ per change yielded.
Policies	A key comparer defining order <i>and</i> identity, and a value equality relation defining semantic no-ops and cancellation.
Limits	Point mutation only. No Clear, no bulk or range update, no diff between two arbitrary retained versions, no confluent merge.
Ports	All nine.

The shape of the design is visible in the field list: a version is a wrapper around three roots and a policy. S answers every ordinary map question and delegates to the underlying sorted dictionary without ceremony. B is shared, not copied — it is simply the state root some ancestor version published as its baseline. D is where the work lives.

What makes D worth maintaining is that it holds the *net* answer, not a history. Four rules produce that, and each one is a specific decision that could plausibly have gone the other way.

The four rules that make “net” exact

A relation-equal write is a semantic no-op. Writing a value that the retained value relation calls equal to the value already stored returns the receiver unchanged — the same object, not an equal one. No record is created, none is disturbed, and no successor is allocated. Removing an absent key behaves the same way.

The first effective write captures before. When a key class enters D , the endpoint it records as `before` is the checkpoint-relative state it is about to displace: `Present(v)` if the class was in the checkpoint, `Absent` if it was not. Presence is carried separately from the value, so `Present(null)` and `Absent` are distinguishable — a distinction the ports without a null-inhabited type spell as `Option` or `Maybe`.

Repeated writes coalesce. Every later effective write to that class replaces only after; the captured `before` is never rewritten. A class written a thousand times still occupies one record, holding the first checkpoint-relative endpoint and the final current one.

Returning a key to its checkpoint state deletes its record. After each effective write the two endpoints are compared — presence first, then the value relation when both are present — and if they agree the record is *removed* rather than stored as a zero. A complete $v_0 \rightarrow v_1 \rightarrow v_0$ round trip therefore leaves no trace, and D never contains a record that is not a real difference.

The fourth rule has a consequence worth spelling out, because it is the one that makes the structure feel different in use: the change set can shrink. An epoch that touches five hundred keys and then puts four hundred and ninety of them back holds ten records, and `ChangeCount` reads that ten as an amortized $O(1)$ field rather than recounting anything.

23.1.1 Canonicalization: a cancelled epoch is storage-clean

If the last surviving record is removed — if the whole epoch cancels — the successor does not merely hold a current root that is extensionally equal to the checkpoint. It holds the checkpoint root *itself*. The state root snaps back, and both logical views name one object again.

This is not needed for the $O(1)$ bound on `Checkpoint` and `Rollback`; those allocate a constant-size wrapper in any case. What the snap buys is that a wholly cancelled

epoch is storage-clean as well as extensionally clean. The identity test at the head of `Checkpoint` and `Rollback` then recognizes the version as already clean and returns the receiver rather than a fresh wrapper, and a later root-identity equality test against the baseline settles in constant time instead of walking an unshared path that a state-only path-copied map would have left behind.

The snap is also the place where the representative rule earns its keep, and it is subtler than it looks. Because the key comparer defines identity, several distinct key objects can name one class, and the map must decide which representative it stores. A changed class that existed in the checkpoint retains the checkpoint's representative through updates and through delete/re-add round trips; a class absent from the checkpoint retains the first insertion's representative for as long as its net-added record survives, and a later insertion after complete cancellation begins a fresh episode. That rule is what makes the snap sound: when D empties, current and checkpoint agree not only extensionally but representative-wise, so substituting one root for the other changes nothing a caller can observe.

```
var map = PersistentDeltaMap<string, int>.CreateRange(
    [new("a", 1), new("b", 2), new("c", 3)]);

var edited = map.SetItem("b", 20) // Updated: Present(2) -> Present(20)
                .SetItem("b", 21) // coalesces; before is still Present(2)
                .Remove("c")      // Removed: Present(3) -> Absent
                .SetItem("d", 4);  // Added: Absent -> Present(4)

var count = edited.ChangeCount;    // 3, not 4: the two writes to "b" coalesced
foreach (var change in edited.GetChanges())
{
    // ("b", Present(2), Present(21))
    // ("c", Present(3), Absent)
    // ("d", Absent, Present(4))  -- ascending key order, once each
}

var restored = edited.SetItem("b", 2); // b's record is deleted: 2 changes
var clean    = edited.Rollback();      // 0(1); map, edited, restored survive
```

23.1.2 Why the enumeration is the point

The change index holds exactly k records, so any operation that visits all of them has an information-theoretic $\Omega(k)$ output cost. An in-order walk of D costs $\Theta(k+1)$ — the $+1$ is enumerator setup — and is therefore output-optimal. It invokes no key or value

policy callback at all: the ordering work was done when the records were inserted.

The comparison worth making is against the alternative the structure exists to replace: reconstructing the same change set from two state-only path-copied trees by pruning at reference-equal subtrees. That technique is genuinely good and is what a library like Rust’s `im` offers, but its cost is the number of nodes the two operands do *not* share, and a point edit necessarily copies its own search spine. Take a complete fixed-fanout tree with N leaf key classes and change k leaves spaced evenly through key order. The union of the copied root-to-leaf paths contains $\Theta(k)$ nodes above the point where the paths separate plus $\Theta(k \log(N/k + 1))$ nodes in the k disjoint lower spines, so a differ whose only certificate of “unchanged” is node identity must inspect a $\Theta(k \log(N/k + 1))$ frontier. At $k = 1$ that is $\Theta(\log N)$ nodes walked to report one change; the delta map reports it in $\Theta(1)$.

The library is careful about what this does and does not claim. It is not a universal lower bound on change discovery. Canonical hashing, hash-consing, per-version logs, or any other maintained metadata changes the problem — and the delta map is itself exactly such an auxiliary index. What it claims is the trade: the reconstruction work is moved forward into `SetItem` and `Remove`, where it costs one extra ordered-map edit — and since $k \leq N$ that edit’s $O(\log(k + 1))$ is inside the state edit’s $O(\log N)$, changing the constants of a point update but not its asymptotic, and buying an output-optimal query in exchange. Liljenzin’s confluent persistent maps, and the Blelloch–Reid–Miller and “Just Join” set-operation results, solve a strictly broader problem — arbitrary operands, confluent combination — and their $\Theta(m \log(n/m + 1))$ bound is in the smaller *input* size, not in this structure’s output size k . They do not contradict the $\Theta(k + 1)$ here, because the delta map is allowed to remember things while the edits happen.

23.1.3 The range-restricted overload is a seek, not a filter

`GetChanges(low, high)` yields the changes whose keys fall in an inclusive range, in the same ascending order. The important property is that it is a genuine boundary seek: the change index is itself an ordered map under the collection’s key comparer, so restricting it is two boundary descents plus a bounded walk of the in-range sub-map. The bound is $O(\log(k + 1))$ for the seek — performed on first enumeration, and invoking only key-order comparisons — plus $\Theta(1)$ per change actually yielded. A window of four changes in an index of two thousand costs what four changes cost, not what two thousand cost.

That distinction is not academic, and the library has the scar tissue to prove it. The C port’s header promised the boundary-only comparison bound while its walk compared

at every visited node — $\Theta(\text{output} + \log k)$ comparator invocations, observable through a counting comparer — and the cross-language parity audit caught it. The walk was rewritten to propagate established bounds, so comparisons now occur only along the two boundary paths and the in-range interior is traversed comparison-free; the comparison-count test that guards it was tightened from 512 to 96. Haskell shipped a differently flawed version of the same idea: a real seek, but with a logarithmic per-element term rather than a constant one, because its shared sorted map exposed no key-range restriction at all. Adding one gave the delta map the baseline’s shape and dropped a measured four-element window over a 2,048-record change index from 61 key comparisons to 22. Every port written afterwards states explicitly that its range enumeration is a seek through the substrate’s own range restriction, and pins the claim with a mutation that reintroduces the filter and must fail.

There is no `Clear`, and that is deliberate

Clearing a checkpoint of N keys creates N net removal records. Maintaining D eagerly through that costs $\Theta(N)$ time and space, and no lazy representation can both keep the explicit-record invariant and match an ordinary immutable map’s root-dropping $O(1)$ clear. Rather than ship a method whose cost violates the structure’s own no-regression premise, the API omits it: start a fresh epoch from `Create` when no old-to-new patch is wanted, or remove entries explicitly when one is. Bulk set, union, merge, range mutation, and `Diff(otherVersion)` are absent for related reasons. `SetItems` exists, but it is documented as exactly the left fold of `SetItem` — an allocation saving, not a different algorithm — so every coalescing, cancellation, and representative rule holds verbatim through it.

A live version retains $O(N + k)$ nodes: two state snapshots and a k -entry index, usually far less physically because of sharing. Retaining every successor adds $O(\log N)$ nodes per changed point update — the same asymptotic version-retention cost as an ordinary path-copied ordered map, with a roughly doubled constant for the second tree.

23.2 The run-delta vector

Specimen — PersistentRunDeltaVector

Purpose	A fixed-length persistent vector with a checkpoint, indexing the exact <i>maximal contiguous runs</i> of positions whose current values differ from that checkpoint — and able to accept or revert one whole run.
Core	Two equal-length immutable RRB vectors, current and checkpoint, plus a persistent ordered map holding one <code>start → endExclusive</code> record per maximal dirty run, plus a cached dirty-position count.
Operations	$O(\log n)$ indexed read of either view; $O(\log n)$ point edit; $O(\log(r + 2))$ dirty membership and run-by-rank selection over r runs; $O(1)$ fork, checkpoint, and rollback.
Enumeration	$\Theta(r)$ run descriptors — output-optimal — against worst-case $\Theta(n)$ discovery from unindexed roots. Complete value enumeration is still $O(n)$.
Hunks	Accepting or reverting one indexed run is $O(\log n)$ through structural RRB splicing, <i>independent of the run's length</i> , and invokes the value relation zero times.
Policies	One retained value equality relation, stable for every derived version's lifetime.
Limits	Length is fixed at construction. One checkpoint per version. No insert, delete, concat, split, rebase, or three-way merge.
Ports	All nine.

The delta map indexes changed keys. That is right when the changes are sparse and unrelated, and wrong when the state is a slab of positions — a property grid, a page table, a simulation parameter vector, a build-state array — where differences arrive in blocks and a consumer does not want them one at a time. A reviewer looking at edited configuration wants hunks, in the sense Git's interactive staging means it: contiguous regions to accept or reject as units.

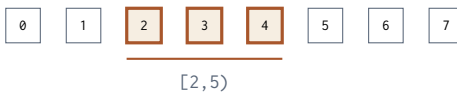
So the index here is not a set of dirty positions but the set of *maximal runs* of dirty positions. Maximality has two halves. Non-overlap is obvious. Non-adjacency is the load-bearing half: $\{[0, 3), [3, 7)\}$ and $\{[0, 7)\}$ describe the same dirty set, and only the second is the maximal encoding. The $\Theta(r)$ enumeration bound means nothing unless r really is the number of maximal runs, so the invariant is stated as ordered, disjoint, *and*

nonadjacent, and the internal validator rejects a version whose records are overlapping, adjacent, or unordered.

Before: positions 2 and 4 differ — *two* records



After one effective write at position 3 — *one* record



Filling the hole at position 3 does not add a record — it *removes* one, by joining the run that ended at 3 with the run that began at 4. The symmetric case is a cancellation at an interior position, which splits one record into two. At most two neighboring records change per edit, which is why the point bound stays logarithmic.

Figure 23.1: The maximal-run index. Run descriptors are stored as $\text{start} \rightarrow \text{endExclusive}$ in a persistent ordered map, so enumerating them is $\Theta(r)$ and locating the run containing a position is a floor query.

23.2.1 The four transitions, and the local cases inside two of them

An edit at position i compares the incoming value with the current value first, and then — before allocating or publishing anything — with the checkpoint value. Only four transitions exist, and only two of them touch the run index.

Transition at position i	What the run index does
relation-equal to the current value	Nothing. The receiver is returned.
clean $\rightarrow$ dirty	Insert i , joining whatever it now touches: join a run ending at i with one starting at $i + 1$; extend the left run rightward; extend the right run leftward; or create the singleton $[i, i+1)$.
dirty $\rightarrow$ dirty	Nothing. The record already covers i .
dirty $\rightarrow$ clean	Remove i from its unique containing run: delete a singleton record; trim its left edge; trim its right edge; or split it into two records.

At most two neighboring records change, so a point edit is one RRB path copy plus a constant number of logarithmic predecessor, successor, and update operations on the run map: $O(\log n)$ time and fresh nodes. If removing the last dirty position empties the run index, the current root snaps back to the checkpoint root exactly as the delta map's does, and the validator enforces it — a clean vector that has not been canonicalized is rejected.

23.2.2 What the run index buys

Two things, and they are different in kind.

The first is discovery. Enumerating run descriptors is $\Theta(r)$, which is output-optimal because the answer has r records. The baseline it improves on is worst-case $\Theta(n)$: from two unindexed roots, an exact set of run boundaries can require inspecting every position, because until a position is examined its two values can be equal or unequal without changing anything observed so far. The gap is unbounded — one contiguous changed block gives $k = n$ dirty positions and $r = 1$ run, and the focused suite pins that with a clustered witness of 8,191 dirty positions represented by two descriptors. Against an explicit point-delta index the unconditional statement is about metadata rather than time: $\Theta(r)$ live records instead of $\Theta(k)$. A linear coalescing scan over point records costs $\Theta(k)$, and the library is candid that this is not a representation lower bound for every order-statistic point index, and that a balanced Discrete Interval Encoding Tree in Erwig's sense — which is essentially what the run index is — matches these bounds exactly. No new interval algorithm is claimed.

The second is the hunk operation, and it is the one that surprises. Accepting one run splices that range of the current root over the checkpoint root; reverting splices

the checkpoint range back over the current root. Both are composed from four RRB splits and two concatenations, so both cost $O(\log n)$ *independently of the run's length* — accepting a run of one position and a run of a million positions cost the same — and both invoke the value relation zero times. The positions outside the interval keep both roots untouched. When the selected run is the only run, the operation folds into Checkpoint or Rollback and becomes a constant-time root exchange.

```
var slab = PersistentRunDeltaVector<int>.CreateRange([0, 0, 0, 0, 0, 0, 0, 0]);

var edited = slab.SetItem(2, 7).SetItem(4, 9).SetItem(3, 8);

var dirty = edited.DirtyCount;          // 3
var runs  = edited.DirtyRunCount;       // 1 -- writing 3 joined [2,3) and [4,5)
var run   = edited.GetDirtyRunAt(0);    // Start = 2, Length = 3

var kept   = edited.AcceptDirtyRunAt(0); // checkpoint takes 7, 8, 9
var dropped = edited.RevertDirtyRunContaining(3); // current back to zeros
var split  = edited.ResetItem(3);       // interior: one run becomes two

// slab and edited remain valid and independently editable.
```

Only the descriptors are run-sensitive

Enumerating, hashing, serializing, or applying all the *changed values* is still $\Omega(k)$ for k dirty positions, and the library says so rather than letting the $\Theta(r)$ headline imply otherwise. Nor is the vector globally optimal as a persistent array: Dietz's fully persistent arrays and Straka's worst-case refinement beat these $O(\log n)$ point operations outright. They simply do not expose dirty-run semantics. The discovery result applies to the declared RRB comparator family and to nothing wider. Starting from one n -element version, m retained effective point edits occupy $O(n + m \log n)$ worst-case total structural space; that history bound is not collapsed to $O(n)$.

23.3 The reference-identity invariant

Both structures canonicalize, and the vector's version of the rule is strict in a way that turns out to constrain the ports: a clean position must reuse its *exact* checkpoint cell, not an equal one. The validator states it as such and fails a version in which a clean

position's current and checkpoint entries are distinct objects.

The rule earns its strictness by being load-bearing, and the cell is what makes it hold. The RRB substrate recognizes a leaf no-op with its *own* default equality, while this collection decides dirtiness with a caller-supplied relation — two different relations judging the same write. Wrapping each value in a private cell whose default equality is reference identity reconciles them: a genuine edit installs a fresh cell, a cancellation installs the checkpoint's own cell, and the substrate can never quietly retain an equal-but-different object on the reasoning that its own comparer saw no change. Without that, a clean position could hold a value equal under one relation and distinct under the other, the two roots would drift apart physically while agreeing extensionally, and the canonicalization the run index depends on would be defeated.

Stating the invariant is easy; making it *testable* is where the languages diverge, because in several of them the natural identity operator on the payload types a test reaches for first is value equality. Python's `is` is true for equal small integers, booleans, and interned strings. TypeScript's `===` is value equality on numbers and interned strings. OCaml's `==` is value equality on immediates. In each of those workspaces the run-delta vector boxes every element in a private, identity-bearing cell so that the invariant has something to be about. In Python and TypeScript there was no substrate rule to defeat — their leaf writers already short-circuit on identity rather than on equality — so the boxing exists purely to make the invariant observable, and the ports say so. C# uses the same device, a private sealed `Cell` class whose default equality is reference equality by construction. Rust replaces it with `Arc<T>` and `Arc::ptr_eq`: the same reference identity with one fewer allocation layer. Haskell has the hardest problem, since pure code exposes no object identity at all, and solves it by giving each cell a globally unique token drawn from an atomic counter through the `unsafePerformIO/NOINLINE` device `Data.Unique` uses — which makes the identity test available to the *pure* validator, with `StableName` diagnostics confirming the same fact independently in the tests.

23.4 The relation that decides everything

Both structures retain a value equality relation, and it is not a convenience. It decides which writes are no-ops, and it decides when a delta cancels — which is to say it decides the observable change set. Two versions built from identical operations under different relations are legitimately different values, which is why the relation is retained on the instance rather than taken as a static constraint.

Its precondition is that it be a genuine equivalence relation: reflexive, symmetric,

transitive, and stable for the lifetime of every version derived from it. Reflexivity is the one that gets violated in practice, and it fails in the most ordinary way imaginable.

A non-reflexive relation corrupts the accounting silently

IEEE floating point says $\text{NaN} \neq \text{NaN}$, so any relation that forwards to primitive floating-point equality is not reflexive. Rewriting a position with the identical NaN then fails the no-op test, the position is recorded as dirty against a checkpoint value it is bit-identical to, and the phantom run corrupts run counts, ranks, and the dirty count for every subsequent operation.

The reason this is worth a caveat box rather than a footnote is that *no internal validation can detect it*. The index remains perfectly consistent with the relation it was given; the relation is simply not an equivalence relation. The library states that limitation outright rather than implying a validator that cannot exist.

The ports therefore defend at the type level or at the policy level. Rust bounds its natural policy constructors on `Eq` rather than `PartialEq`, so `f64` — which is not `Eq` — fails to *compile* against the natural policy instead of silently diverging, and canonical `EqualityPolicy::reflexive_ieee` constructors supply the .NET-matching behavior for callers who do want floats. Python’s `natural_value_equality` consults identity first and `reflexive_ieee_equality` collapses every NaN into one class. OCaml’s default relation is `compare x y = 0` rather than `( = )`, because on that toolchain `nan = nan` is false while `compare nan nan` is zero. Kotlin needs no escape hatch at all: its natural policy is generic, so it boxes and reaches `java.lang.Double.equals`, which compares raw bits and *is* reflexive on NaN — the residual difference from .NET being signed zero, which the boxed comparison separates and `EqualityComparer<double>.Default` does not.

OCaml recovers a partial check the others do not attempt: its structural audit rejects a version whose relation is observably non-reflexive *at a clean position*, where current and checkpoint are literally the same cell and the relation must therefore answer true. That catches the cause, even though the consequence at dirty positions stays invisible — which is exactly the condition the Rust port documents as undetectable.

Failure atomicity follows the same discipline in both structures: every policy call happens before any successor is constructed, so a throwing relation or comparer publishes nothing and leaves the receiver and every retained branch intact. What

cannot be undone is a side effect inside a callback, and the documentation says that plainly rather than implying transactionality it does not have.

23.5 Where the ports differ

The repository’s cross-language parity work left these two structures unusually well characterized, including in the places where the ports are not equal.

Divergence	Where, and why
Delta-map extremes	$O(1)$ in C#, C++, Haskell, OCaml, Python, and TypeScript, where the ordered substrate is a finger tree and <code>MinEntry/MaxEntry</code> are digit reads. $\Theta(\log N)$ in C, Kotlin, and Rust, whose ordered cores cache subtree sizes but no extremes, making an extreme a rank select like any other. Each port documents its own.
Run-splice qualifier	C#, Rust, Python, and OCaml state the point edit and the run splice worst case outright, their RRB and run-index substrates deferring nothing. TypeScript splits the claim: the splice is worst case, the run-index write amortized through its finger tree’s concatenation. C++ states every run-index-routed operation amortized, because that substrate may force a pending lazy spine on a read.
Change-enumeration setup	C#’s <code>GetChanges()</code> is a lazy iterator with $\Theta(1)$ setup; Rust’s <code>get_changes</code> materializes, so setup is $\Theta(k)$. Full consumption is $\Theta(k + 1)$ in both.

The run-splice row is a small monument to how that work proceeded. The reference implementations originally documented those bounds as amortized — a true statement, but weaker than their eager code paths deliver. A freshly ported workspace stated the worst-case bound instead, and under a governing principle that says the stronger true statement wins, the C# and Rust documentation was raised to match the port rather than the port lowered to match the reference.

The Monotone-Action Heap

Rewriting every priority should not cost you every entry.

Specimen — PersistentMonotoneActionHeap

Purpose	Apply one order-preserving transformation to every priority currently in a heap, in constant time, without giving up meld, worst-case bounds, or persistence.
Core	The bootstrapped skew-binomial representation of BrodalOkasakiHeap (Brodal and Okasaki, <i>JFP</i> 1996), with one immutable <i>pending action</i> on every tree node and forest spine cell.
Operations	Insert, minimum, meld, and TransformAll: $O(1)$ worst case. Delete-minimum: $O(\log n)$ worst case. Enumeration and validation: $\Theta(n)$. Forking: $O(1)$.
Algebra	Identity, identity test, associative Compose(outer, inner) denoting $\text{outer}(\text{inner}(p))$, and Apply — each $O(1)$ over fixed-size actions, every action monotone for the retained comparer.
Policies	Trusted: no finite runtime check establishes associativity or monotonicity, so a policy that breaks its contract invalidates the semantics and the bounds together.
Ports	All nine.

A branchable scheduler holds a hundred thousand pending jobs in a persistent heap, and a policy change arrives: nothing may run before Tuesday. Every priority in the heap has to move up to that floor.

The ordinary way to do that is to take the heap apart and put it back together. The

Haskell heaps package’s `Data.Heap` — itself a persistent Brodal–Okasaki heap, with the same worst-case $O(1)$ insert, minimum, and union and worst-case $O(\log n)$ delete-minimum that this repository’s `BrodalOkasakiHeap` delivers (Chapter 19) — offers `mapMonotonic` at $O(n)$, and that is an honest price for an arbitrary promised-monotone function. Emulating the same change through this library’s own state-only heap costs the same: enumerate, transform, rebuild — $\Theta(n)$.

`PersistentMonotoneActionHeap<TElement, TPriority, TAction>` pays $O(1)$ instead, worst case, allocating $O(1)$ new structure — and it does so without weakening any of the bounds it inherits. The trade that buys this is narrow and worth stating up front: the transformation may not be an arbitrary function. It must come from a fixed, closed family of actions that compose in constant time and preserve the priority order. Under that restriction, and only under it, transforming the whole heap becomes a single note written on the root.

The scope of the claim

Recursively tagged heaps are, by construction, an equal-bound implementation of this idea; the proposal for this structure says so explicitly. What is claimed is the aggregate: a worst-case-optimal fully persistent meldable heap, a generic action policy that may be *non*-invertible, constant-time whole-heap transformation with version-local temporal semantics, and exact priority representatives under comparer-equivalent clamp boundaries. Tarjan’s `addtokeys` is the important ancestor here, and it is an invertible translation — which, as the next pages show, is a materially easier problem.

24.1 The algebra, and the one law that carries the structure

The policy interface has four members and no others:

- ▶ an *identity* action;
- ▶ `IsIdentity`, which recognizes it;
- ▶ `Compose(outer, inner)`, associative; and
- ▶ `Apply`, which runs an action on one priority.

The laws are the ones you would guess. Applying the identity returns the priority unchanged; applying a composite equals applying the inner action and then the outer

one. Two further requirements are not laws about values but about cost, and they are just as binding. Every operation above must be $O(1)$, and every composed action must stay *fixed size*. The second condition is the one that quietly kills naive designs: storing a chain of pending transformations instead of collapsing them would make `Minimum` linear in the number of retained transforms, and the advertised bound would evaporate without a single test failing.

The law that actually holds the structure together, though, is monotonicity:

The monotonicity requirement

For every action a and all priorities p, q ordered by the heap's retained comparer,

$$p \leq q \implies \text{Apply}(a, p) \leq \text{Apply}(a, q).$$

Everything depends on this. A tag attached to a node is a promise about that node *and its entire subtree*: every priority underneath is logically the stored value with the pending action applied. The heap property says that each parent's priority is no greater than each child's. If the pending action can reverse a single one of those relations — if some parent/child pair with $p \leq q$ becomes $a(p) > a(q)$ — then attaching one tag has silently broken the heap, everywhere below that node at once, and nothing in the operation that attached it looked at those nodes to notice. A whole-heap transform is only sound because a monotone action preserves *every* existing order relation simultaneously; that is precisely why `Minimum` may apply the root's tag and return the same root without re-examining anything, and why melding, linking, and splitting keep working on tagged components as if the tags were not there.

Negation is the obvious excluded case, and it is excluded for exactly this reason. So is any permutation of the priority domain that reverses a pair. Neither is repairable by cleverness in the heap; both require a rebuild, and the structure does not pretend otherwise.

24.2 Composition has a direction

`Compose(outer, inner)` means: apply *inner* first, *outer* second.

Equivalently, `Compose(outer, inner)` denotes `outer(inner(p))`. At every site where a tag is pushed down or merged, the *newer* action is the outer one: a node already

carrying a pending b that receives a fresh a ends up carrying $\text{Compose}(a, b)$, because a arrived later and therefore runs last.

This deserves the box because getting it backwards is a bug that hides. Swap the two arguments and the composite is still a member of the action family, and it is still monotone — so the heap remains perfectly well formed. Ranks are untouched. Parents still compare no greater than children. The structural validator, which checks rank encodings, logical heap order through every pending action, and the count, passes without complaint. Only the *values* are wrong, and only for entries that happened to accumulate two or more non-commuting actions. The C++ port review treats this as a first-class hazard and confirms the composition-direction case by mutation: the test fails against a deliberately reversed kernel, which is the only evidence worth having for a property this quiet. The same book-wide convention appears in the range-update sequence’s tag monoid (Chapter 13); the two structures state the direction in the same words for the same reason.

24.3 Where the tags live

A version retains a root tree, a count, the comparer, and the policy. Two kinds of cell carry pending actions, and both are load-bearing:

Cell	Tag scope
Tree node	The node and every descendant. A tree can be relinked as a unit, so its action must travel with it.
Forest spine cell	Uniformly, every tree in that spine <i>suffix</i> — which is what lets an action be pushed down one cell at a time without walking the rest of the suffix.

Three logical views do all the work. `TagTree` and `TagForest` compose a newer action onto a node or spine cell. `Expose` normalizes one tree: it applies the node’s pending action to that node’s own priority and moves the same action onto the child forest — one new spine cell, not a traversal — leaving an identity-tagged node behind. Every structural consumer reaches children through tag-composing accessors rather than through raw fields, so a detached tail can never lose the action that governed it.

Two consequences follow. First, tags do not change ranks or forest shape, so the skew-binomial rank encoding is preserved by tagging exactly as it stands; the insert path exploits this by reading raw spine ranks to decide whether a link is needed *before*

materializing any tagged tail node. Second, tags are pushed down only at places where the underlying Brodal–Okasaki algorithm already exposes a component — which is why the ordinary bounds are not merely preserved in spirit but unchanged: insert, minimum, and meld stay $O(1)$ worst case, and delete-minimum stays $O(\log n)$ worst case, each counting the $O(1)$ policy calls and comparisons it performs.

24.4 The clamp family, and why it is the interesting case

`OrderClampPolicy<T>` ships one closed family: `AtLeast` (a floor), `AtMost` (a cap), `Between` (a validated two-sided clamp, which rejects a lower bound comparing greater than the upper), and `Constant`. A missing bound denotes an infinity. The identity is the clamp with neither bound.

Clamp is the interesting family precisely because, unlike the familiar additive heap offset, it is *not invertible*. After flooring a heap’s priorities at zero, there is in general no raw priority you could assign to a later insertion such that one flat normalization recovers the value the caller asked for. The proposal’s witness is small enough to check by hand. Take a heap holding -10 and 100 , clamp it to $[0, 10]$, and then insert 50 . Draining must yield $0, 10, 50$: the new entry never existed when the clamp happened. Applying the old root’s tag to the newcomer would produce $0, 10, 10$. And in a second retained branch off the same clamped heap, inserting -20 must yield $-20, 0, 10$ — so there is no single flat clamp that could stand in for the pair of histories. Recursive fixed-size tags are what close the representation under the full operation set; a flat wrapper simply does not implement this ADT.

Composition of two clamps must still be one clamp, in constant time. The rules are worth reading closely, because three of the four are asymmetric:

Case	Result
Either side is the identity	The other side, unchanged.
The <i>outer</i> is a constant	The outer constant. Nothing the inner action did can survive a later constant function.
The <i>inner</i> is a constant	A constant carrying the outer action applied to the inner’s value.
Disjoint bounds	A constant carrying the <i>outer</i> boundary. Cap everything at 5 and then floor at 10, and every output is 10.

Overlapping bounds

The intersection $[\max(iL, oL), \min(iU, oU)]$, keeping the *inner* representative when two boundary values compare equal.

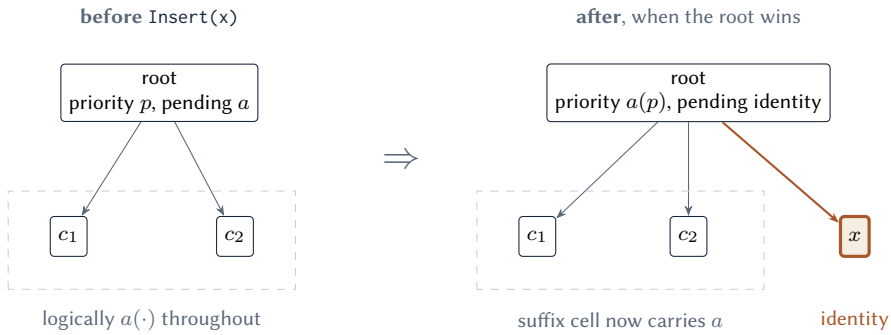
Two of those rows encode a distinction that is easy to miss. Composition produces an explicit `Constant` rather than the inclusive interval $[k, k]$, and it keeps the older — inner — representative on tied boundaries. Both choices exist for the same reason: a comparer may treat distinct priority *values* as equal. An inclusive $[k, k]$ clamp passes through an input that compares equal to k , preserving that input’s own representative, whereas the constant function must return the exact k the caller supplied. So the family preserves exact sequential results, not merely comparison equivalence — which is a stronger promise than a heap has any obligation to make, and is the kind of promise that only survives if someone writes it down.

Because monotonicity is a property *relative to a comparer*, `OrderClampPolicy` implements a second interface that names the exact comparer object for which it is monotone, and heap creation adopts that comparer and rejects any other. Melding likewise requires comparer and action-policy object identity. These are not defensive gestures; a policy monotone for one order is generally not monotone for another, and silently mixing the two would reintroduce the failure that monotonicity exists to prevent.

24.5 Temporal semantics: expose before you attach

`TransformAll` transforms every priority *present in this version*. Entries inserted later are not retroactively transformed, and neither are entries arriving through a later meld. That sentence is the whole reason the structure is usable as a scheduling primitive: a floor imposed on Monday’s backlog must not silently apply to a job filed on Wednesday under the current external priority scale.

The mechanism is one ordering rule, applied everywhere components with different histories meet. `Insert` compares an untagged singleton against the *logical* root. If the singleton wins, it becomes a fresh identity-tagged global root and the old tagged root becomes its child, carrying its own action down with it. If the old root wins, it is *exposed first* — its pending action pushed onto its own children — and only then is the new singleton skew-inserted into the resulting child forest, through an identity-tagged spine cell (Figure 24.1). `Meld`, `Link`, and `SkewLink` follow the same rule: compare logical roots, expose only the winner, attach the loser through an identity cell. `Delete-minimum` exposes the global root, selects the least logical tree in its child forest, and exposes that tree before splitting its fused children.



The old root is *exposed* — its pending a applied to its own priority and moved onto its child suffix in one spine cell — *before* the new entry is attached. The newcomer therefore enters through an identity-tagged cell and inherits nothing. Exposure touches one node and one spine cell, so insertion stays $O(1)$ worst case.

Figure 24.1: Expose-before-attach. The rule is what makes “every priority *currently* in this version” a meaningful phrase rather than an approximation.

Melding two independently transformed heaps is the case that shows the design earning its keep. Continue the earlier example: a second heap holds 50 and 200 and is independently clamped to $[60, 70]$. Melding it with the $[0, 10]$ -clamped heap must drain 0, 10, 60, 70. Losing the second heap’s tag would expose 50 and 200; leaking the first heap’s tag into the second would turn its entries into tens. Each operand keeps its own action history until an ordinary structural operation exposes a component — and the meld itself remains $O(1)$ worst case, with both operands still valid and independently extendable afterwards.

24.6 Where the guarantees stop

The honest limits

Enumeration and `ValidateStructure` apply pending actions as they walk, so both are $\Theta(n)$; the $O(1)$ claim is about transformation, not about reading everything.

Persistence does not make total history free. A single current version occupies $O(n)$; starting from n_0 entries, u retained updates use $O(n_0 + u \log N)$ worst-case total structural space for the maximum size N encountered — a safe upper

bound that is smaller for histories dominated by constant-allocation updates. The policy is trusted, and a throwing comparer or policy call publishes no successor, leaving every existing version untouched — but side effects the callback performed cannot be rolled back.

There are no element handles, no decrease-key, no deletion by handle, and no arbitrary heap mapping. Equal-priority tie order is unspecified. Payload identity is preserved when priorities collapse, which is why payload and priority are separate fields rather than one comparable value.

Two port notes are worth recording. Haskell takes the ordering and the action family from a retained policy *value* rather than a class constraint, which keeps a fully drained heap usable — but a record of functions has no identity to compare, so unlike the managed ports it cannot reject operands built with semantically different policies. Its documentation says so plainly, including the consequence that melding two non-empty heaps adopts the left operand’s policy while an empty operand returns the other heap verbatim, so the surviving policy depends on emptiness. And in OCaml this heap held, for a while, the only genuine bootstrapped skew-binomial kernel in the workspace: when the complexity-parity campaign found that workspace’s plain `Brodal_okasaki_heap` storing its elements in a list, the repair was to rebuild it as the untagged specialization of the kernel its own action heap already contained.

The Ancestral Connection Forest

Not whether two vertices are connected, but when — and answered without consulting the history.

Specimen — PersistentAncestralConnectionForest

Purpose	Persistent, branching, insertion-only connectivity over a fixed integer vertex universe, plus one query ordinary union-find cannot answer: at which ancestor version did two vertices first become connected?
Core	A union-by-size parent forest stored sparsely in a CHAMP map, with every union edge labelled by the version that created it.
Operations	Find, Connected, GetComponentSize, Link, and FirstConnected: $O(w \log n)$ worst case, for bounded CHAMP path cost w . Create and ComponentCount: $O(1)$. Forking: $O(1)$.
Complexity	FirstConnected costs exactly one connectivity query and is independent of history depth. A black-box persistent disjoint-set forest needs $O(w \log n \log(K + 2))$ for K successful unions on the queried lineage.
Limits	No deletion, no confluent merge of sibling versions, no retroactive update, no path compression, and no semantic guarantee on representative choice.
Ports	All nine.

Union-find answers “are x and y connected?” A persistent union-find answers it per version, which is already useful: retain a version, fork it, ask each branch separately.

But there is a question just past that boundary which the persistent version still cannot answer without brute force. Given a version v and two vertices, *which ancestor version of v first put them in the same component?*

The brute-force answer is a search. Connectivity along a lineage is monotone — once connected, always connected, since edges are only added — so the versions on the root-to- v path split into a disconnected prefix and a connected suffix, and binary search finds the boundary. The strongest form of that comparator keeps an append-only index of only the successful-union versions and probes $\log(K + 2)$ of them, where K is the number of successful unions on the lineage. Since K can be $n - 1$, that is a full logarithmic factor above the cost of a single connectivity query — $O(\log^2 n)$ against $O(\log n)$ in the CHAMP specialization, once the bounded trie factor is folded in.

`PersistentAncestralConnectionForest` answers the question at the price of one connectivity query, and — this is the part worth dwelling on — it never looks at the version history at all.

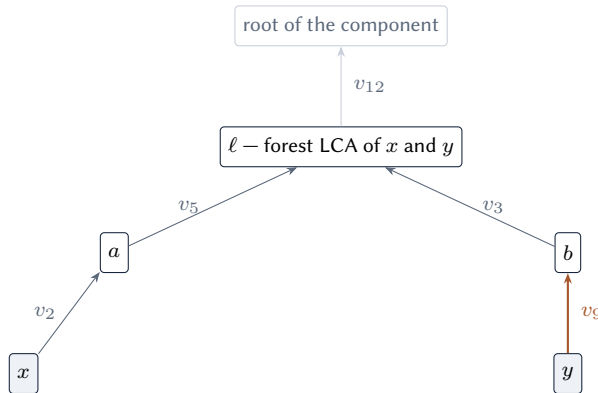
25.1 The mechanism: an edge remembers its own birthday

Each successful `Link` does one thing beyond ordinary union by size: it labels the single new root-parent edge with the child version that created it. Nothing else is recorded. There is no per-version snapshot, no event log, no index over the lineage.

The query then works entirely inside the queried version's own parent forest. Walk from each vertex to its root, collecting the edge labels on the way. If the two roots differ, the vertices are disconnected and the answer is nothing. If they agree, delete the shared rootward suffix of the two paths — what remains is exactly the two paths strictly below their forest lowest common ancestor — and return the *latest* label among those remaining edges (Figure 25.1).

Why is that the right answer, and why does it need no history scan? The argument rests on a small fact about how union edges accumulate. When a root becomes a child, both endpoints were roots an instant earlier, so every edge already below the losing root was created strictly earlier — and any edge later placed above the winner is created strictly later. Labels therefore *strictly increase* toward the root along any path in any version's forest. Given that, the edges below the pair's lowest common ancestor are exactly the edges whose existence the connection required, and the last of them to appear is the moment the connection happened. Everything above the ancestor is later history that both vertices now share — it moved them together, never joined them.

So the version history is not consulted because the answer is not stored in the history.



The answer is v_9 : the latest label *strictly below* ℓ . Every edge below ℓ must exist before the forest path from x to y exists, so the pair cannot be connected any earlier; and at v_9 the last missing edge closes the path, so it is connected then. The edge v_{12} above ℓ is deliberately excluded — it attached an already-joined component to something else and must not overwrite the pair's own history.

Figure 25.1: FirstConnected. Two parent-path walks and a maximum; no version is ever visited.

It is stored, distributed, one label at a time, in the shape of the forest the queried version is already holding. Reading it costs the same two root walks that any connectivity query costs: $O(w \log n)$ worst case, with $O(\log n)$ transient path space, and — worth stating twice — independent of how deep the history is.

Why a query must not shorten the path

Path compression, the reflex of every ephemeral union-find implementation, is excluded here. There are two reasons, and the second is the load-bearing one. The mechanical reason is that compression is a *write performed by a read*. In a structure whose queries must not disturb versions other readers hold, a find cannot rewrite shared parent cells; it could only publish a new version, which is not what a query is.

The semantic reason is that compression destroys the very boundary FirstConnected reads. Union a with b at t_1 , c with d at t_2 , join the two components at t_3 , then compress a and b straight to the final root. The correct answer for the pair (a, b) is still t_1 ; but if the compressed edges carry the maximum

label they skipped, both paths now report t_3 , and the original lowest common ancestor — the whole basis of the answer — has vanished. Richer summaries could preserve selected history queries, but they are a different design and would owe a new proof.

The price of the exclusion is stated rather than hidden: bounds here are worst-case logarithmic in component size, not the inverse-Ackermann amortized bound an ephemeral union-find enjoys.

25.2 Union by size, and the ceiling it alone guarantees

With compression gone, the entire height argument rests on union by size. The larger component's root wins; on a tie, the first endpoint's root stays the representative, which makes results deterministic without touching the proof. Every time a vertex's distance to its root grows by one, the component containing it has at least doubled, and no component exceeds n — so every parent path in every version has at most $\lfloor \log_2 n \rfloor$ edges. The structural validator enforces exactly that number as a hard ceiling rather than a comfortable approximation.

Persistence does not weaken the argument, because it applies independently to the one forest each queried version selects. Nor can a sibling branch's update contaminate a query: path copying gives siblings different roots at their changed paths and shared immutable nodes everywhere else, so every cell and every label a query observes belongs to the queried version or one of its ancestors. Labels are compared only within a lineage, and a version's depth is meaningful only against its own ancestors — a sibling allocated later is not a later state of this branch.

25.3 What “branching” means here

Every `Link` produces a distinct child version. That includes a redundant one: linking two vertices already in the same component still yields a new version token, which shares the complete connectivity index unchanged and adds $O(1)$ space. The parent is untouched, remains valid, and can be extended again in a different direction. That is the entire content of the word “branching”: the version graph is a tree, not a line, and old versions are not merely readable but independently queryable and independently extendable.

Version identity is reference identity in the reference port, so two sibling versions at

equal depth are different versions even though their depths match. Rust reproduces this with pointer identity behind a shared handle. Haskell diverges deliberately and documents it: a version there is an ordinary immutable value whose identity is structural, recording its link endpoints so that equal-depth siblings stay distinct — with the visible consequence, asserted in its test suite rather than hidden, that two versions built by genuinely identical operations from the same parent are one value.

The sparse representation is what keeps all this cheap. An absent cell means “singleton root”, so a million-vertex forest allocates nothing per vertex and `Create` is $O(1)$ for any universe size. A stored root is therefore always non-singleton, and it caches its component’s size — which is why `GetComponentSize` is one root walk plus a cached read, at the same cost as `Find`. A successful `Link` allocates $O(w)$ new CHAMP nodes; a redundant one allocates only its version wrapper and token.

25.4 The path factor, and a guarantee that was strengthened

$O(w \log n)$ has two factors, and the second one has a story.

The CHAMP path cost w is bounded by the number of trie levels a hash can address — at most seven for a 32-bit hash (Chapter 4). For a general CHAMP with arbitrary user keys, that per-level cost is only *expected* constant, because distinct keys can share a hash by pigeonhole and a collision bucket must then be scanned. Here the keys are vertices from a fixed integer universe, and distinct integers can be given distinct hashes — so the bound is unconditional, and the whole operation cost is worst case in both factors.

Seven of the nine ports had that property from the start. Two — Rust and Haskell — did not: each had reached for a general-purpose hasher whose output was truncated, so distinct vertices could in principle collide and the honest statement was “expected”. The complexity-parity campaign of August 2026 treated that as a violation rather than a documentation matter, and both were fixed by pinning an injective 32-bit vertex hash — MurmurHash3’s `fmix32` bijection, every step invertible modulo 2^{32} — so that no collision bucket can ever hold two distinct vertices. Haskell additionally caps its universe at $2^{31} - 1$, the universe its siblings already had, because no 32-bit hash is injective over a larger domain; `create` now rejects one, and the cap is load-bearing rather than cosmetic. Both ports carry an inverse round-trip test exhibiting the bijection.

The episode is a small, clean example of the campaign’s governing rule at work. The easy resolution — write “expected” in two ports’ documentation and move on — was available and would have been perfectly truthful. It was rejected because a probabilistic

guarantee and a worst-case guarantee are different promises, and the library’s claim is that the same structure delivers the same bound in every language. Strengthening the code was cheaper than explaining the divergence forever.

25.5 Where the guarantees stop

Deliberate non-goals

Edges can be added, never removed. After deletion, connectivity is no longer monotone along a lineage and the whole first-connected question would need redefining.

There is no confluent merge of sibling versions, no retroactive update of an existing version, and no reconciliation across branches. `FirstConnected` is defined only relative to one version’s ancestor chain; it never returns a version from another branch.

The structure returns the first connectivity *event*, not a shortest path, an original edge witness, bridge information, or an explanation. Representatives returned by `Find` are implementation artifacts offered for interoperability; the same component is not promised the same representative in sibling versions.

Version tokens retain their parent chains, so a deep history stays reachable from any live version at $O(\text{depth})$ token space — a cost per `Link`, not per successful union.

Finally, the comparison this structure wins is the *online, branching* one. A sealed linear history can be preprocessed into a component tree with $O(1)$ lowest-common-ancestor queries, and no superiority is claimed in that model.

PART VI

Authenticated Collections

The Merkle Search Tree

Structural sharing works across versions. Content addressing works across machines.

Specimen — MerkleSearchTree

Purpose	An ordered, content-addressed persistent map whose root digest names the entire collection, so two processes that agree on a digest agree on every entry.
Core	A $B = 16$ wide tree in which each key's layer is derived from the count of leading zero nibbles of its SHA-256 digest — a geometric distribution — so the shape is independent of insertion order.
Identity	Policy <code>mst-sha256-b16-v2</code> : a versioned semantic id, comparer semantics, canonical per-type codecs, and a SHA-256 domain digest.
Wire	The canonical MST2 block format — byte-identical across all nine ports.
Order	Policy comparer order, with range enumeration.
Ports	All nine, wire-compatible.

Everything up to this point exploits sharing *within a process*. Two versions of a persistent map diff cheaply because they share pointers, and pointers do not survive a network. If the two collections you want to compare live in different processes — or on different machines, or were built a week apart by unrelated code — pointer identity tells you nothing and you are back to comparing everything.

Content addressing fixes this by making the *name* of a subtree a function of its contents. Give every node a digest of what it holds, including the digests of its children, and equal digests prove equal subtrees no matter who built them or when. That is the Merkle idea, and applying it to an ordered map gives the structure behind AT Protocol repositories (Bluesky) and Dolt’s versioned SQL tables.

26.1 Shape from hashes, not from history

A Merkle tree is only useful if equal contents produce equal digests, and that requires equal contents to produce the *same shape*. An ordinary B-tree fails this immediately: insert the same entries in a different order and you get a different — equally valid — tree.

Aurolat and Taïani’s Merkle search trees (SRDS 2019) fix the shape by deriving it from the data. Hash each key; count its leading zero nibbles; that count is the key’s *layer*. Since each additional zero nibble is a 1-in-16 event, layers follow a geometric distribution with $B = 16$, which is exactly the branching behavior a B-tree wants — but assigned by the hash function rather than by insertion history (Figure 26.1).

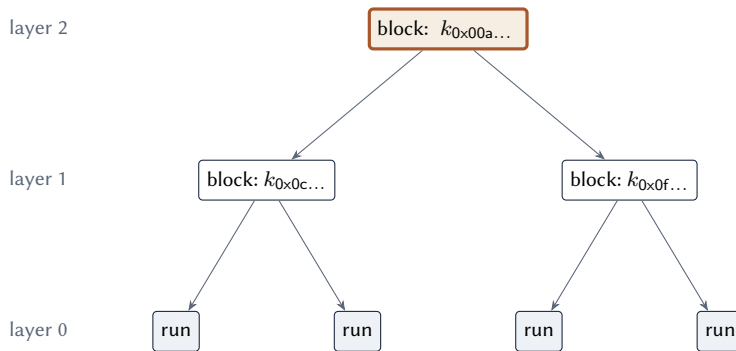


Figure 26.1: Layer assignment in a Merkle search tree. A key’s layer is the number of leading zero nibbles of its SHA-256 digest; one block holds each consecutive same-layer run, and that block’s digest covers the policy domain, its entries, its subtree count, and its child digests. Because the shape comes from the hashes, two processes that insert the same entries in different orders build the same tree and compute the same root digest.

One block holds each consecutive run of same-layer keys, and each block’s MST2 bytes cover the policy domain, its entries, its subtree count, and its child digests. The root digest therefore names the whole collection.

Prolly trees, the convergent cousin

Probabilistic B-trees — “prolly trees”, from Noms around 2016 and now Dolt — reach the same destination by a different route: instead of hashing each key to a layer, they run a rolling hash over the entry stream and place a node boundary wherever it crosses a threshold. This is *content-defined chunking*, the same technique that powers rsync-style deduplication. Both designs produce uniquely represented, content-addressed, history-independent search trees; Durable7 implements the Merkle search tree variant.

26.2 Two prerequisites, taken seriously

The repository treats content addressing as a capability that must be *earned* by satisfying two conditions, and records them as mandatory for any future content-addressed family.

Prerequisite 1 — pinned deterministic hashing

Default string hashing on .NET is randomized per process. That is a fine defense against hash-flooding and completely fatal for cross-process addressing. A Merkle policy therefore binds a *versioned semantic id*, canonical codecs, comparer semantics, and a SHA-256 domain digest. The policy id versions the application’s comparer semantics: if you change what “equal” means, you change the id, and old digests stop colliding with new ones by construction.

Prerequisite 2 — canonical serialization

A digest is only as canonical as the bytes it hashes. A strict versioned codec must *encode and decode*, and its decoder must accept exactly one canonical encoding while rejecting malformed, non-canonical, and trailing input. Codec ids end in -v followed by decimal digits. The built-ins cover integers, nullable strings, nullable byte arrays, and GUIDs.

Do not use a codec whose output depends on process-randomized hashes, culture, object identity, ambient serialization settings, or mutable state. Every one of those turns a content address into a lie.

```
var policy = MerkleSearchTreePolicy<int, string?>.Create(
```

```

    "document-index-v1",
    Comparer<int>.Default,
    MerkleCodecs.Int32,
    MerkleCodecs.UTF8String);

var baseline = MerkleSearchTree<int, string?>.Create(policy)
    .SetItem(10, "ten").SetItem(20, "twenty");
var updated = baseline.SetItem(20, "TWENTY").SetItem(30, "thirty");

MerkleDigest address = updated.RootHash;

foreach (var (key, value) in updated.EnumerateRange(15, 30))
    Console.WriteLine($"{key}: {value}");

```

Recreate the same policy id, comparer semantics, codec ids, and logical entries in any other language, and you get the same block graph and the same root address. That is not an aspiration in this repository — golden wire vectors are shared test data, and a tree built in Rust verifies against a proof produced in TypeScript.

Diff is not free just because digests exist

Equal block digests prune whole regions, and in-process Diff exploits that. But a topology-changing edit can move a block separator, and when it does, the implementation honestly falls back to ordered subtree comparison rather than pretending the digest pruning still applies. The shipped API therefore does *not* promise $O(\text{key divergence})$ for every diff. Block synchronization — Chapter 27 — is the divergence-oriented path, and it is the right tool when the two trees live in different processes.

Blocks, Proofs, Synchronization, and Merge

The tree is the easy half. Not trusting the network is the other half.

A content-addressed collection is only interesting if it can leave the process. This chapter covers the tier that lets it: storing blocks, moving them, verifying them without trust, proving individual claims about them, and merging two divergent descendants of a common ancestor.

Everything here ships in all nine languages with byte-identical wire behavior.

27.1 Blocks and stores

A *block* is one immutable node's canonical bytes, addressed by its digest. A *store* is a mapping from digest to block — the persistence boundary. The library's in-memory implementations are concurrent (an `RLock` in Python, a shared mutex in C++, a synchronized three-state store in C), and every store operation is atomic with respect to publication.

```
var store = new InMemoryMerkleBlockStore();
int blocksAdded = updated.Save(store);

var loaded = MerkleSearchTree<int, string?>.Load(
    updated.RootHash, policy, store);
```

The critical property of `Load` is what it refuses to assume.

Load never trusts the store

Loading recomputes every block digest, strictly decodes *and re-encodes* every entry to confirm canonicity, follows and validates every child reference, checks the result against the expected root, and applies a finite verification budget throughout. A store that returns tampered, malformed, non-canonical, or merely inconsistent blocks produces an error, not a tree.

The re-encode step is the one people skip. Decoding alone proves the bytes are *parseable*; re-encoding and comparing proves they are *canonical*, which is what the digest actually claims.

27.2 Packs, and the transport boundary

A *pack* is a set of blocks prepared for transport. A *complete* pack is self-contained. A *partial* pack can be completed from blocks the destination already holds — which is the common case, since two replicas of a slowly changing tree share almost everything.

```
MerkleBlockPack outbound = updated.ExportPack();

var replica = MerkleSearchTree<int, string?>.Import(
    outbound, policy, replicaStore);

Console.WriteLine(replica.RootHash == updated.RootHash); // True
```

Import verifies every supplied block *and* the requested root closure before it performs destination conflict preflight, and it commits nothing until all of that succeeds. A rejected import leaves the destination store exactly as it was.

27.3 Seven budgets

Verification of untrusted input is where authenticated data structures go wrong, and the failure mode is rarely a wrong answer — it is resource exhaustion. A malicious pack can be a decompression bomb, a cycle, a pathologically deep closure, or a codec that allocates a gigabyte from four bytes.

Bounded verification

Every port enforces *seven* positive finite budgets during verification, and the ordering of checks is itself part of the contract: query and proof-shape limits run *before* verifier allocation, hashing, or codec work. A malformed request is rejected before it can cost anything.

Local proof and synchronization construction consumes a read-only trusted topology view rather than applying network budgets or re-decoding values that are already retained. The budgets exist for the trust boundary, and they are applied at the trust boundary.

27.4 Proofs

A proof lets a party who does *not* have the tree verify a claim about it, given only the root digest. The canonical MSP2 format carries three kinds:

Membership — this key maps to this value in the tree named by this digest.

Non-membership — this key is *absent* from that tree. This is the harder and more valuable direction, and it works because the tree is ordered: the proof exhibits the adjacent entries that would bracket the key, and the structure guarantees nothing can hide between them.

Range — these are exactly the entries between these bounds, with nothing omitted.

Proof verification fixes query, shape, and envelope precedence before any decoder or codec work runs, and tampering with any part of a proof — the claimed value, a sibling digest, the envelope — produces a rejection rather than a wrong answer. The test suites verify this adversarially rather than by inspection.

27.5 Synchronization

Synchronization answers: *I have root A, you have root B, send me what I am missing.* The mechanism is a frontier walk. Compare digests from the root down; wherever they agree, an entire region is already shared and the walk stops; wherever they differ, descend. The blocks the destination lacks form the frontier, and the resulting sync pack is closure-pruned so that nothing already present is retransmitted.

The walk is iterative rather than recursive — a hostile tree should not be able to overflow a stack — and repair is likewise iterative, so a partially populated store

converges over successive rounds rather than requiring a complete restart.

27.6 Three-way merge

The final operation is the one that turns an authenticated map into a version control system: given a common ancestor and two descendants, produce a merged tree.

Merge obligations

The merge is typed and *present-null aware*: a key that was deleted in one branch and a key whose merged value is present-and-null are different outcomes, and the API distinguishes them rather than collapsing both into “no value”.

A merged tree with unresolved conflicts is never published. Conflict detection completes before any publication, so a caller either receives a complete merged tree or receives the conflicts — never a tree that is silently missing the contested keys.

That refusal to publish partial results is the same failure-atomicity rule from Chapter 3, applied where the stakes are highest. A partially merged authenticated tree would still have a valid root digest, which means it would be indistinguishable from a correct one — a wrong answer that carries a proof of its own correctness. The only safe design is to not produce it.

What this replaced

An earlier design sketch in the repository proposed a `MerkleHamt`: content addressing over the *unordered* hash trie. It remains deferred. The Merkle search tree is its ordered sibling and subsumes most of its uses while adding range queries and range synchronization — neither of which an unordered trie can offer. Should `MerkleHamt` ever be revived, it must reuse equally explicit hashing, encoding, verification-budget, and trust-boundary contracts; the prerequisites are not negotiable per family.

PART VII

Navigation

Cursors

A position that remembers which version it belongs to.

Persistent collections are excellent at whole-collection operations and awkward at local editing. Inserting one character at position 4,000 of a rope is a logarithmic descent; inserting sixteen characters one at a time is sixteen logarithmic descents, and every one of them re-walks the same path.

A cursor fixes this by being what an iterator is not: an immutable value that owns one collection version *and* a position within it, from which both navigation and editing are cheap and which can be retained and branched like any other persistent value.

What a cursor is, and is not

Every initialized cursor owns or retains one immutable logical version and a valid gap, rank, or measure boundary. Navigation returns a cursor over the *same* version. Editing returns a cursor over *one new* version and never changes the receiver, a retained ancestor, or a sibling branch. Snapshot is non-consuming. A cursor is not a detachable position, a mutable iterator, a builder, a transient, a bookmark, a rebase operation, or a live concurrent view. There is no implicit redirection to a later version and no cross-version application.

28.1 Gap semantics

A positional cursor denotes a *boundary*, not an element — one of the $n + 1$ gaps in a sequence of length n (Figure 28.1). This is the same model a text editor’s caret uses, and it is chosen for the same reason: an insertion point between two elements is a first-class thing, and a cursor that denoted an element could not represent “before everything”

and “after everything” without special cases.

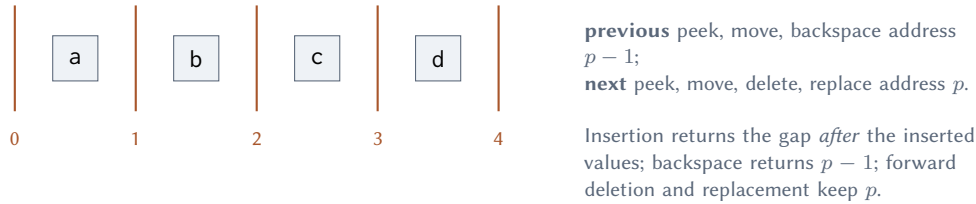


Figure 28.1: Gap semantics. Empty, start, and end gaps are ordinary first-class states rather than special cases.

Ordered cursors use the analogous idea over comparer order: the gap before the next ordered candidate. A lower-bound miss leaves a *usable* cursor rather than an error, exact search carries a separate hit discriminator, and an edit validates that the key or value actually belongs at the current gap. Nested Ordered multimap cursors keep group and value positions explicit rather than inventing a private flattened order that would not correspond to anything the collection promises.

28.2 Which families have one

Not every collection has a meaningful “next”. The library’s rule is that a cursor requires a stable *semantic* neighbor axis, and hash traversal order is not one.

Family group	Semantic axis
Patricia integer maps and sets	Ascending signed 32/64-bit key order, with rank gaps.
Measured sequences, dequeues, reversible dequeues, RRB vectors, range-update sequences, ropes and text	Position, or an ordered measure boundary. Text units remain language-local.
Sorted bags, sets, and maps; canonical sorted sets; priority search queues; interval trees and maps; chunked bit sets	Comparer order, interval order, key order, or population-rank gaps, as owned by the family.
Neutral Ordered sets, maps, multimaps	Explicit position; the multimap additionally exposes nested group and value positions.

Merkle search trees	Policy comparer order, with authenticated persistent edits.
CHAMP maps, sets, and their derived facades	<i>No public cursor.</i> The lookup and edit path is private, because hash enumeration is not semantic order.
Ctrie live views, graphs, heaps without a stable order, builders, sessions, stores, proofs, packs, DabaLite	<i>No persistent cursor.</i> No stable neighbor axis, or not a persistent aggregate at all.

28.3 One focused representation, many checkpoints

Here, once more, is the reference-versus-checkpoint pattern — and this is its most consequential instance, because the difference is a complexity claim rather than an implementation detail.

The C# positional rope cursor uses the *focused* representation selected after an explicit design comparison: a readonly struct over immutable version and navigation-context references, holding a bounded 16-element active focus with at most one partial carry of fewer than 256 elements on each side. Movement and edits return cursor values; navigation shares the logical version and its snapshot memo; an edit creates a new version state. The measured cursor extends the same model with exact ordered `MeasureBefore` and `MeasureAfter` and an absolute measure seek, preparing element measures in the cursor lineage and sharing them with descendants.

Snapshots and racing readers

A cursor created from a rope starts with that exact rope as its clean cached snapshot. A dirty first `Snapshot()` performs bounded focus and carry packing plus a canonical tree join, and publishes one winner through an interlocked compare-exchange — racing callers all receive that same rope reference. A failed candidate publishes nothing, so the cursor stays reusable. Later snapshots from any navigation cursor over the same logical version are $O(1)$ and reference-identical. Initialized cursor values are immutable and safe for concurrent reads, including concurrent first snapshots.

The proven complexity scope is a lineage, not a DAG

Creation and arbitrary seek are $O(\log n)$. Focus-local peeks, movement, and single-element edits are $O(1)$ amortized *along one linear version lineage*, with $O(\log n)$ worst-case boundary repair. Range insertion of m elements is $O(m + \log n)$ amortized. A dirty snapshot is bounded packing plus an $O(\log n)$ join; clean repeated snapshots are $O(1)$.

There is no arbitrary-version-DAG amortized bound. Editing b independently retained cursors at a carry, chunk, or lazy-spine boundary carries the conservative $O(b \log n)$ aggregate bound plus bounded per-branch copying, because potential consumed by one child cannot pay for a sibling. This is Chapter 1's branching caveat, arriving exactly where it was predicted to.

The other eight ports ship *snapshot-plus-gap* checkpoints: an already-canonical retained collection plus a validated position, delegating edits to ordinary persistent operations. Creation, navigation, positional seek, and snapshot are $O(1)$; measures, peeks, point edits, and search are $O(\log n)$ plus bounded chunk work; range insertion is $O(m + \log n)$. They preserve every observable behavior — gap semantics, branching, unconditional replacement, retained snapshots, presence-safe peeks, ordered before/after measures, absolute prefix search — and they claim no focused representation, no snapshot memo, no allocation or callback ceiling, no amortized locality, and no benchmark parity. Their API notes say so in those words.

Measured cursor result semantics, everywhere

MeasureBefore aggregates $[0, p)$ and MeasureAfter aggregates $[p, n)$; combining them *in that order* yields the whole version's measure — without assuming an inverse, commutativity, element equality, or a default-value identity.

Absolute measure seek selects the gap before the first element whose inclusive prefix satisfies a lawful monotone predicate. A predicate already true at the identity selects zero for a non-empty collection; misses and empty collections return false together with an end cursor whose before-measure is the whole measure.

A same-position seek and an empty range insertion preserve the exact stored version state. ReplaceNext deliberately does *not* consult element equality: it always creates a new logical version, and measured replacement invokes the

element-measure callback for the supplied replacement even when the object is identical.

Why cursors instead of self-adjusting structures

The classical answer to “make repeated local access cheap” is a self-adjusting structure — a splay tree, a move-to-front list. Both are rejected here for two independent reasons worth recording so the idea is not re-litigated.

First, **reads would allocate**. Adapting on read means path-copying on read: every lookup produces a new version or mutates shared state, destroying both the allocation-free read contract and thread-safe sharing of versions.

Second, **shared versions have contradictory access patterns**. Two consumers holding the same version would each want it shaped for *their* access sequence, and the amortized potential argument assumes one linear history — the same failure mode as the branching caveat above.

The cursor is the substitute: per-consumer locality, held by the consumer, without ever reshaping shared state.

PART VIII

Reference



Complexity at a Glance

Bounds hold in all nine ports unless a row says otherwise. That is a stronger statement than this appendix could make before August 2026, when five ports still ran balanced trees under finger-tree names; Chapter 2 records how that was closed. Where a row does still differ it names the ports, and the workspace’s own API notes remain authoritative. Throughout, n is the receiver’s size, m the argument’s, c a collision-bucket or chunk count, and d a distance from the nearer end.

Structure	Operation	Bound
<i>Hash tries</i>		
PersistentHashMap / Set	lookup, insert, remove GetOrAdd / AddOrUpdate equality / diff, shared ancestry	$O(\log_{32} n)$, + $O(c)$ on collision one hash, one descent visited non-shared regions + differences
PersistentHashBag	equality / diff, independent point ops, algebra	$O(n + m)$; $O(c^2)$ per bucket as the underlying map
PersistentBiMap	add, set, remove Inverse	two map operations $O(1)$
PersistentHashMultimap	pair add / remove	$O(\log_{32} n)$
PersistentRelation	Inverse	$O(1)$ index swap
PersistentChunkedBitSet	membership, rank, select algebra	$O(\log c)$ sparse word-stream merge
<i>Integer tries</i>		
PersistentIntMap / Set	point operations	$O(\min(n, W))$

	union / intersect / except no-op merge	visited non-shared prefixes; $O(n + m)$ worst case reference identity preserved
<i>Sequences</i>		
FingerTreeDeque	push / pop at either end index, split at position concat	$O(1)$ amortized $O(\log d)$ $O(\log \min(n, m))$
RrbVector	index endpoint operations concat	$O(\log_{32} n)$, uniform $O(\log_{32} n)$ (no persistent tail) $O(\log_{32}(n + m))$
Rope / MeasuredRope	index, point edit concat / split	$O(\log n) + \text{bounded chunk work}$ $O(\log \min) / O(\log n)$
RangeUpdateSequence	whole-sequence tag subrange tag, range measure point edit, split, concat	$O(1)$, one new root $O(\log n)$ $O(\log n)$
ReversibleDeque	Reverse	$O(1)$
DabaLite	insert / evict / query Clear	$O(1)$ worst case; $\leq 3/2/1$ combines $O(1)$ managed; $O(n + c)$ native
<i>Ordered and priority</i>		
SortedBag / Set / Dictionary	lookup, insert, remove rank, select	$O(\log n)$ $O(\log n)$
CanonicalSortedSet	insert, remove inequality via digest equality, independent sets	expected $O(\log n)$ path copy; $O(n)$ adversarial $O(1)$ in the usual case $O(n)$
PersistentOrderedSet / Map	membership, append positional read, move	$O(\log_{32} n) + \text{sequence cost}$ $O(\log n)$
PriorityQueue	peek enqueue, dequeue meld	$O(1)$ amortized $O(\log n)$ amortized $O(\log \min)$ amortized
BrodalOkasakiHeap	minimum, insert, meld delete-minimum	$O(1)$ worst case $O(\log n)$ worst case

PrioritySearchQueue	minimum	$O(1)$
	keyed update, remove, delete-min	$O(\log n)$
	key-range + priority query	$O(\log n + v)$, $v \leq n$
IntervalTree / PersistentIntervalMap	insert, remove, exact lookup	$O(\log n)$
	stabbing / overlap	$O(\log n + \text{reported})$, with subtree pruning
<i>Authenticated</i>		
MerkleSearchTree	lookup, insert, remove	$O(\log_{16} n)$
	root digest	$O(1)$ cached
	Save / Load	$O(\text{blocks})$, fully re-verified
	synchronize	frontier blocks only
<i>Lifecycle and navigation</i>		
Bulk builder	per entry	unpublished in-place node mutation
Editing session	adopt, publish	$O(1)$ trie work
	per edit (C# owner-token)	owned-node mutation, no path copy
	per edit (sibling ports)	ordinary persistent path copy
Cursor (focused, C# rope)	local move, single edit	$O(1)$ amortized, one lineage
	arbitrary seek	$O(\log n)$
	b retained branches at a boundary	$O(b \log n)$ aggregate
Cursor (checkpoint ports)	create, navigate, seek, snapshot	$O(1)$
	peek, point edit, measure, search	$O(\log n) + \text{chunk work}$
	range insert of m	$O(m + \log n)$
<i>Research-derived collections</i>		
Incremental ancestor arena	leaf add	$O(1)$ amortized, over M the nodes the arena has ever published; a block-boundary call pays $\Theta(\sqrt{M})$ for the next odd block

	leaf add, Haskell	$O(1)$ worst case — that port has no arena; a node is its own handle
	ancestor at depth	$O(\log M)$ worst case, over Myers jump links
	parent, depth, value	$O(1)$
AncestralSliceQueue	AddLast	one leaf add
	Count, Last, RemoveFirst, RemoveLast, Drop	$O(1)$, no arena call
	First, index, Take, Slice, nontrivial SplitAt	one ancestor query; $O(\log M)$ on the shipped backend
	enumerate n visible values	$\Theta(n)$ time and $\Theta(n)$ transient slots
BilateralAncestralDeque	AddFirst / AddLast	one leaf add
	endpoint read, Count, Reverse, Clear	$O(1)$, no arena call
	end removal	$O(1)$ on its own nonempty arm; one query when it crosses the centre
	index; slice, split	at most one query; at most two
ContextualRankSequence	final state, total event count	$O(1)$, the cached root summary
	prefix evaluate, event rank, event select	$O(s \log n)$ for a machine of s states
	point edit, insert, remove, split, slice	$O(s \log n)$
	index	$O(\log n)$ where the descent reads cached sizes and composes no summary; $O(s \log n)$ in C# and C++
	Prepend / Append	$O(s)$ amortized; $O(s \log n)$ worst case per call
	concat	$O(s \log \min(n, m))$ amortized
	summary space	$O(sn)$, not $O(n)$
PersistentDeltaMap	lookup, rank, neighbour, effective point edit	$O(\log n)$ over the n key classes of checkpoint and current
	least, greatest current entry	$O(1)$ digit read
	Checkpoint, Rollback, fork	$O(1)$, root exchange

	TryGetChange	$O(\log(k+1))$ for k changed key classes
	consume the change stream	$\Theta(k+1)$, output-optimal and sorted
PersistentRun	read, point edit	$O(\log n)$ worst case, not amortized
DeltaVector	checkpoint, rollback, fork	$O(1)$
	dirty membership, run by rank	$O(\log(r+2))$ worst case, over r maximal dirty runs
	run descriptors	$\Theta(r)$; state-only roots need $\Theta(n)$
	accept / revert one run	$O(\log n)$, independent of the run's length
PersistentMonotone	minimum, insert, meld	$O(1)$ worst case
ActionHeap	transform every current priority	$O(1)$ worst case, $O(1)$ new structure
	delete-minimum	$O(\log n)$ worst case
	enumerate, validate	$\Theta(n)$
PersistentAncestral	create, component count, fork	$O(1)$; absent cells denote singletons
ConnectionForest	Find, Connected, Link	$O(\log n)$ over n vertices, the CHAMP factor unconditional rather than expected
	FirstConnected	$O(\log n)$ — one connectivity query, without the extra logarithmic factor a search over the version lineage would pay



The Same Structure in Nine Languages

Names are idiomatic per port; semantics are not. Where a port's spelling differs structurally — a nested type, a module, a handle — the difference is notational.

Family	Spellings
Persistent hash map	<code>PersistentHashMap<TKey, TValue></code> (C#, Kotlin, TypeScript, Python); <code>PersistentHashMap<K, V, S></code> (Rust); <code>persistent_hash_map<Key, T, Hash, KeyEqual, ValueEqual></code> (C++); <code>HashMap k v</code> (Haskell); <code>d7_hamt_map</code> (C); <code>Persistent_hamt</code> (OCaml).
Persistent hash set	<code>PersistentHashSet<T></code> ; <code>persistent_hash_set<T, Hash, KeyEqual></code> ; <code>HashSet a</code> ; <code>d7_hamt_set</code> ; <code>Persistent_hash_set</code> .
Editing session	<code>nested Transient</code> (C#, Kotlin, C++, OCaml); <code>TransientHashMap/Set</code> (Rust, TypeScript, Python); <code>MapTransient/SetTransient</code> (Haskell); <code>d7_hamt_map_transient</code> (C).
Hash bag	<code>PersistentHashBag</code> ; <code>persistent_hash_bag</code> ; <code>HashBag a</code> ; <code>d7_hamt_bag</code> ; <code>Persistent_hash_bag</code> .
Bidirectional map	<code>PersistentBiMap</code> ; <code>persistent_bi_map</code> ; <code>BiMap k v</code> ; <code>d7_hamt_bi_map</code> ; <code>Persistent_bi_map</code> .
Integer Patricia	<code>PersistentIntMap/IntSet/LongMap/LongSet</code> ; <code>persistent_int_map</code> etc.; <code>IntMap32/IntMap64/ IntSet32/IntSet64</code> (Haskell); <code>d7_int_map</code> etc.; <code>Persistent_patricia</code> .
Concurrent trie	<code>ConcurrentHashTrie</code> (C#, Kotlin — full protocol; TypeScript, Python — facade); <code>Concurrent_hash_trie</code> (OCaml facade).
Merkle search tree	<code>MerkleSearchTree<TKey, TValue></code> ; <code>merkle_search_tree</code> ; <code>MerkleSearchTree k v</code> ; <code>d7_merkle_search_tree</code> ; <code>Merkle_*</code> modules (OCaml).

Persistent deque	<code>FingerTreeDeque<T></code> (C#); <code>PersistentDeque<T></code> (Kotlin, Rust, TypeScript, Python); <code>persistent_deque<T></code> (C++); <code>Deque a</code> (Haskell); <code>ft_persistent_deque</code> (C); <code>Persistent_deque</code> (OCaml).
Measured tree	<code>FingerTree<TElement, TMeasure, TMeasureOps></code> (C#); <code>FingerTree<T, M></code> / <code>FingerTree<T, P></code> (Kotlin, TypeScript, Rust); <code>finger_tree<Element, MeasurePolicy></code> (C++); <code>FingerTree v a</code> (Haskell); <code>ft_tree</code> (C); <code>Measured_tree</code> / <code>Measured_sequence</code> (OCaml).
RRB vector	<code>RrbVector<T></code> and <code>builder</code> ; <code>rrb_vector<T></code> / <code>rrb_vector_builder<T></code> ; <code>RrbVector a</code> ; <code>ft_rrb_vector</code> / <code>ft_rrb_builder</code> ; <code>Rrb_vector</code> .
Reversible deque	<code>ReversibleDeque<T></code> ; <code>reversible_deque<T></code> ; <code>ReversibleDeque a</code> ; <code>ft_reversible_deque</code> ; <code>Reversible_deque</code> .
Range-update sequence	<code>RangeUpdateSequence<TElement, TMeasure, TTag, TOps></code> (C#); <code>RangeUpdateSequence<T, M, Tag></code> (Kotlin, TypeScript); <code>range_update_sequence<Element, Algebra></code> (C++); <code>RangeUpdateSequence</code> (Haskell, Rust, Python); <code>ft_range_update_sequence</code> (C); <code>Range_update_sequence</code> (OCaml).
Ropes and text	<code>Rope<T></code> , <code>MeasuredRope<. . .></code> , <code>TextRope</code> / <code>RopeText</code> ; <code>rope<T></code> , <code>measured_rope<. . .></code> , <code>text_rope</code> ; <code>Rope a</code> , <code>MeasuredRope v a</code> , <code>TextRope</code> ; <code>ft_rope</code> , <code>ft_measured_rope</code> , <code>ft_text_rope</code> ; <code>Rope</code> , <code>Measured_rope</code> , <code>Text_rope</code> .
Sorted collections	<code>SortedBag<T></code> , <code>SortedSet<T></code> , <code>SortedDictionary<TKey, TValue></code> (C#); <code>SortedMap<K, V></code> elsewhere; <code>sorted_bag/sorted_set/sorted_map</code> (C++); <code>ft_sorted_multiset/_set/_map</code> (C); <code>Sorted_bag/Sorted_set/Sorted_map</code> (OCaml).
Canonical sorted set	<code>CanonicalSortedSet<T></code> and <code>ZipTreeRankPolicy<T></code> ; <code>canonical_sorted_set<T></code> and <code>zip_tree_rank_policy<T></code> ; <code>ft_canonical_sorted_set</code> and <code>ft_canonical_policy</code> (C); <code>Canonical_sorted_set</code> (OCaml).
Priority queue	<code>PriorityQueue<TElement, TPriority></code> ; <code>priority_queue<Element, Priority, Comparison></code> ; <code>PriorityQueue p a</code> (Haskell); <code>ft_priority_queue</code> ; <code>Priority_queue</code> .
Brodal–Okasaki heap	<code>BrodalOkasakiHeap<T></code> ; <code>brodal_okasaki_heap<T, Less></code> ; <code>BrodalOkasakiHeap a</code> ; <code>ft_brodal_heap</code> ; <code>Brodal_okasaki_heap</code> .

Priority search queue	PrioritySearchQueue<TKey, TPriority, TValue>; priority_search_queue<K, P, V, KeyLess, PriorityLess>; PrioritySearchQueue k p v; ft_priority_search_queue; Priority_search_queue.
Interval tree and map	IntervalTree<T>, Interval<T>, PersistentIntervalMap; interval_tree<T, Comparison>; IntervalTree a; ft_interval_tree, ft_persistent_interval_map; Interval_tree, Persistent_interval_map.
Sliding-window aggregator	DabaLite<T, TMonoid> (C#); DabaLite<T> / DabaLite<T, M>; daba_lite<T, P>; ft_daba_lite; Daba_lite.
Derived compositions	PersistentHashMultimap, PersistentRelation, PersistentOrderedMap, PersistentOrderedMultimap, PersistentMapPatch, PersistentDirectedGraph, PersistentIndexedMap, PersistentChunkedBitSet — with snake_case in C++/OCaml/Python, d7_/ft_ prefixes in C, and bare names in Haskell.
Incremental ancestor arena	MyersIncrementalAncestorArena<T> behind IIncrementalAncestorArena<T> (C#); the same class behind IncrementalAncestorArena<T> (Kotlin, TypeScript), and MyersIncrementalAncestorArena behind an IncrementalAncestorArena protocol in durable7.finger_tree.incremental_ancestor (Python); MyersAncestorArena<T> behind the IncrementalAncestorArena<T> trait (Rust); myers_incremental_ancestor_arena<T> (C++); ft_incremental_ancestor_arena, created by ft_incremental_ancestor_myers_arena_create (C); Incremental_ancestor (OCaml); Durable7.FingerTree.IncrementalAncestor (Haskell) — which ships no arena at all: a Node a is its own handle.
Ancestral slice queue	AncestralSliceQueue<T> (C#, Kotlin, TypeScript, Rust), and AncestralSliceQueue in durable7.finger_tree.ancestral_slice_queue (Python); ancestral_slice_queue<T, Arena> (C++); AncestralSliceQueue a (Haskell); ft_ancestral_slice_queue (C); Durable7.FingerTree.Ancestral_slice_queue (OCaml).

Bilateral ancestral deque	BilateralAncestralDeque<T> (C#, Kotlin, TypeScript, Rust), and BilateralAncestralDeque in durable7.finger_tree.bilateral_ancestral_deque (Python); bilateral_ancestral_deque<T, Arena> (C++); BilateralAncestralDeque a (Haskell); ft_bilateral_deque — the one spelling shortened away from its own header name (C); Durable7.Finger_tree.Bilateral_ancestral_deque (OCaml).
Contextual rank sequence	ContextualRankSequence<TElement, TMachine> (C#); ContextualRankSequence<T, M> (Rust); ContextualRankSequence<T> (Kotlin, TypeScript), and ContextualRankSequence in durable7.finger_tree.contextual_rank_sequence (Python); contextual_rank_sequence<T, Machine> (C++); ContextualRankSequence element (Haskell); ft_contextual_rank_sequence, with its machine carried by an ft_crs_policy handle (C); Durable7.Finger_tree.Contextual_rank_sequence (OCaml).
Checkpoint-differential map	PersistentDeltaMap<TKey, TValue> (C#); PersistentDeltaMap<K, V> (Kotlin, TypeScript, Rust), and PersistentDeltaMap in durable7.finger_tree.persistent_delta_map (Python); persistent_delta_map<Key, Value, KeyLess, ValueEqual> (C++); PersistentDeltaMap key value (Haskell); ft_delta_map (C); Durable7.Finger_tree.Persistent_delta_map (OCaml).
Run-delta vector	PersistentRunDeltaVector<T> (C#, Kotlin, TypeScript, Rust), and PersistentRunDeltaVector in durable7.finger_tree.persistent_run_delta_vector (Python); persistent_run_delta_vector<T, EqualityPolicy> (C++); PersistentRunDeltaVector a (Haskell); ft_run_delta_vector (C); Durable7.Finger_tree.Persistent_run_delta_vector (OCaml).
Monotone-action heap	PersistentMonotoneActionHeap<TElement, TPriority, TAction> (C#); PersistentMonotoneActionHeap<E, P, A> (Kotlin, TypeScript, Rust), and PersistentMonotoneActionHeap in durable7.finger_tree.persistent_monotone_action_heap (Python); persistent_monotone_action_heap<Element, Priority, Policy> (C++); PersistentMonotoneActionHeap element priority action (Haskell); ft_monotone_action_heap (C); Durable7.Finger_tree.Persistent_monotone_action_heap (OCaml).

Ancestral connection
forest

PersistentAncestralConnectionForest with
AncestralConnectionVersion — alone in this group it takes no
type parameter anywhere, its vertices being a fixed integer universe
(C#, Kotlin, TypeScript, Rust), and
PersistentAncestralConnectionForest in
durable7.hamt.persistent_ancestral_connection_forest
(Python); persistent_ancestral_connection_forest (C++);
Durable7.Hamt.PersistentAncestralConnectionForest
(Haskell); d7_hamt_ancestral_connection_forest — the one
member of this group that lives in the hash-trie library, so it carries
the d7_hamt_prefix rather than ft_ (C);
Durable7.Hamt.Persistent_ancestral_connection_forest
(OCaml).

Roads Not Taken

A library is defined as much by what it declines to build. These decisions are recorded so that they are not silently re-litigated, and each carries the reason that would have to change for the answer to change.

C.1 Rejected outright

Self-adjusting structures. Splay trees, move-to-front lists, and their relatives fight persistence twice over: adapting on read means path-copying on read, destroying the allocation-free read contract; and two consumers sharing a version want it shaped for two different access sequences, so the amortized potential argument has no single history to be proved over. The substitutes are cursors and the planned frozen read-phase optimization.

Strict Fibonacci and hollow heaps. Both achieve optimal decrease-key bounds through aggressive pointer mutation, which is precisely the surgery persistent path copying destroys.

Kaplan–Tarjan real-time dequeues. Their $O(1)$ *worst-case* endpoints and catenation are genuinely attractive, but the shipped deque’s amortization spikes are already rare and logarithmic — memoized suspensions spread the debt — and the implementation is notoriously intricate (redundant counting systems, multiple bootstrapping layers) with constants that typically lose on real workloads. The useful deliverable instead is to *measure and publish* the shipped deque’s worst observed pause, so a future latency-critical consumer can decide with data.

C.2 Postponed, with a named gate

Size-tiered small representations. Below a threshold, represent the collection as a flat array; promote at the threshold; demote with hysteresis. Clojure’s `PersistentArrayMap` and Rust’s `im::Vector` both do it, and it targets exactly the regime where persistent maps lose worst — the three-entry map that most programs hold thousands of. It is classified as *strong but postponed*: thresholds must be benchmark-derived rather than guessed, hysteresis must prevent flapping, and representation dispatch must not tax the large case. Two supporting arguments are already recorded: a general-purpose library owns its own equality, enumeration, and no-op identity contracts and can therefore guarantee representation-blindness; and a *constant* promotion threshold costs $O(1)$ worst case, so unlike an $O(n)$ -at-threshold scheme it survives branching persistence.

The frozen collection tier. Designed, prototyped, and gated on lookup, enumeration, memory, and realistic construction break-even evidence (Chapter 8).

Key-type-specialized map construction. A factory that routes integer keys to the Patricia trie, byte-comparable keys to an adaptive radix tree, and everything else to the hash trie. The honest counterpoint is that one facade unifying three cores costs either virtual dispatch or a fat discriminated union on *every* operation, and the cheaper design is separate public types plus a documentation-level decision table. Postponed in the separate-types form; the ART core itself is a real project, gated on a consumer with genuine prefix-query or byte-ordered-key needs.

The order-maintenance list. The Dietz–Sleator structure that answers “does *A* precede *B*?” in $O(1)$ with amortized $O(1)$ insertion, via two-level integer labels and local relabeling. Plausible, and waiting for a named general consumer — dependency graphs, document anchors, CRDT position identifiers. The honest caveat is already written: relabeling amortization is a linear-history argument, so a version branched before a relabel can re-pay it, and the public type would have to state its worst case per produced version.

MerkleHamt. Content addressing over the unordered hash trie. The ordered Merkle search tree subsumes most of its uses and adds range queries and range synchronization; if revived, it must reuse equally explicit hashing, encoding, verification-budget, and trust-boundary contracts.

C.3 Shipped as documentation rather than as a type

The styled-text rope. A text rope carrying formatting runs — a measured rope for content plus a run sequence (ideally the range-update sequence, so “bold lines 10–500” is one logarithmic tag) keeping style spans aligned under edits. This is a composition showcase, and by the repository’s own parity-economics rule it ships as a *sample plus documentation pattern* rather than as a tenth family in nine languages. The editor samples are its natural home.

C.4 Analyzed, found implementable, and declined

Real-time two-table migration for the ancestor arena. The append-only arena beneath the two ancestry-backed collections keeps its nodes in blocks of odd sizes 1, 3, 5, The scheme is chosen for its arithmetic: because the slots before block r number exactly r^2 , a handle resolves to a block and an offset by an integer square root and a subtraction, and wasted capacity after M published nodes is bounded by $O(\sqrt{M})$. The price is paid all at once. When a block fills, the next one — about $2\sqrt{M}$ slots — is allocated in a single step, so adding a leaf is $O(1)$ *amortized* and one unlucky call pays $\Theta(\sqrt{M})$.

The Haskell port has no arena. A node there is an ordinary heap object that is its own handle, so its leaf add is $O(1)$ worst case. Under the parity campaign’s charter — a stronger guarantee anywhere becomes the target everywhere, *where possible at all* — that made the spike a candidate for removal rather than a fact of life, and the question was whether the other eight workspaces could be brought up to it.

They could. The mechanism is *real-time two-table migration*: keep the old and the new backing tables live simultaneously and migrate a constant number of slots on every operation, so that no single call ever performs a large copy. It was analyzed, found implementable in all eight arena workspaces, costed — permanent extra complexity in the hottest path of eight implementations, to shave off an allocation the memory allocator performs anyway — and declined.

One correction is worth recording, because it is what made the analysis honest. The first mechanism written into the work order was the obvious one: pre-initialize the next block incrementally, a few slots per call. That cannot deliver the bound in a garbage-collected language, because allocating an array *zeroes* it, which is $\Theta(\text{block})$ work inside one call; growing a list by appends instead merely

relocates the spike into the allocator’s own geometric resizes. The proposal was corrected before anyone implemented it, and only then was there a real alternative to decline.

What parity demanded afterwards was uniform documentation rather than uniform code. All eight arena workspaces now carry the identical two-part statement — $O(1)$ amortized, with a block-boundary call paying $\Theta(\sqrt{M})$ for the next odd block — and Haskell’s module documentation marks its $O(1)$ worst case as *exceeding* the shared profile rather than quietly diverging from it. The reason that would have to change is a consumer with a latency bound on a single append, at which point the mechanism is already written down and costed.

C.5 Regressions taken in the open

The flat array’s constant-time rank-select, in OCaml. The OCaml workspace began as a deliberately simplified checkpoint port: correct public surface, simplest possible storage underneath, weaker bounds honestly documented. Six of its structures were still those checkpoints when the parity campaign reached them, and several stored each version as a flat immutable array — in key order for the sorted families. Such an array is a wonderful *read-only* structure. Selecting by rank is direct indexing, $O(1)$; the extremes are its two ends; binary search gives $O(\log n)$ lookup. Its catastrophe is writing: an immutable array cannot be edited in place, so every insertion or removal copies all n slots. That is $\Theta(n)$ per write, $\Theta(n^2)$ to build a collection by repeated insertion, and $\Theta(n \cdot m)$ set algebra. The placeholders were therefore simultaneously *better* than the reference at rank-select and catastrophically worse at everything that writes.

Fixing the writes forces a trade, and the trade has no clever way out: no known persistent structure supports both $O(\log n)$ writes and $O(1)$ rank-select. Constant-time select effectively requires array-like contiguity — an element’s position must be computable without traversal — and persistent updates to a contiguous representation are exactly the $\Theta(n)$ copies being eliminated. A tree representation makes both operations logarithmic. One cannot keep the array’s select and gain the tree’s write in the same persistent value.

The ruling was to adopt the reference profile: $O(\log n)$ writes, $O(\log n)$ rank-select, and $O(1)$ extremes, which a finger tree keeps through its digits, so the most common fast read is not the one that is lost. For the sorted map, concretely: writes went from $\Theta(n)$ to $O(\log n)$, bulk construction from $\Theta(n^2)$ to $O(n \log n)$,

union from $\Theta(n \cdot m)$ to $O(m \log(n + m))$, extremes stayed at $O(1)$, and n th went from $O(1)$ to $O(\log n)$ — the single loss.

The discipline is in the last step. The regression was not absorbed quietly: it is stated in every affected interface file — the sorted set, bag, and map, the priority search queue, the range-update sequence’s point index, the insertion-ordered family, and the checkpoint-differential map that sits on top of them — so that nobody reading the OCaml documentation can mistake it for an accident, and nobody “fixes” it back. The reason that would have to change is a consumer that selects by rank constantly and writes almost never, which is not a persistent collection at all but a frozen read tier, and is recorded above as its own gated project.

Further Reading

The papers behind the structures, roughly in the order the book meets them.

Topic	Source
Persistence, formally	Driscoll, Sarnak, Sleator & Tarjan, “Making Data Structures Persistent”, <i>JCSS</i> , 1989.
The functional canon	Okasaki, <i>Purely Functional Data Structures</i> , Cambridge University Press, 1998.
Hash array mapped tries CHAMP	Bagwell, “Ideal Hash Trees”, EPFL technical report, 2001. Steindorfer & Vinju, “Optimizing Hash-Array Mapped Tries for Fast and Lean Immutable JVM Collections”, <i>OOPSLA</i> , 2015.
Mergeable integer maps	Okasaki & Gill, “Fast Mergeable Integer Maps”, <i>ML Workshop</i> , 1998.
Concurrent hash tries	Prokopec, Bronson, Bagwell & Odersky, “Concurrent Tries with Efficient Non-Blocking Snapshots”, <i>PPoPP</i> , 2012.
Finger trees	Hinze & Paterson, “Finger Trees: A Simple General-Purpose Data Structure”, <i>JFP</i> , 2006.
RRB trees	Bagwell & Rompf, “RRB-Trees: Efficient Immutable Vectors”, EPFL technical report, 2011; Stucki, Rompf, Ureche & Bagwell, “RRB Vector: A Practical General Purpose Immutable Sequence”, <i>ICFP</i> , 2015.
Zip trees	Tarjan, Levy & Timmel, “Zip Trees”, 2018/2019.
Zip-zip trees	Gila, Goodrich & Tarjan, “Zip-Zip Trees: Making Zip Trees More Balanced, Biased, Compact, or Persistent”, 2023.
Worst-case meldable heaps	Brodal & Okasaki, “Optimal Purely Functional Priority Queues”, <i>JFP</i> , 1996.

Strict Fibonacci heaps	Brodal, Lagogiannis & Tarjan, <i>STOC</i> , 2012. (<i>Recorded rejection.</i>)
Hollow heaps	Hansen, Kaplan, Tarjan & Zwick, <i>ICALP</i> , 2015. (<i>Recorded rejection.</i>)
Priority search queues	Hinze, “A Simple Implementation Technique for Priority Search Queues”, <i>ICFP</i> , 2001.
Sliding-window aggregation	Tangwongsan, Hirzel & Schneider, <i>DEBS</i> , 2017; and the <i>VLDB Journal</i> , 2021, article introducing the Lite representation.
Merkle search trees	Auvolat & Taïani, “Merkle Search Trees: Efficient State-Based CRDTs in Open Networks”, <i>SRDS</i> , 2019.
Real-time dequeues	Kaplan & Tarjan, “Purely Functional, Real-Time Deques with Catenation”, <i>JACM</i> , 1999. (<i>Recorded rejection.</i>)
Order maintenance	Dietz & Sleator, 1987; Bender et al., “Two Simplified Algorithms for Maintaining Order in a List”, 2002. (<i>Postponed.</i>)
Adaptive radix trees	Leis, Kemper & Neumann, “The Adaptive Radix Tree”, <i>ICDE</i> , 2013. (<i>Postponed.</i>)

Within the repository itself, the authoritative documents are the *data-structure catalog*, the *semantic contracts reference*, the *frontier* and *derived structure catalogs*, and each workspace’s own API specification, header, and validation guide.

Index

- action policy, 140
- adversary argument, 129
- Alstrup and Holm, 105
- amortization
 - under branching, 5
 - under persistence, 10, 122
- AncestralSliceQueue, 106
- anchored empty, 107
- applicative random-access stack, 103
- arena, incremental ancestor, 187

- bijection, 37
- BilateralAncestralDeque, 108
- block, 159
- Brodal–Okasaki heap, 94
- BrodalOkasakiHeap, 94
- bulk builder, 50

- canonicalization, 127
- CanonicalSortedSet, 85
- CHAMP
 - bitmap split, 28
 - canonical deletion, 28
- change index, 126
- checkpoint, 125
- checkpoint port, 7
- collision bucket, 30
- complexity parity, 122, 137
- composition direction, 141
- composition-first families, 35
- ConcurrentHashTrie, 45
- content addressing, 156
- contextual rank, 116
- ContextualRankSequence, 115
- counting probe, 16

- Ctrie
 - generation stamping, 46
- cursor, 165
 - focused representation, 167
 - gap semantics, 165
 - snapshot-plus-gap, 168

- DabaLite, 77
- defunctionalization, 14
- delta, 125
- DIET, 133
- digit, 57
- directed graph, 39
- dirty run, 131

- editing session, 50
- effect table, 117

- failure atomicity, 21
- finger tree, 57
- FingerTree, 57
- FingerTreeDeque, 57
- frozen collection tier, 52

- GCAS, 46

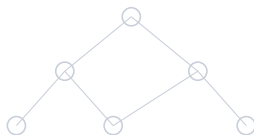
- HAMT, 28
- hash trie, *see* CHAMP

- IIncrementalAncestorArena, 103
- incremental ancestor arena, 102
- insertion order, 89
- interval tree, 97
- IntervalTree, 97

- join tree, 15

- jump link, 104
- laziness, 9, 122
- lazy propagation, 72
- level ancestor, 102
- map patch, 38
- measure, 61
 - guided descent, 118
- MeasuredRope, 69
- memoization, 9
- Merkle search tree
 - layers, 156
- MerkleSearchTree, 155
- monoid
 - associativity, 61, 117
- multimap, 38
- multiset, 36
- Myers, Eugene, 103
- MyersIncrementalAncestorArena, 103
- NaN, 136
- no-op identity, 21
- odd-block store, 104
- OrderClamp, 143
- Ordered family (insertion-ordered), 89
- ordering contracts, 22
- orientation, 75
- output-optimal, 128
- owner-token transient, 51
- pack, 160
- path compression, 149
- path copying, 3
- Patricia trie
 - maps and sets, 41
 - path compression, 42
- pending tag, 72
- PersistentAncestralConnectionForest, 147
- PersistentBiMap, 36
- PersistentChunkedBitSet, 40
- PersistentDeltaMap, 126
- PersistentDirectedGraph, 39
- PersistentHashBag, 35
- PersistentHashMap, 27
- PersistentHashMultimap, 37
- PersistentHashSet, 27
- PersistentIndexedMap, 39
- PersistentIntervalMap, 97
- PersistentIntMap, 41
- PersistentIntSet, 41
- PersistentMapPatch, 38
- PersistentMonotoneActionHeap, 139
- PersistentOrderedMap, 89
- PersistentOrderedMultimap, 89
- PersistentOrderedSet, 89
- PersistentRelation, 37
- PersistentRunDeltaVector, 131
- policy, 20
- population count, 40
- priority queue, 93
- priority search queue, 95
- PriorityQueue, 93
- PrioritySearchQueue, 95
- proof, 161
- radix trie, 66
- RangeUpdateSequence, 71
- rank policy, 86
- RDCSS, 46
- reference identity, 135
- reference pruning, 29
- representative
 - first-representative rule, 21
- research-derived collections, 101
- ReversibleDeque, 75
- Rope, 69
- rope, 69
- RRB vector
 - regular branch, 66
 - relaxed branch, 66

- RrbVector, 65
- secondary index, 39
- single-descent factories, 31
- sliding window, 78
- sorted collections, 83
- SortedBag, 83
- SortedDictionary, 83
- SortedSet, 83
- structural merge, 43
- structural sharing, 4
- suspension
 - memoized, 9
- synchronization, 161
- tag algebra, 73
- text rope, 70
- TextRope, 69
- three-way merge, 162
- tomb node, 46
- transition monoid, 117
- two-table migration, 187
- version DAG, 4
- zip tree, 85
- zip-zip tree, 86
- ZipTreeRankPolicy, 85



THE DURABLE7 FIELD GUIDE

Every operation returns a new version
and leaves the old one untouched.